

第2章　图搜索与问题求解

2.1　状态图问题求解

2.2　状态图搜索

2.3　与或图问题求解

2.4　与或图搜索

2.5　博弈树搜索

2.6　蒙特卡洛树搜索



搜索技术

所谓搜索，就是为了达到某一“目标”，而连续地进行推理的过程。搜索技术就是对推理进行引导和控制的技术，它也是人工智能的基本技术之一。 搜索技术也是一种规划技术。因为对于有些问题，其解就是由搜索而得到的“路径”。

事实上，许多智能活动的过程，甚至所有智能活动的过程，都可看作或抽象为一个“问题求解”过程。而所谓“问题求解”过程，实质上就是在显式的或隐式的状态空间图中进行搜索的过程，即所谓图搜索。

2.1 状态图问题求解

2.1.1 状态图

我们通过例子引入状态图的概念。

例1 走迷宫是人们熟悉的一种游戏，如图2–1就是一个迷宫。

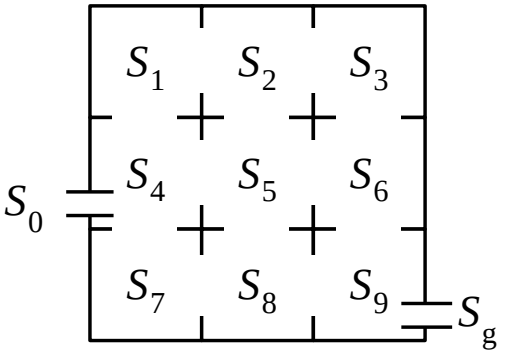


图2-1 迷宫游戏

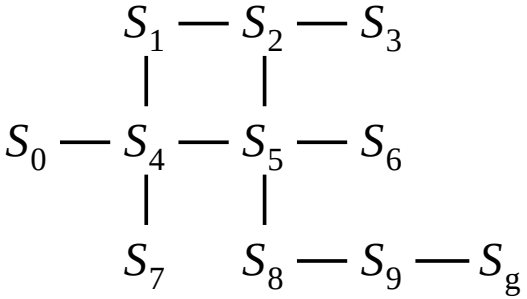


图2-2 迷宫的有向图表示

如果我们把该迷宫的每一个格子以及入口和出口都作为节点，把通道作为边，则该迷宫可以由一个有向图表示(如图2-2)。那么，走迷宫其实就是从该有向图的初始节点(入口)出发，寻找目标节点(出口)的问题，或者是寻找通向目标节点(出口)的路径的问题。

例2 在一个3×3的方格棋盘上放置着1,2,3,4,5,6,7,8八个数码，每个数码占一格，且有一个空格。这些数码可在棋盘上移动，其移动规则是：与空格相邻的数码方可移入空格。

现在的问题是：对于指定的初始棋局和目标棋局(如图2–3所示)，给出数码的移动序列。该问题称为八数码难题或重排九宫问题。

2	8	3
1		4
7	6	5

初始棋局

8	1	3
2		4
7	6	5

目标棋局

图2-3　八数码问题

分析：如果把一个棋局作为一个节点，相邻的节点就可以通过移动数码，一个一个地产生出来。这样所有节点就可由它们的相邻关系连成一个有向图。图中的一条边就对应一次数码移动，它代表一个移动规则或者移动规则的一次执行。于是，八数码问题也就成了在该有向图中寻找目标节点，或找一条从初始节点到目标节点的路径问题。

以上两个问题虽然内容不同，但抽象地来看，它们都是在某个有向图中寻找目标或路径的问题。我们把这种描述问题的有向图称为状态空间图，简称状态图。其中图中的节点代表问题中的一种格局，一般称为问题的一个状态；边可以是某种操作、规则、变换、算子、通道或关系等。



在状态图中，从初始节点到目标节点的一条路径，或者所找的目标节点，就是相应问题的一个解。

状态图实际上是一类问题的抽象表示。事实上，有许多智力问题(如梵塔问题、旅行商问题、八皇后问题、农夫过河问题等)和实际问题（如路径规划、定理证明、演绎推理、机器人行动规划等）都可以归结为在某一状态图中寻找目标或路径的问题。因此，研究状态图搜索具有普遍意义。

2.1.2 问题的状态图表示— 状态空间表示法

1. 状态(state)

- 状态就是问题在任一确定时刻的状况。它表征了问题特征和结构等，描述了问题求解过程中任一时刻状况的数据结构。
- 状态是为描述某类不同事物间的差别而引入的一组最少变量s_0，s_1，…，s_n的有序集合,其矢量形式为：$S=[s_0, s_1, \ldots, s_n]^T$，式中每个元素$s_i$ $(i=0,1,...)$为集合的分量称为***状态变量***。
- 状态一般用一组数据表示。在程序中用字符、数字、记录、数组、结构、对象等表示。

2. 算符（Operator）　表示对状态的操作

能使问题状态改变的某种操作、规则、行为、变换、关系、函数、算子、过程等等，也叫状态转换规则，有时也称为操作。问题的状态也只能经定义在其上的这种操作而改变。操作描述了状态之间的关系，在状态图中表示为边。在程序中状态转换规则可用数据对、条件语句、规则、函数、过程等表示。

3. 状态空间（State space）　　状态图表示

- 一个问题的全体状态及其关系，就构成一个状态空间，一般是一个赋值有向图，称为**状态空间图（状态图）**。

一个问题的状态图用一个三元组表示

〈S, F, G〉

其中S是问题的初始状态集合，

F是问题的状态转换规则集合，

G是问题的目标状态集合。

- 状态空间的一个解路径是一个有限的操作算子序列，它使初始状态转化为目标状态：S_0-O_1→S_1-O_2→... S_k-O_k→G
- 在状态空间表示法中,问题求解过程转化为在图中寻找从初始状态出发到达目标状态的路径问题,也就是寻找操作序列的问题。

状态空间法问题求解举例

例3：修道士和野人问题(MC问题)

问题描述：设N个传教士(Missionary)带领N个野人(Cannibal)划船渡河，为了安全起见，需遵循两个约束：

- 船上的人数不得超过载重限制K人；
- 任何时候两岸和船上的野人数不得超过传教士。

现考虑N=3,K=2的简单情况。



初始状态



目标状态

- 1.首先描述问题的状态

采用3元组来描述即 S=(m, c, b)，其中m是左岸传教士个数，c是左岸的野人数，b代表船是否在左岸，$m+c\leq 2$, $m\geq c$。

- 2.明确初始状态和目标状态。

左岸的状态(*m*,*c*,*b*)可以决定右岸的状态，所以整个问题的状态就可统一用左岸的状态来描述。设初始状态下传教士、野人和船都在左岸，则问题求解任务可描述为：(3,3,1) → (0,0,0)

- 状态空间可能的状态总数4×4×2=32，但由于要遵守安全约束，有12个状态是不合法的，如(1,0,1), (1,2,1), (2,3,1)等。包括S4,S6,S7,S8,S9,S11,S20,S22,S23,S24,S25,S27。鉴于存在不合法的状态，还会导致4个合法状态不可达，如(0,0,1), (0,3,0)等, 包括S3,S15,S16,S28。所以这个问题中总共只有16个可达的合法状态。

状态	m,c,b	状态	m,c,b	状态	m,c,b	状态	m,c,b
S0	3,3,1	S8	1,3,1	S16	3,3,0	S24	1,3,0
S1	3,2,1	S9	1,2,1	S17	3,2,0	S25	1,2,0
S2	3,1,1	S10	1,1,1	S18	3,1,0	S26	1,1,0
S3	3,0,1	S11	1,0,1	S19	3,0,0	S27	1,0,0
S4	2,3,1	S12	0,3,1	S20	2,3,0	S28	0,3,0
S5	2,2,1	S13	0,2,1	S21	2,2,0	S29	0,2,0
S6	2,1,1	S14	0,1,1	S22	2,1,0	S30	0,1,0
S7	2,0,1	S15	0,0,1	S23	2,0,0	S31	0,0,0

表1 修道士和野人问题的全部状态

• 3.确定算子集合。

本题中称每次过河的一种安排为一个操作或算子。操作算子可以定义两类：*L*(*m*,*c*)和*R*(*m*,*c*)，分别表示*m*个传教士和*c*个野人将船从左岸划到右岸和相反方向的操作。

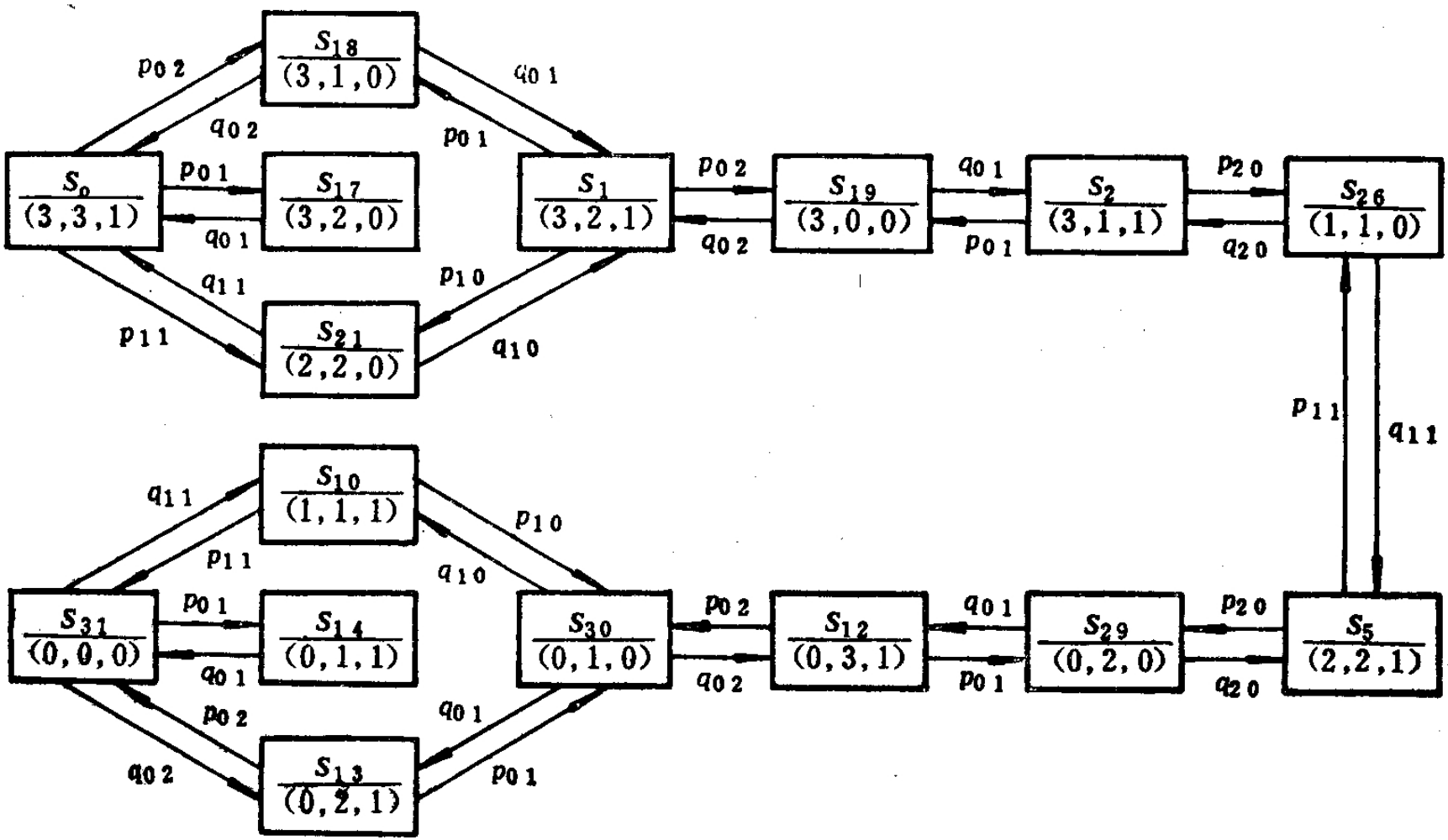
由于*m*和*c*取值的可能组合只有5个：10, 20, 11, 01, 02，因此总共有10个操作算子（如下图），操作集为：

$F=\{P_{01}，P_{10}，P_{11}，P_{02}，P_{20}，Q_{01}，Q_{10}，Q_{11}，Q_{02}，Q_{20}\}$

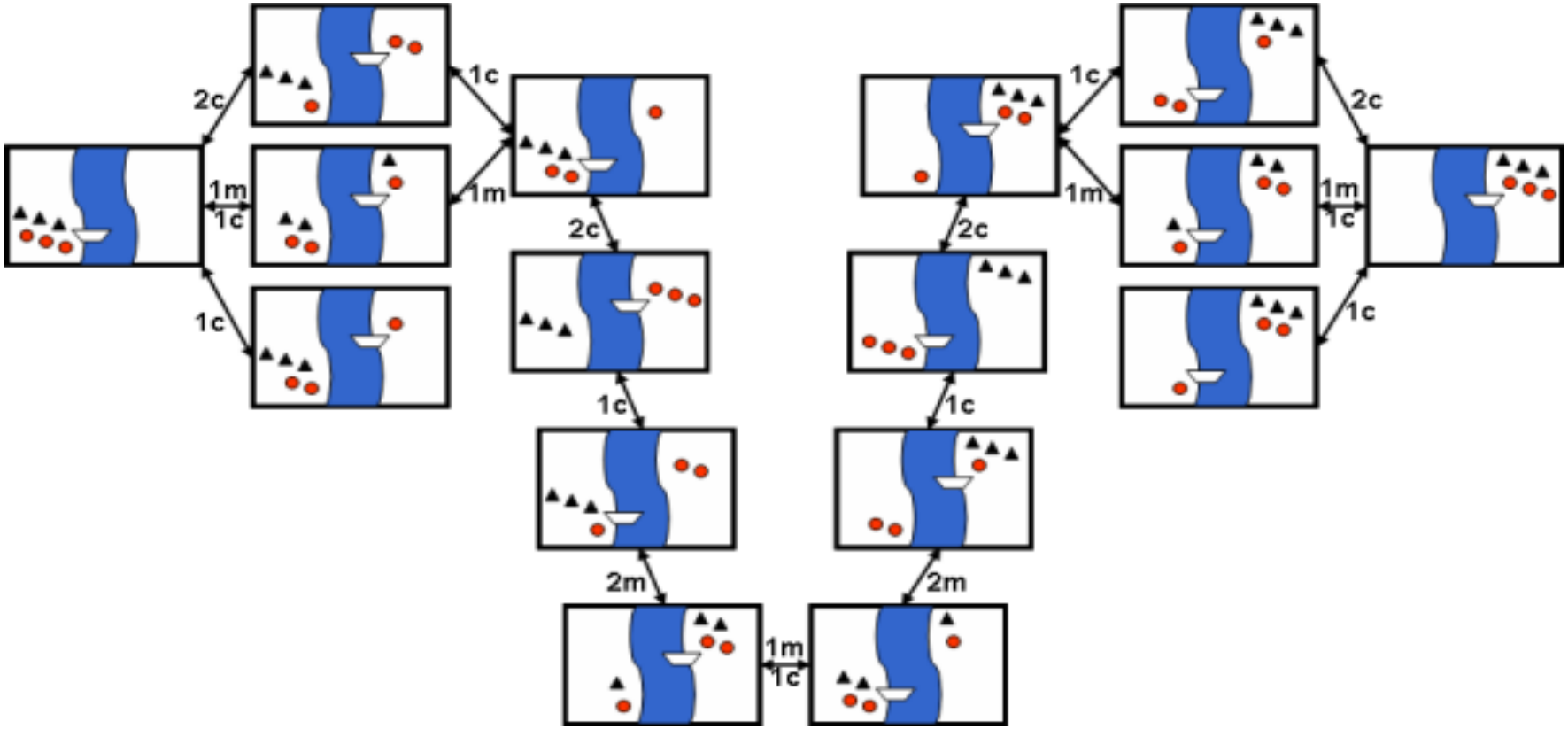
• 4. 画出状态空间图如下。

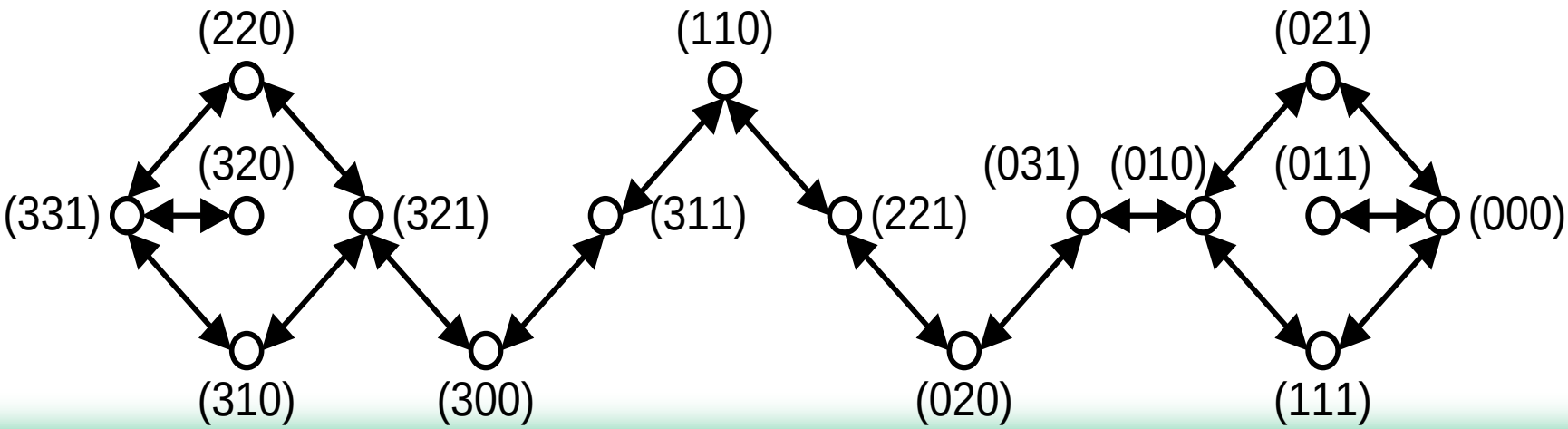
• 5.指出状态图中从初始状态到目标状态的任一路径（操作序列）就是问题的一个解。

由于划船操作是可逆的，所以节点间的弧是双向弧。



修道士野人问题的状态空间图


2c
1c
1m
1c
1c
1m
2c
1c
2m
1m
1c
1c
1m
2c
1c
2c
1m
1c
2c
1c
1c
2m
1m
1c


(220)
(320)
(331)
(321)
(310)
(300)
(110)
(311)
(221)
(020)
(031)
(010)
(021)
(011)
(000)
(111)

作业题: 三枚钱币问题。

设有三枚钱币，处在“反、正、反”状态，每次只允许翻动一枚钱币，问连翻三次后能否出现“正、正、正”“反、反、反”状态？

例4　梵塔问题。传说在印度的贝那勒斯的圣庙中，主神梵天做了一个由64个大小不同的金盘组成的“梵塔”，并把它穿在一个宝石杆上。另外，旁边再插上两个宝石杆。然后，他要求憎侣们把穿在第一个宝石杆上的64个金盘全部搬到第三个宝石杆上。搬动金盘的规则是：一次只能搬一个；不允许将较大的盘子放在较小的盘子上。于是，梵天预言：一旦64个盘子都搬到了3号杆上，世界将在一声霹雳中毁灭。这就是梵塔问题。

经计算，把64个盘子全部搬到3号杆上，需要穿插搬动盘子 $2^{64}-1=18\ 446\ 744\ 073\ 709\ 511\ 615$ 次。所以直接考虑原问题，将过于复杂。

我们仅考虑二阶梵塔（即只有两个金盘）问题。设有三根宝石杆，在1号杆上穿有A、B两个金盘，A小于B，A位于B的上面。要求把这两个金盘全部移到另一根杆上，而且规定每次只能移动一个盘子，任何时刻都不能使B位于A的上面。

设用二元组(S_A,S_B)表示问题的状态，S_A表示金盘A所在的杆号，S_B表示金盘B所在的杆号，全部可能的状态有9种：

(1,1),(1,2),(1,3)　　(2,1),(2,2),(2,3)　　(3,1),(3,2),(3,3)

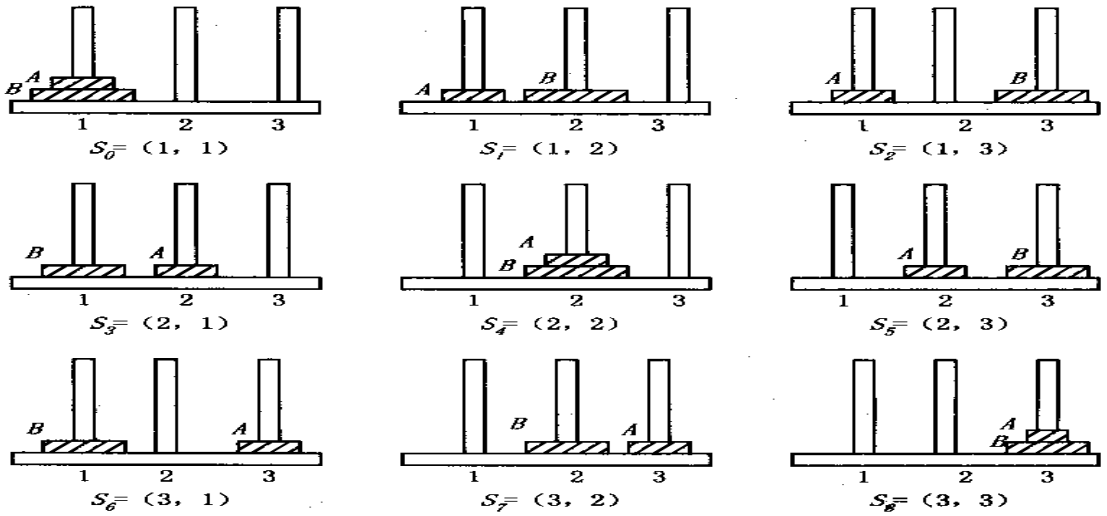
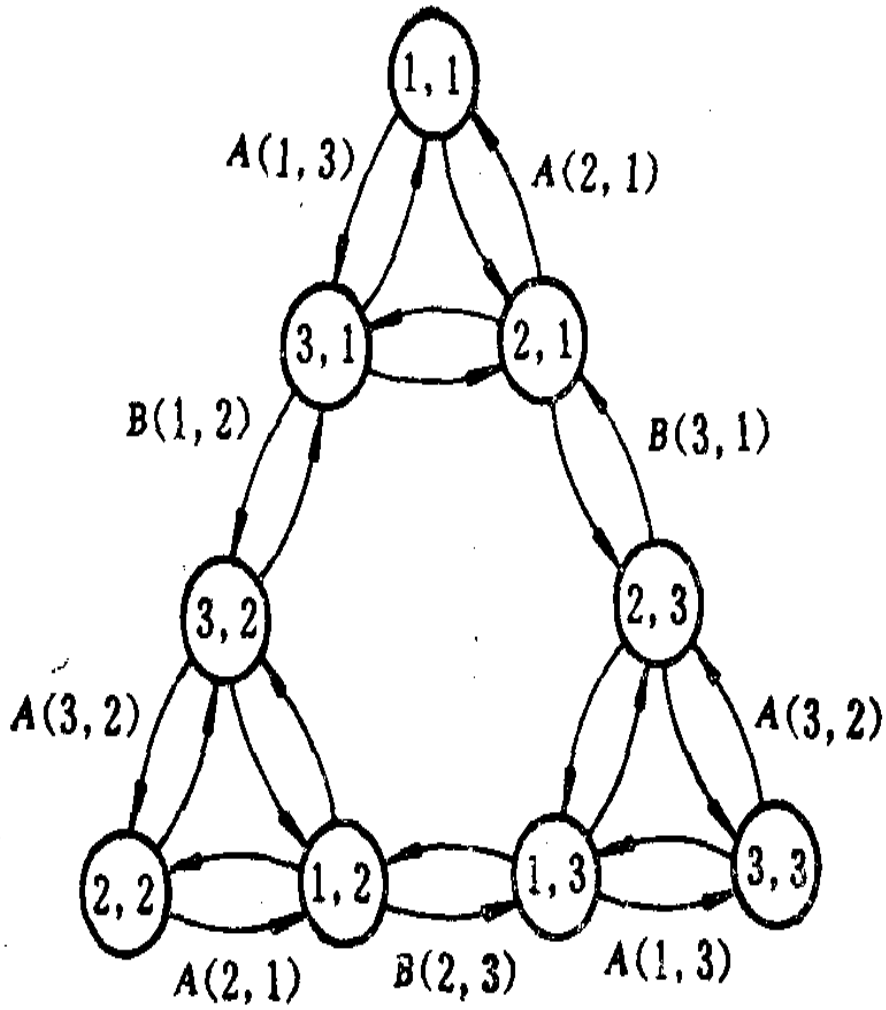


图　二阶梵塔的全部状态

这里的状态转换规则就是金盘的搬动规则，分别用A(*i*,*j*)及B(*i*,*j*)表示：A(*i*,*j*)表示把A盘从第i号杆移到第j号杆上；B(*i*,*j*)表示把B盘从第i号杆移到第j号杆上。因此共有12个操作，它们分别是：

A(1,2), A(1,3), A(2,1), A(2,3), A(3,1), A(3,2)

B(1,2), B(1,3), B(2,1), B(2,3), B(3,1), B(3,2)



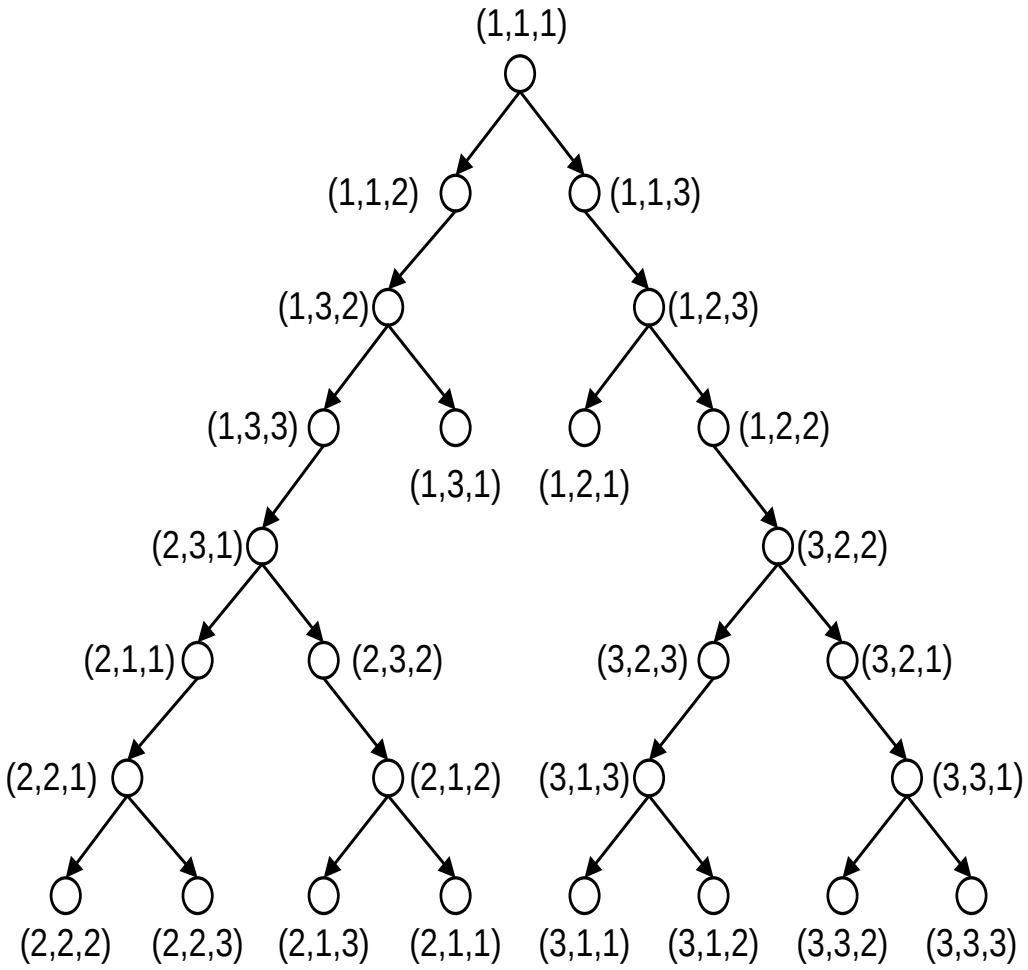
二阶梵塔状态空间图

例5：　三阶Hanoi塔问题

用状态(i, j, k)表示最大盘片C在第i根柱上，盘片B在第j根柱上，最小盘片A在第k根柱上。如果同一根柱上有两片以上的盘片，则假设较大的在下面。用$M(N, i, j)$表示操作算子，即把盘片N从第i根针移到第j根针上。可画出状态图如下图。

从图可见它的解更多，最短的路径长度是7。一般讲n阶梵塔问题的最佳解的长度是

2^n-1。



三阶 Hanoi塔问题状态图

例6 八数码难题的状态图表示。我们将棋局用向量A表示

X_1	X_2	X_3
X_6	X_0	X_4
X_7	X_6	X_5

A =(X0,X1,X2,X3,X4,X5,X6,X7,X8)

其中Xi为变量，Xi的值就是方格Xi内的数字。于是，向量 A 就是该问题的状态表达式。

数码的移动规则就是该问题的状态变换规则即操作。

经分析，该问题共有24条移码规则，可分为9组。

于是，八数码问题的状态图可表示为

$\{S_0\},\{r_1,r_2,\ldots,r_{24}\},\{S_g\}$）

0组规则:

r_1 $(X_0=0)\wedge(X_2=n)\rightarrow X_0=n\wedge X_2=0;$

r_2 $(X_0=0)\wedge(X_4=n)\rightarrow X_0=n\wedge X_4=0;$

r_3 $(X_0=0)\wedge(X_6=n)\rightarrow X_0=n\wedge X_6=0;$

r_4 $(X_0=0)\wedge(X_8=n)\rightarrow X_0=n\wedge X_8=0;$

1组规则:

r_5 $(X_1=0)\wedge(X_2=n)\rightarrow X_1=n\wedge X_2=0;$

r_6 $(X_1=0)\wedge(X_8=n)\rightarrow X_1=n\wedge X_8=0;$

2组规则:

$r_7(X_2=0)\wedge(X_1=n)\rightarrow X_2=n\wedge X_1=0;$

$r_8(X_2=0)\wedge(X_3=n)\rightarrow X_2=n\wedge X_3=0;$

$r_9(X_2=0)\wedge(X_0=n)\rightarrow X_2=n\wedge X_0=0;$ …

8组规则:

$r_{22}(X_8=0)\wedge(X_1=n)\rightarrow X_8=n\wedge X_1=0;$

$r_{23}(X8=0)\wedge(X_0=n)\rightarrow X_8=n\wedge X_0=0;$

$r_{24}(X8=0)\wedge(X_7=n)\rightarrow X_8=n\wedge X_7=0;$

当然，上述24条规则也可以简化为4条:即空格上移、下移、左移、右移。不过，这时状态(即棋局)就需要用矩阵来表示。

可以看出，这个状态图中仅给出了初始节点和目标节点并未给出其余节点。而其余节点需用状态转换规则来产生。类似于这样表示的状态图称为隐式状态图，或者说状态图的隐式表示。



2.2　状态图搜索

在状态图中寻找目标或路径的基本方法就是搜索。所谓搜索，就是从初始节点出发，沿着与之相连的边试探地前进，寻找目标节点的过程(也可以反向进行)。那么，当目标节点找到后，路径也就找到了。所以寻找目标和寻找路径是一致的。

搜索过程中经过的节点和边，按原图的连接关系，便会构成一个树型的有向图，这种树型有向图称为搜索树。随着搜索的进行，搜索树会不断地生长，直到当搜索树中出现目标节点时，搜索便停止。

由于搜索具有探索性，所以要提高搜索效率(尽快地找到目标节点)，或要找最佳路径(最佳解)就必须注意搜索策略。



1. 搜索策略

- 盲目搜索：　无“向导”　的搜索
- 启发式搜索：有“向导”的搜索，利用启发式信息引导搜索。

所谓启发信息，就是与问题有关的有利于尽快找到问题解的信息或知识。例如：格言“欲速则不达”，“学如逆水行舟不进则退”。启发式搜索往往会提高搜索效率，而且可能找到问题的最优解。

2. 搜索算法设计

由于搜索是为了寻找初始节点到目标节点的路径，所以在搜索过程中就得随时记录搜索轨迹。为此我们用一个称为CLOSED表的动态数据结构来专门记录考查过的节点。显然，对于树式搜索来说，CLOSED表中存储的正是一棵不断成长的搜索树；而对于线式搜索来说，CLOSED表中存储的是一条不断伸长的折线，它可能本身就是所求的路径。

另一方面，对于树式搜索来说，还得不断地把待考查的节点组织在一起，并做某种排列，以便控制搜索的方向和顺序。为此，我们采用一个称为OPEN表的动态数据结构，来专门登记当前待考查的节点。



2.2.3 穷举式搜索

为简单起见，下面我们先讨论树型结构的状态图搜索，并仅限于树式搜索。按搜索树生成方式的不同，树式穷举搜索又分为广度优先和深度优先两种搜索方式。这两种方式也是最基本的树式搜索策略，其他搜索策略都是建立在它们之上的。

1. 广度优先搜索

广度优先搜索就是始终先在同一级节点中考查，只有当同一级节点考查完之后，才考查下一级节点。或者说，是以初始节点为根节点，向下逐级扩展搜索树。所以，广度优先策略的搜索树是自顶向下一层一层逐渐生成的。又称宽度优先或横向搜索。

广度优先搜索算法

算法描述:

步1 把初始结点 S。放入 OPEN 表中

步2 若OPEN表为空，则搜索失败，问题无解。

步 3　取OPEN表中最前面的结点N放在 CLOSED表中，并冠以顺序编号 n;

步4 若目标结点 S_g＝n，则搜索成功，问题有解。

步5　若n无子结点，则转步2;

步6　扩展n，将其所有子结点配上指向n的返回指针后，依次放入OPEN表的后面，转步2。

1, G:=G0(G0=s), OPEN:=(s), CLOSED:=( )

2, LOOP: IF OPEN=( ) THEN EXIT (FAIL)

3, n:=FIRST(OPEN)

4, IF GOAL(n) THEN EXIT (SUCCESS)

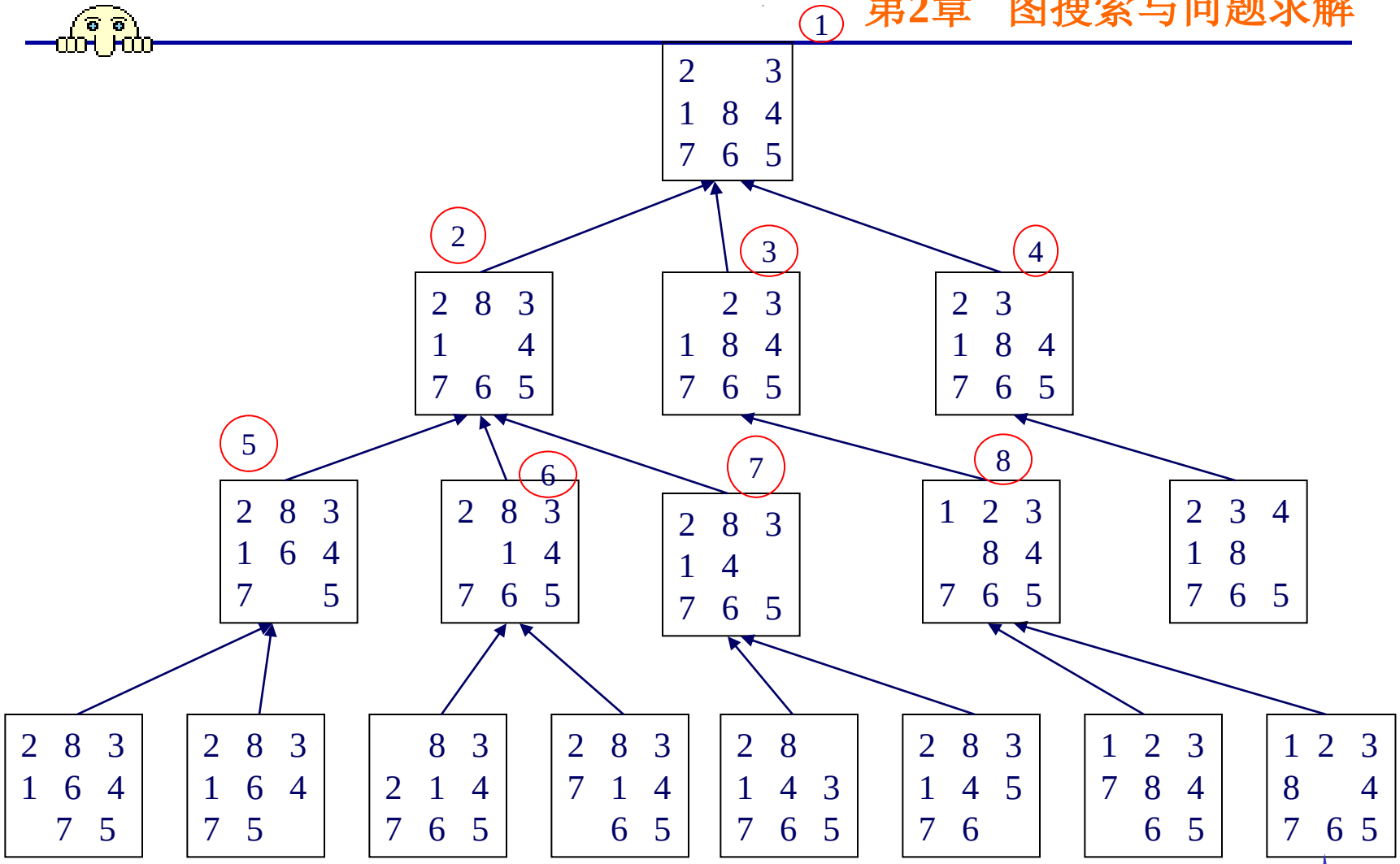
5, REMOVE(n, OPEN), ADD(n, CLOSED)

6, EXPAND(n) →{m_i}, G:=ADD(m_i, G)

7, IF 目标在{m_i}中 THEN EXIT(SUCCESS)

8, ADD(OPEN, m_j), 并标记m_j到n的指针

9, GO LOOP



例7 用广度优先搜索策略解八数码难题。设初始节点S_0和目标节点S_g分别如图2-3初始棋局和目标棋局所示，我们用广度优先搜索策略，则可得到如图2-5所示的搜索树。

搜索算法的常见评价指标

• **完备性**：当问题存在解时，算法是否能保证找到一个解，虽然这个解可能不是最优解。

定义1：给定一类可解问题的状态空间<S,F,G>和一个搜索过程A，称搜索过程A对问题类<S,F,G>是完备的，当且仅当对任一$S_0 \in S$，搜索过程A总可以发现一条从S_0到达$S_g \in G$ 的解(路径)。

•**最优性（可采纳性）**：是否保证找到的第一个解是最优解；

定义2：若A找到的总是最佳解，则称它是可采纳的，记作A*

•**时间复杂度**：找到一个搜索路径所需要的时间；

•**空间复杂度**：算法运行时所需的内存空间。

完备性和最优性刻画了算法找到解的能力及所求解的质量，时间和空间复杂度衡量算法的资源消耗。



完备性和最优性刻画了算法找到解的能力及所求解的质量，时间复杂度和空间复杂度衡量算法的资源消耗。在状态图中，时间复杂度将通过扩展的结点数量，空间复杂度将通过算法同时记录的结点数量来衡量。具体而言，搜索算法的复杂度与以下变量有关：

- 分支因子b：搜索树中每个结点最大的分支数目；
- 根结点到最浅的目标结点的路径长度（深度)d;
- 搜索树中路径的最大可能长度m;
- 状态空间的大小n(状态空间中状态的数量）



宽度优先搜索的性质

- 实现考虑：扩展当前节点后生成的子节点总是置于*OPEN*表的尾端，即*OPEN*表作为*FIFO*的队列使用，使搜索优先向横广方向发展。CLOSED表是一个顺序表，正被考察的节点在表中编号最大。
- 方法与问题无关，具有通用性；
- 适用的问题：需要找到最优解，解的位置比较浅，树的宽幅不大；
- 优点：能确保找到最短解路径。如果问题的解存在，用此搜索法一定能找到解(完备的)，且找到的解还是最优解(可采纳的)；
- 缺点是搜索效率低。



2.深度优先搜索

深度优先搜索就是在搜索树的每一层始终先只扩展一个子节点，不断地向纵深前进，直到不能再前进(到达叶子节点或受到深度限制)时，才从当前节点返回到上一级节点，沿另一方向又继续前进。这种方法的搜索树是从树根开始一枝一枝逐渐形成的。

步1 把初始节点S0放入OPEN表中；

步2 若OPEN表为空，则搜索失败，退出。

步3 取OPEN表中前面第一个节点N放入CLOSED表中，并冠以顺序编号n；

步4 若目标节点S_g=N，则搜索成功，结束。

步5 若N不可扩展，则转步2；

步6 扩展N，将其所有子节点配上指向N的返回指针依次放入OPEN表的首部，转步2。

- 可以看出，这里的OPEN表为一个堆栈，这是与广度优先算法的一个区别。

算法步骤

例8 对于八数码问题，应用深度优先搜索策略，可得如图2–6所示的搜索树。

深度优先搜索亦称为纵向搜索。由于一个有解的问题树可能含有无穷分枝，深度优先搜索如果误入无穷分枝(即深度无限)，则不可能找到目标节点。所以，深度优先搜索策略是不完备的。另外，应用此策略得到的解不一定是最佳解(最短路径)。

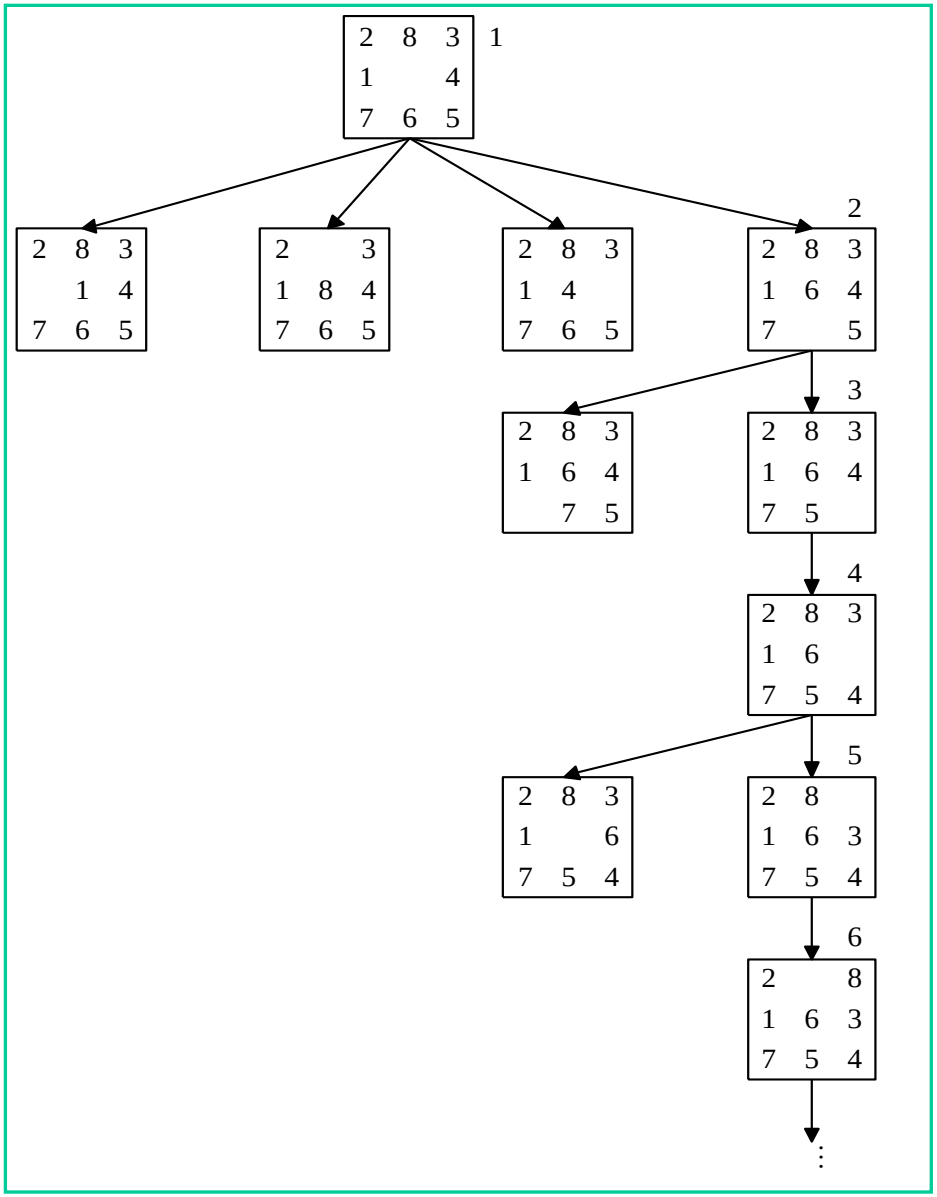


图2—6　八数码问题的深度优先搜索



深度优先搜索的性质

- 一般不能保证找到最优解
- 当深度限制不合理时，可能找不到解，可以将算法改为可变深度限制
- 最坏情况时，搜索空间等同于穷举
- 是一个通用的与问题无关的方法
- 特点：不完备的，不可采纳的，能很快地深入问题；
- 适用的问题：有很多解，但只要找到一个近似解就可以。



3.有界深度优先搜索

广度优先和深度优先是两种最基本的穷举搜索方法，在此基础上，根据需要再加上一定的限制条件，便可派生出许多特殊的搜索方法。例如有界深度优先搜索。

有界深度优先搜索就是给出了搜索树深度限制，当从初始节点出发沿某一分枝扩展到一限定深度时，就不能再继续向下扩展，而只能改变方向继续搜索。节点x的深度(即其位于搜索树的层数)通常用d(x)表示，则搜索算法如下：

有界深度优先搜索

步1 把S_0放入OPEN表中，置S_0的深度$d(S_0)=0$；

步2 若OPEN表为空，则失败，退出。

步3 取OPEN表中前面第一个节点N，放入CLOSED表中，并冠以顺序编号n；

步4 若目标节点S_g=N，则成功，结束。

步5 若N的深度$d(N)=d_m$(深度限制值)，或者若N无子节点，则转步2；

步6 扩展N，将其所有子节点N_i配上指向N的返回指针后依次放入OPEN表中前部，置$d(N_i)=d(N)+1$，转步2。

1, G:=G0(G0=s), OPEN:=(s), CLOSED:=( );

2, LOOP: IF OPEN=( ) THEN EXIT (FAIL);

3, n:=FIRST(OPEN);

4, IF GOAL(n) THEN EXIT (SUCCESS);

5, REMOVE(n, OPEN), ADD(n, CLOSED);

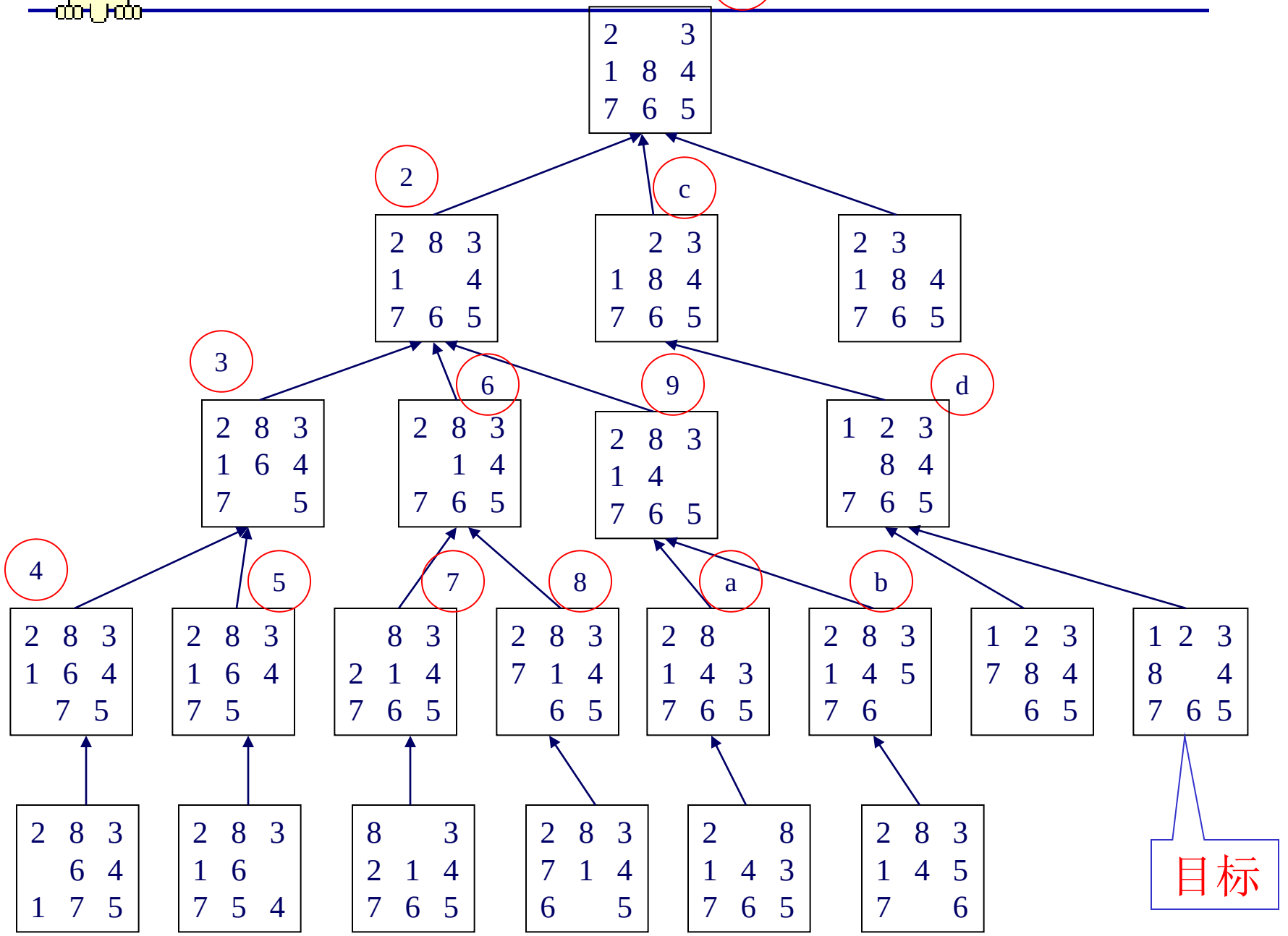
6, IF DEPTH(n)≥Dm GO LOOP;

7, EXPAND(n) →{m_i}, G:=ADD(mi, G);

8, IF 目标在{m_i}中 THEN EXIT(SUCCESS);

9, ADD (m_j, OPEN), 并标记m_j到n的指针;

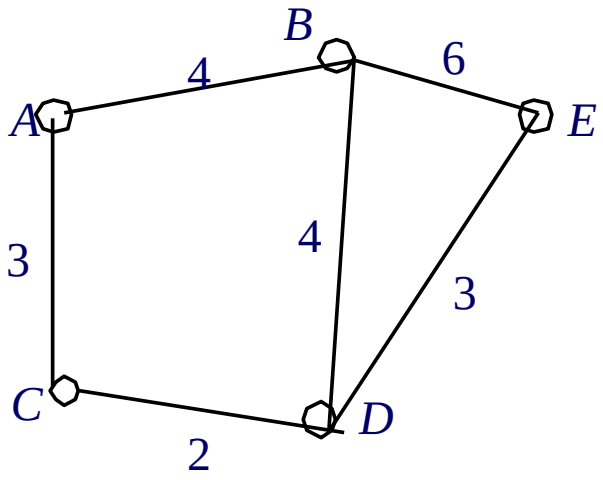
10, GO LOOP;


1
2
c
3
6
9
d
4
5
7
8
a
b
目标

2.2.4 加权状态图搜索

1. 加权状态图与代价树

例9　图2–8(a)是一个交通图，设A城是出发地，E城是目的地，边上的数字代表两城之间的交通费。试求从A到E最小费用的旅行路线。



可以看出，这个图与前面的状态图不同的地方是边上附有数值。它表示边的一种度量（此例中是交通费，当然也可以是距离）。一般地，称这种数值为权值，而把边上附有数值的状态图称之为加权状态图。

显然，加权状态图的搜索与权值有关，并且要用权值来导航。具体来讲，加权状态图的搜索算法，要在一般状态图搜索算法基础上再增加权值的计算与传播过程，并且要由权值来确定节点的扩展顺序。

为简单起见，我们先考虑树型的加权状态图—代价树的搜索。所谓代价，可以是两点之间的距离、交通费用或所需时间等等。通常用$g(x)$表示从初始节点S_0到节点x的代价，用$c(x_i,x_j)$表示父节点x_i到子节点x_j的代价，即边(x_i,x_j)的代价。从而有

$$g(x_J)=g(x_i)+c(x_i,x_j)$$

而 $g(S_0)=0$

也可以将加权状态图转换成代价树来搜索，其转换方法是，从初始节点起，先把每一个与初始节点相邻的节点作为该节点的子节点；然后对其他节点依次类推，但对其他节点x，不能将其父节点及祖先再作为x的子节点。

例如，把图2–8(a)所示的交通图转换成代价树如图2—8(b)所示。

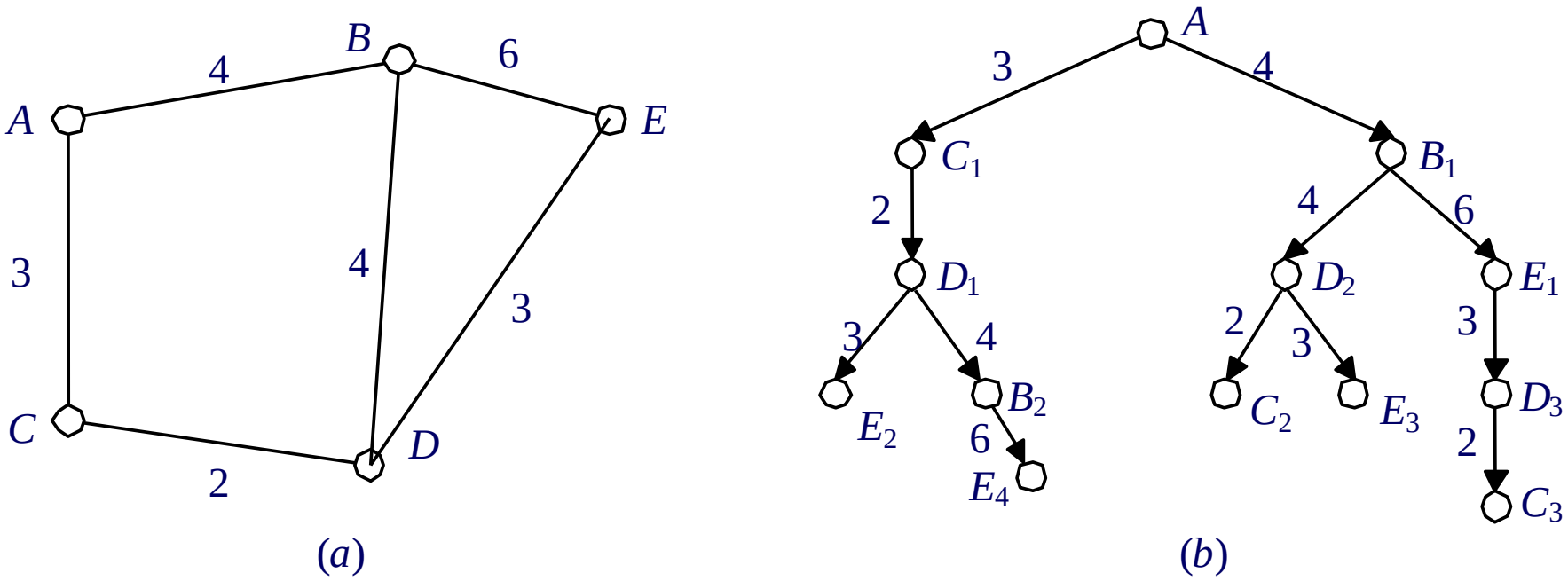


图2-8 交通图及其代价树

转换规则Q(n)：

结点n的子结点是网络图中与n有直接边相连的那些结点，但n在树中的所有先辈结点应该除外。



1.分支界限法(最小代价优先法)

其基本思想是：每次从OPEN表中选出g(x)值最小的节点进行考察，而不管这个节点在搜索树的什么位置上。

可以看出，这种搜索法与前面的广度优先法的区别仅是选取扩展节点的标准不同，一个是代价值g(x)（最小），一个是路径值d(x)（最小）。

2.最近择优法(瞎子爬山法)

代价树的深度优先搜索法俗称瞎子爬山法(Hill Climbing)，其基本思想可以概括为：

从结点n继续向前搜索的准则是选其有最小代价变Min（n，n'）的子结点n'。

犹如瞎子爬山一样，他无力瞻前顾后，纵观全局，只能用拐杖四下探测，择其最高者而登之。故名瞎子爬山法。这种方法实际同深度优先法类似，区别也仅是选取扩展节点的标准不同，一个是代价值g(x)（最小），一个是路径值d(x)（最小）。

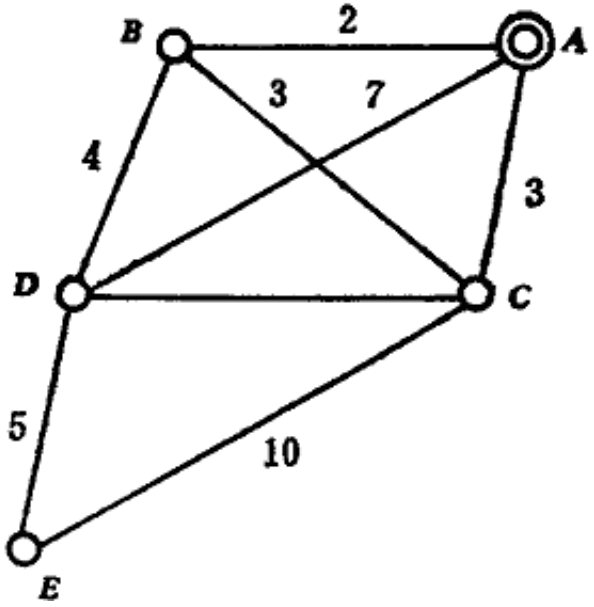
瞎子爬山法在许多简单情况下十分有效，特别是一维参数问题中的单极值情况（只有一个解存在）。但在多维参数问题中，“地形””往往很复杂，爬山法会遇到诸多困难，常见的有：

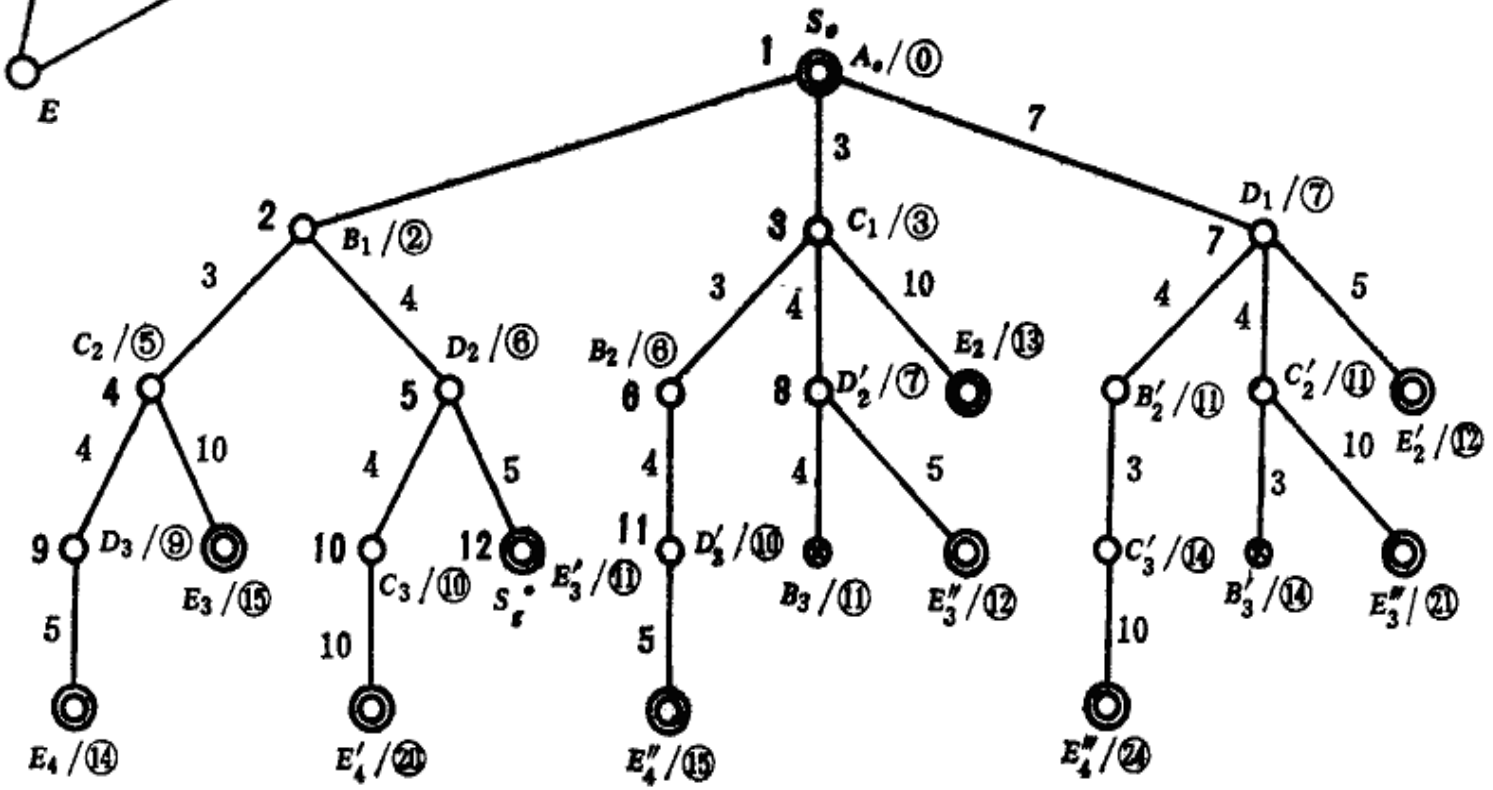
(1)小丘问题。如果问题存在多个解，类似多个山头并立，有一个最高峰（最佳解）和多个次高峰甚至小丘（非最佳解），我们的目标是要登上最高峰，但“瞎子”有可能登上一个小丘后就宣告胜利了。

(2)山脊问题。如果“瞎子”正站在一条从东北到西南走向的刀峰般的山脊上。他的标准试探法是用棍子东、西、南，北各测一点，结论是“各个方向都是下坡路，我已到达最高峰”，其实该点连一个小丘都不是。克服办法是适当增加测试点，即让每个结点有更多的分支。

(3)平地问题。一块平地把各个孤立的山峰分开，“瞎子正站在平地中央，无论他如何测试，都无法找到登高的线索，失去了继续前进的方向。这是最难处理的一个问题，较好的一个办法是适当增加“瞎子的视力”，使其看到较大的范围。但那已不是瞎子爬山法了，成了“近视眼爬山法”（见局部择优法）。最简单的办法是随机选一个方向继续搜索，但这要增大搜索的风险性（即不完备性）

例10


B
A
D
C
E
2
3
7
4
3
5
10


S_0
A_0/⓪
B_1/②
C_1/③
D_1/⑦
C_2/⑤
D_2/⑥
B_2/⑥
D_2'/⑦
E_2/⑬
B_2'/⑪
C_2'/⑪
E_2'/⑫
D_3/⑨
E_3/⑮
C_3/⑩
S_g
E_3'/⑪
D_3'/⑩
B_3/⑪
E_3''/⑫
C_3'/⑭
B_3'/⑭
E_3'''/㉑
E_4/⑭
E_4'/⑳
E_4''/⑮
E_4'''/㉔

解：从A开始，利用Q(n)可以生成一个代价树，$S_0=A_0$，S_g是图中所有的E，图中各边上的数字为边的代价，结点旁的带圈的数字为结点的代价。显然最佳解是（A_0,B_1,D_2,E'_3），它的代价是11。

- 如果用分支界限法求解，搜索过程是：

A_0-B_1-C_1-C_2-D_2-B_2-D_1-D'_2-D_3-C_3-D'_3-E'_3，

其解是(A, B_1, D_2, E'_3),代价为11，正好是最佳解。

- 如果用瞎子爬山法求解，其搜索过程是：

A_0-B_1-C_2-D_3-E_4，发现的是非最佳解，代价为14.

•如果用代价树的有界深度优先搜索法求解，规定g_m=10,其搜索过程是A_0-B_1-C_2-D_3-D_2-C_2-C_1-B_2-D'_3-D'_2-D_1（未发现解，令g_m=15再次搜索)-E'_2（得一解E'_2，代价为12，令g_m=12继续搜索更好的解）-C'_2-B'_2-E''_3(又得一解E''_3但代价也是12，继续搜索更好的解)-E'_3（又得一解E'_3，代价11），E'_3是最佳解，路径为（A_0-B_1-D_2-E'_3）。

现在我们用代价树搜索求解例9中给出的问题。我们用分支界限法得到的路径为

A→C→D→E

这是一条最小费用路径(费用为8)。



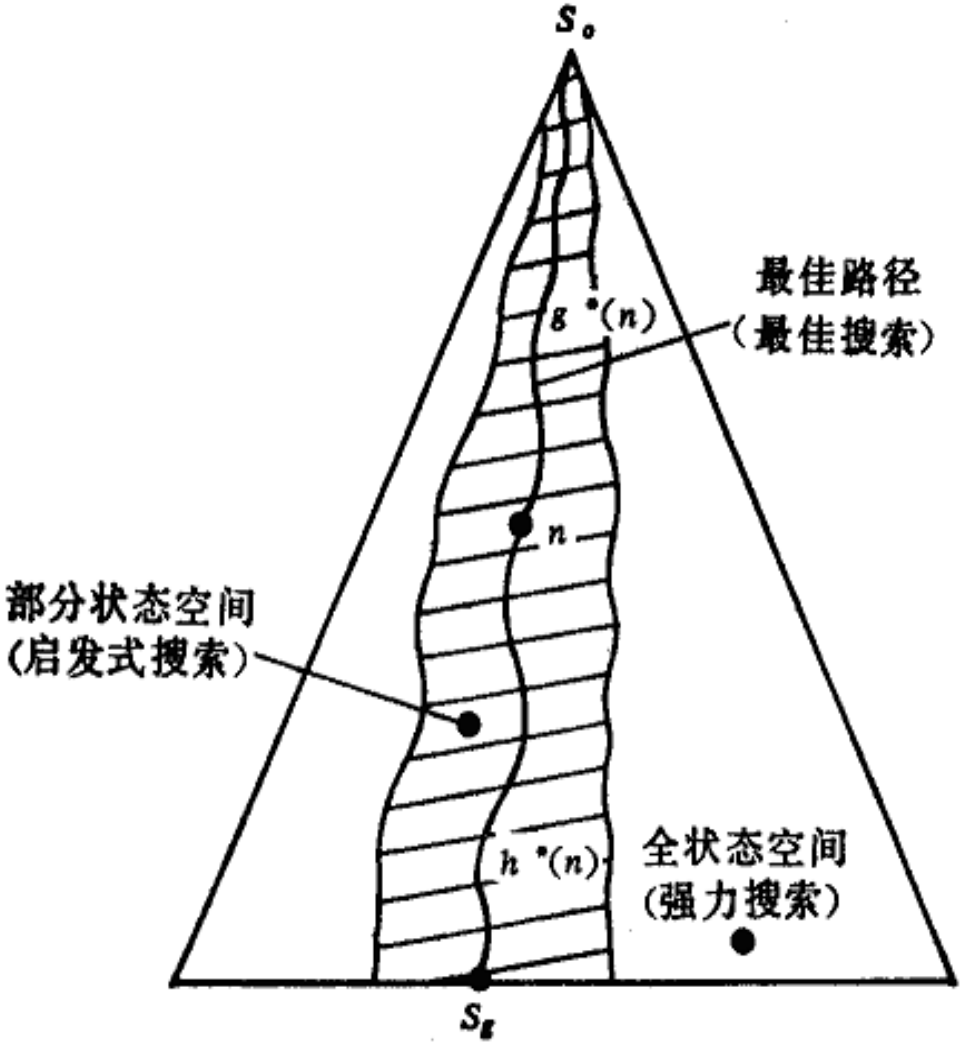
2.2.5 启发式搜索

1. 问题的提出

广度优先搜索和深度优先搜索都属于盲目型搜索算法，在选择下一个搜索节点的时候没有一个准则，去选择最优的。很多情况下需要穷举整个空间，因此只**适用于规模较小的搜索**。

从理论上讲，穷举搜索法似乎可以解决任何状态空间的搜索问题，但实践表明，穷举搜索只能解决一些状态空间很小的简单问题，而对于那些大状态空间问题，穷举搜索就不能胜任了。因为大空间问题，往往会导致“组合爆炸”。

如果已知一个可解问题的全部状态空间，我们就可以设计一个搜索算法来机械地求解这个问题。但是由于计算机的时、空限制以及认识方面的限制，在许多问题中只能使用部分状态空间，使得求解问题的过程中失去了关于该问题的许多有用信息。



弥补失去的有用信息的办法是启发式搜索。

- 用增加控制性知识的方法来弥补由于减少搜索空间造成的信息丢失。方法是利用关于该问题解的特性的先验信息，把它设计成启发函数h(n)，用它来指导搜索朝着有解的方向前进。
- 一般来讲，h(n)携带的启发信息越多，搜索成功的可能性越大，搜索效率越高。

如果在f(x)中包含有关于被解问题的某些先验知识，如解的特性、解的分布规律等，那么搜索过程就能有选择地朝目标结点有效地推进，就称该搜索过程A为关于问题类<S,F,G>的启发式搜索过程。启发式搜索就是利用启发性信息进行制导的搜索。

非启发式搜索过程仅含有一般的控制性知识，搜索没有针对性，所以它是一种基于全部状态空间的搜索。从这种意义上讲，人们又称其为强力（或盲目、穷举）搜索过程。

2. 启发性信息

启发性信息就是有利于尽快找到问题之解的信息，它反映在启发函数中。

按其用途划分，启发性信息一般可分为以下三类：

(1)用于扩展节点的选择，即用于决定应先扩展哪一个节点，以免盲目扩展。

(2)用于生成节点的选择，即用于决定应生成哪些后续节点，以免盲目地生成过多无用节点。

(3)用于删除节点的选择，即用于决定应删除哪些无用节点，以免造成进一步的时空浪费。

3. 启发函数

在启发式搜索中，通常用所谓启发函数来表示启发性信息。启发函数是用来估计搜索树上节点x与目标节点Sg接近程度的一种函数，通常记为h(x)。

如何定义一个启发函数呢？启发函数并无固定的模式，需要具体问题具体分析。通常可以参考的思路有：一个节点到目标节点的某种距离或差异的度量；一个节点x处在最佳路径上的概率，或者根据经验的主观打分，或是x结点和目标结点之间的距离，或者是x 格局的得分等等。

例如，对于八数码难题，用h(x)就可表示节点x的数码格局同目标节点相比，数码不同的位置个数。



4.启发式搜索算法

启发式搜索就是在状态空间中对每一个搜索位置进行评估（估计），选择最好的位置，再从这个位置进行搜索直到目标。这样可以省略大量无谓的搜索路径。在启发式搜索中，对位置的估计尤其重要，采用不同的估计函数会有不同的效果。

启发式搜索要用启发函数来导航，其搜索算法就要在状态图一般搜索算法基础上再增加启发函数值的计算与传播过程，并且由启发函数值来确定节点的扩展顺序。为简单起见，下面我们仅给出树型图的树式搜索的两种策略。



1) 局部择优搜索

局部优先搜索是在搜索过程中选择最佳节点后舍弃其他的兄弟节点，其很明显的缺陷是可能舍弃了最好的节点。

局部择优搜索的思想是：扩展节点n后仅对n的子节点按启发函数值大小以升序排序，再将它们依次放入OPEN表的首部。



2) 全局择优搜索

全局择优搜索亦称为最好优先搜索法，它没有舍弃节点，有效地防止了最佳节点的丢失。

该方法的基本思想是：

在OPEN表中保留所有已生成而未考察的节点，并用启发函数h(x)对它们全部进行估价，从中选出最优节点进行扩展，而不管这个节点出现在搜索树的什么地方。

全局择优搜索算法如下：

步1 把初始节点S_0放入OPEN表中，计算$h(S_o)$；

步2 若OPEN表为空，则搜索失败，退出。

步3 移出OPEN表中第一个节点n放入CLOSED表中，并冠以序号n；

步4 若目标节点S_g=n，则搜索成功，结束。

步5 若n不可扩展，则转步2；

步6 扩展n，计算每个子节点x的函数值h(x)，并将所有子节点配以指向n的返回指针后放入OPEN表中，再对OPEN表中的所有子节点按其函数值大小以升序排序，转步2。

例11　用全局择优搜索法解八数码难题。初始棋局和目标棋局同例3。

解 设启发函数h(x)为节点x的格局与目标格局相比数码不同的位置个数。以这个函数制导的搜索树如图2-9所示。图中节点旁的数字就是该节点的估价值。由图可见此八数码问题的解为：S_0，S_1，S_2，S_3，Sg

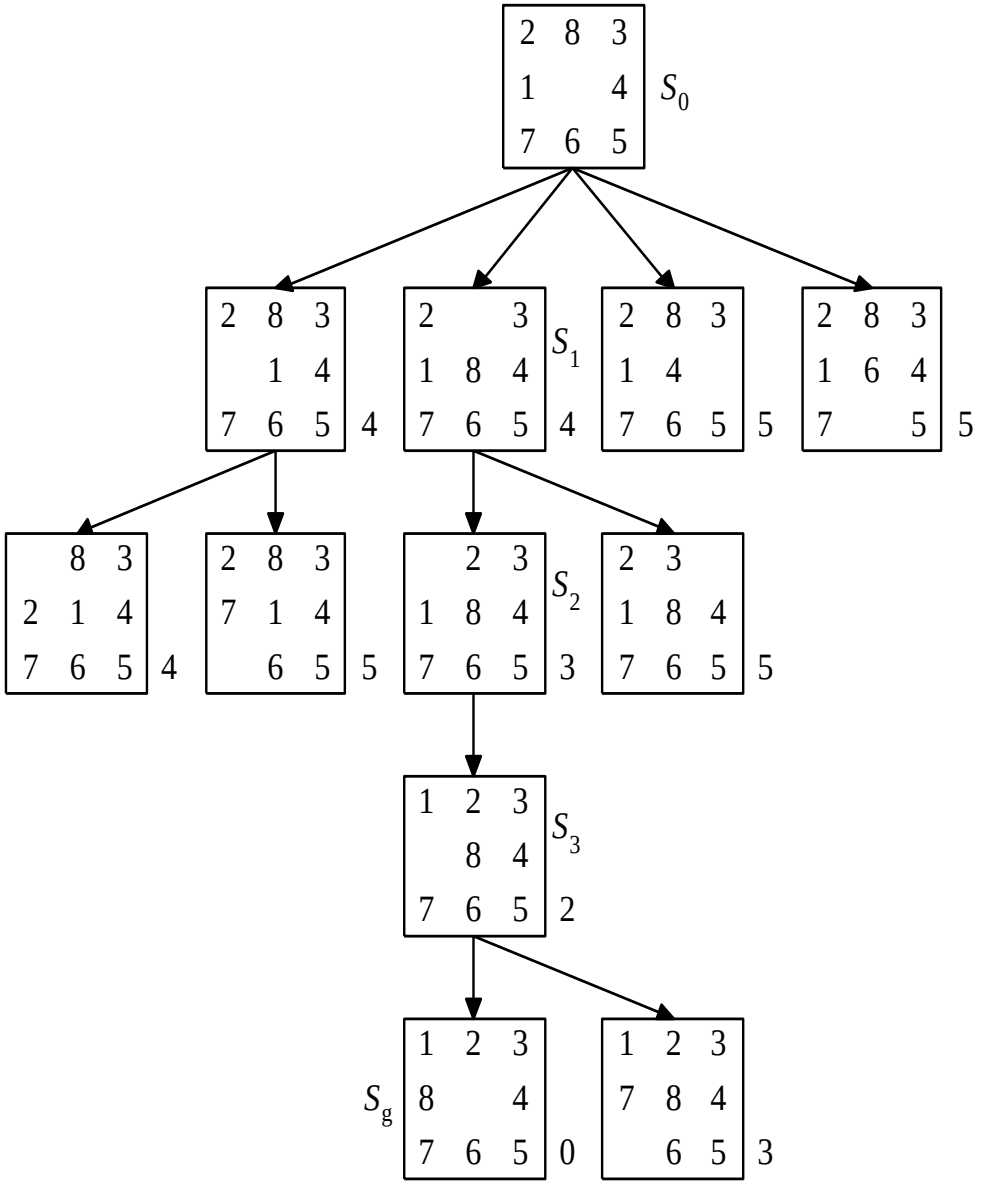


图2-9　八数码问题的全局择优搜索

启发式搜索之——贪婪最佳优先搜索

两个重要函数：评价函数与启发函数

辅助信息	所求解问题之外、与所求解问题相关的特定信息或知识。	
评价函数（evaluation function） f(n)	从当前节点*n*出发，根据评价函数来选择后续结点	下一个结点是谁?
启发函数（heuristic function） h(n)	计算从结点*n*到目标结点之间所形成路径的最小代价值，这里将两点之间的直线距离作为启发函数	完成任务还需要多少代价?

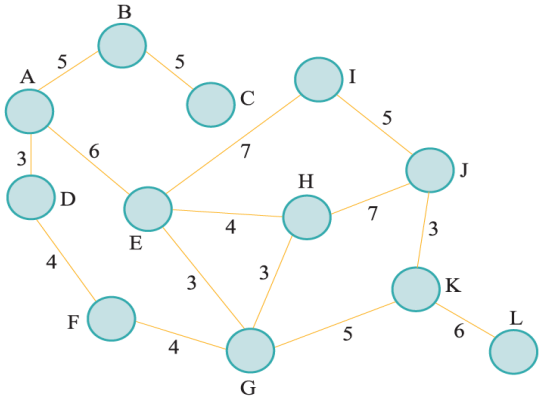


图 3.1　某地区的交通网络图

贪婪最佳优先搜索(Greedy best-first search)：
评价函数f(n)=启发函数h(n)

表 3.2　每个城市到目标城市的直线距离（启发函数取值）

状态	*A*	*B*	*C*	*D*	*E*	*F*	*G*	*H*	*I*	*J*	*K*	*L*
h(*n*)	13	10	6	12	7	8	5	3	6	3	0	6

求取从城市A到城市K之间一条行驶时间最短路线

每个城市到目标城市（即城市K）的直线距离（启发函数取值）

贪婪最佳优先搜索：将启发函数作为评价函数的搜索过程

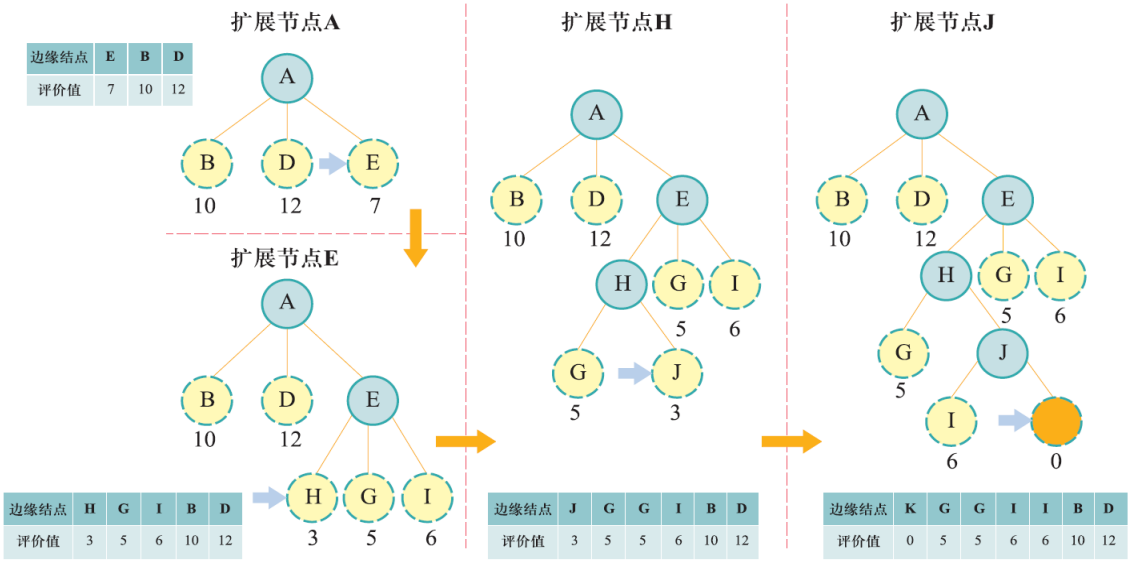


图 3.4　贪婪最佳优先搜索的过程

（注：图中边缘结点下的数字表示其评价函数取值）

贪婪最佳优先搜索的过程。
图中边缘结点下数字表示其评价函数取值

- 图中仍然用实线结点表示已经被扩展过的结点，虚线结点表示边缘结点。
- 边缘结点下的数字表示该结点到目标结点的直线距离，这一距离取值被用来评价当前结点的后续结点选择。需要提醒的是，这一距离取值既是启发函数取值，也是评价函数取值。

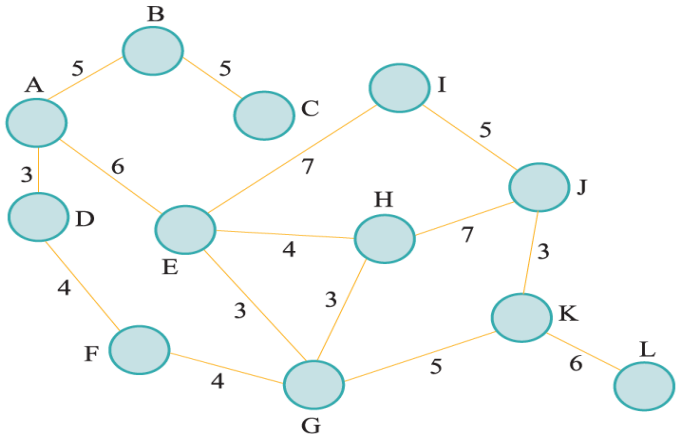


图 3.1　某地区的交通网络图

求取从城市A到城市K之间
一条行驶时间最短路线

表 3.2　每个城市到目标城市的直线距离（启发函数取值）

状态	A	B	C	D	E	F	G	H	I	J	K	L
$h(n)$	13	10	6	12	7	8	5	3	6	3	0	6

每个城市到目标城市（即城市K）的直线距离（启发函数取值）

贪婪最佳优先搜索：将启发函数作为评价函数的搜索过程

算法找到了一条从起始结点到目标结点的路径A → E → H → J → K
但这条路径并不是最短路径，实际上最短路径为A → E → G → K

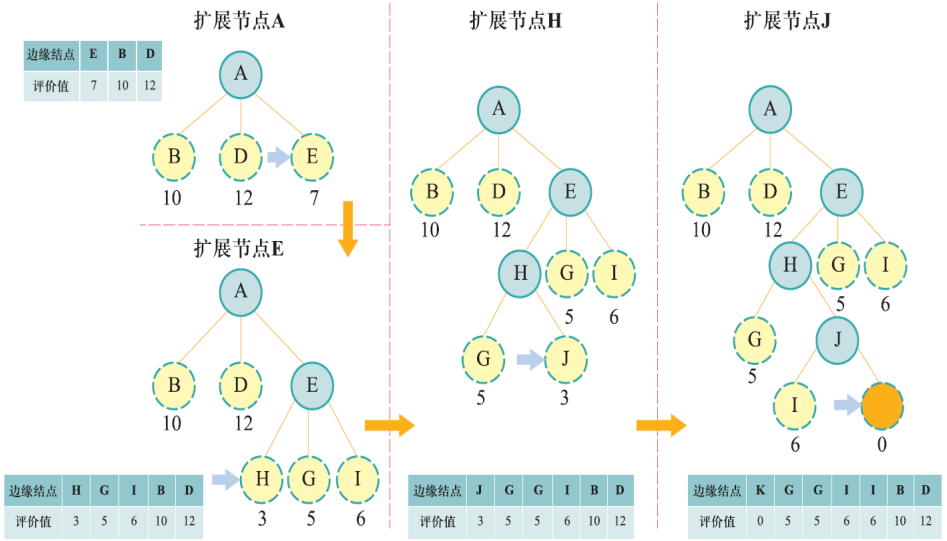


图 3.4　贪婪最佳优先搜索的过程

（注：图中边缘结点下的数字表示其评价函数取值）

贪婪最佳优先搜索的过程

A → E → H → J → K （✗）

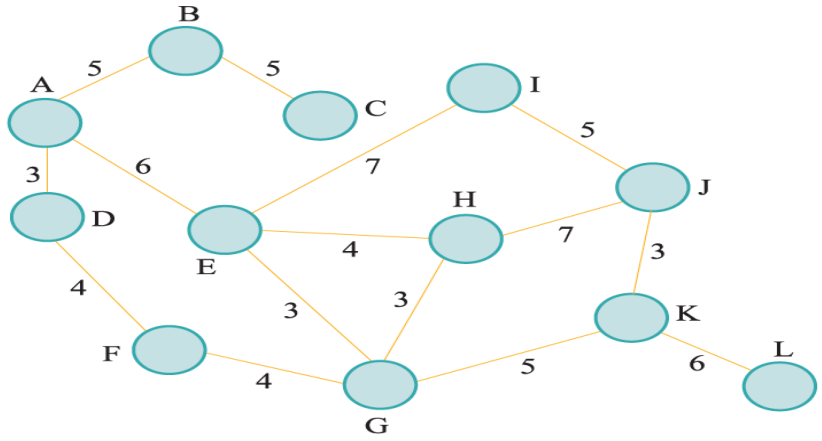


图 3.1　某地区的交通网络图

求取从城市A到城市K之间一条行驶时间最短路线

A → E → G → K (✔)

真正的最短路线

贪婪最佳优先搜索：为什么会失败

欲速则不达

- 贪婪最佳优先搜索算法采用了排除环路的剪枝方法，因此它是完备的，即总是可以找到一个解。但是该算法并不是最优的。
- 在最坏的情况下，贪婪最佳优先搜索算法时间和空间复杂度均为$O(b^m)$，其中b和m分别表示搜索树的分支因子和最大深度。
- 贪婪最佳优先搜索算法的问题在于其总是贪婪地扩展与目标结点之间代价最小（直线距离最近）的结点，而不考虑从初始结点到达该结点所需的代价。
- 在上述搜索步骤中，虽然第四步扩展的J结点距离目标结点的估计代价很小（取值为3），但从起始结点到达该结点的路径A → E → H → J代价为17，已经超过了实际上最短路径的代价（值为14），因此路径A → E → H → J并不是一个好的探索方向。

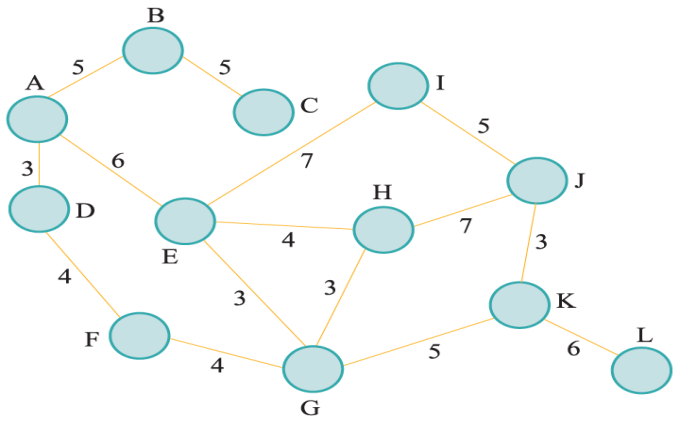


图 3.1　某地区的交通网络图

求取从城市A到城市K之间一条行驶时间最短路线

表 3.2　每个城市到目标城市的直线距离（启发函数取值）

状态	A	B	C	D	E	F	G	H	I	J	K	L
$h(n)$	13	10	6	12	7	8	5	3	6	3	0	6

每个城市到目标城市（即城市K）的直线距离（启发函数取值）

在评价函数中考虑从起始结点到当前结点的路径代价

$$\underbrace{f(n)}_{\text{评价函数}\atop\text{(选择后续结点)}} = \underbrace{g(n)}_{\text{起始结点到结点}n\text{代价}\atop\text{(当前最小代价)}} + \underbrace{h(n)}_{\text{结点}n\text{到目标结点代价}\atop\text{(后续估计最小代价)}}$$

辅助信息	所求解问题之外、与所求解问题相关的特定信息或知识。	
评价函数（evaluation function）f(n)	从当前节点n出发，根据评价函数来选择后续结点。	下一个结点是谁?
启发函数（heuristic function）h(n)	计算从结点n到目标结点之间所形成路径的最小代价值，这里将两点之间的直线距离作为启发函数。	完成任务还需要多少代价?

评价函数$f(n)$被修改为：$f(n)=g(n)+h(n)$，即评价函数等于从起始结点到达结点n的的实际代价取值与结点n到目标结点估计代价取值之和。根据每个结点的评价函数之值不断选择后续结点的搜索算法被称为A算法。



2.2.6 启发式图搜索的A算法和A*算法

1.估价函数

利用启发函数h(x)制导的启发式搜索，虽然很高效，但也可能误入歧途。所以，为了更稳妥一些，人们把启发函数扩充为估价函数。一般来讲，估计一个结点的价值，必须综合考虑两方面的因素：已经付出的代价和将要付出的代价。即把f(x)定义为从初始结点经过x结点到达目标结点的最小代价估计值。其一般形式是

$$f(x)=g(x)+h(x)$$

其中g(x)为从初始节点S_0到节点x已经付出的代价，h(x)是启发函数。即估价函数f(x)是从初始节点S_0到达节点x处已付出的代价与节点x到达目标节点S_g的接近程度估计值之总和。



2. A算法——评价函数和启发函数各司其职

A算法是基于估价函数f(x)的一种加权状态图启发式搜索算法。

3. A*算法

如果对A算法再限制其估价函数中的启发函数h(x)满足：对所有的节点x均有　$h(x) \le h^*(x)$

其中$h^*(x)$是从节点x到目标节点的最小代价,即最佳路径上的实际代价(若有多个目标节点则为其中最小的一个)，则它就称为A*算法。

定义：在A算法中，如果对所有的x有$h(x) \le h^*(x)$，则称h(x)为$h^*(x)$的下界。它代表某种偏于保守的估计。采用$h^*(x)$的下界h(x)为启发函数的A算法叫A*算法。

h*(x)只是理论上存在的理想的实际最短路径值，并不是已知的，实际中无需也不可能求出来的。

用它只是作为参照，给h(x)的设计有一个约束，我们的目标就是要使h(x)尽可能逼近h*(x)所代表的最短路径。有了h(x) <= h*(x)这一条件可以在理论上保证A*算法是可采纳的。

为更好地理解A星算法，可以类比这样一个原理：已知平面上不重合两点A，B。又知两点 C ，D。求从A经过C或D到B最短路径。我们知道对于三角形两边之和大于第三边。两点之间直线最短（即h*(x)）。对于C，D两点，越靠近A到B直线的那么经过他的就是最短的。

A* Algorithm


从起点开始
开始
把起点S放入OPEN表，计算f(S)
目标判定
OPEN表为空表?
是
失败
否
把OPEN表中第一个节点(N)移至CLOSED表
N为目标节点吗?
是
成功
否
节点扩展
计算N的所有后继节点的f值并进行节点扩展
估价排序
按f值升序重排OPEN表

A*搜索算法搜索过程

- 右图中用实线结点表示已经被扩展过的结点，虚线结点表示边缘结点。每个边缘结点下方等式即为$g(n)+h(n)=f(n)$的取值。
- 首先扩展根结点（起始结点）A，得到三个边缘结点B、D和E。其中路径A → B的实际代价为5，结点B的启发函数取值为10，因此结点B的评价函数取值为 $5+10=15$ 。同理可算出D和E的评价函数值分别为15和13。算法选择评价函数值最小的E结点进行扩展，得到评价函数取值分别为13、14、19的三个新的边缘结点H、G、I。算法从当前边缘集合中选择评价函数值最小的结点H进行扩展，得到评价函数值为18和20的两个结点。到目前为止，算法的流程和贪婪最佳优先搜索保持一致，两者区别体现在下一步操作上。
- 不同于贪婪最佳优先搜索选择扩展结点J，A*算法选择扩展评价函数值最小的第三层中结点G（A → E → G），接着又扩展了评价函数值最小的结点K，顺利到达终止状态。此处扩展次序的差异使得A*算法能够顺利找到最短路径A → E → G → K，而贪婪最佳优先搜索则不能。
- 值得注意的是，每次从边缘结点集合中所扩展的结点并非总是当前结点的相邻后继结点，如在图中，所扩展的第三层结点G（A → E → G）并不是先前扩展结点H(A → E → H）在搜索树上的相邻后继结点。

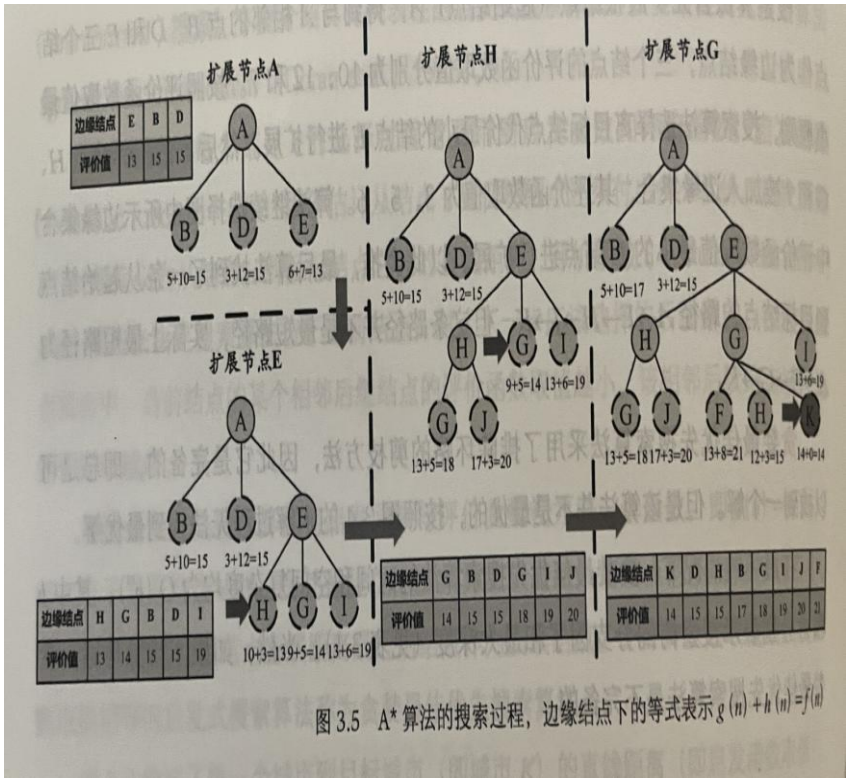
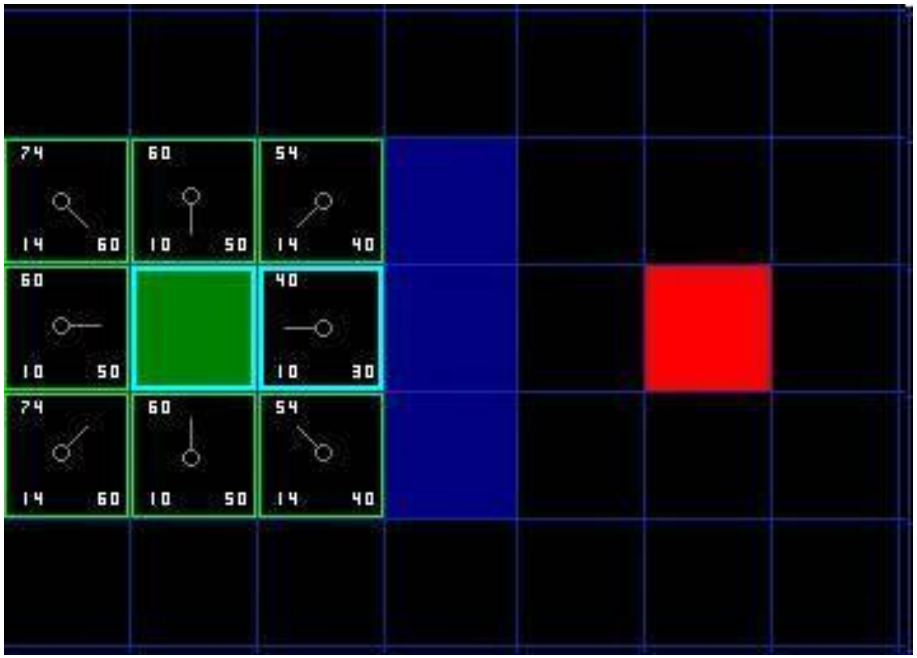
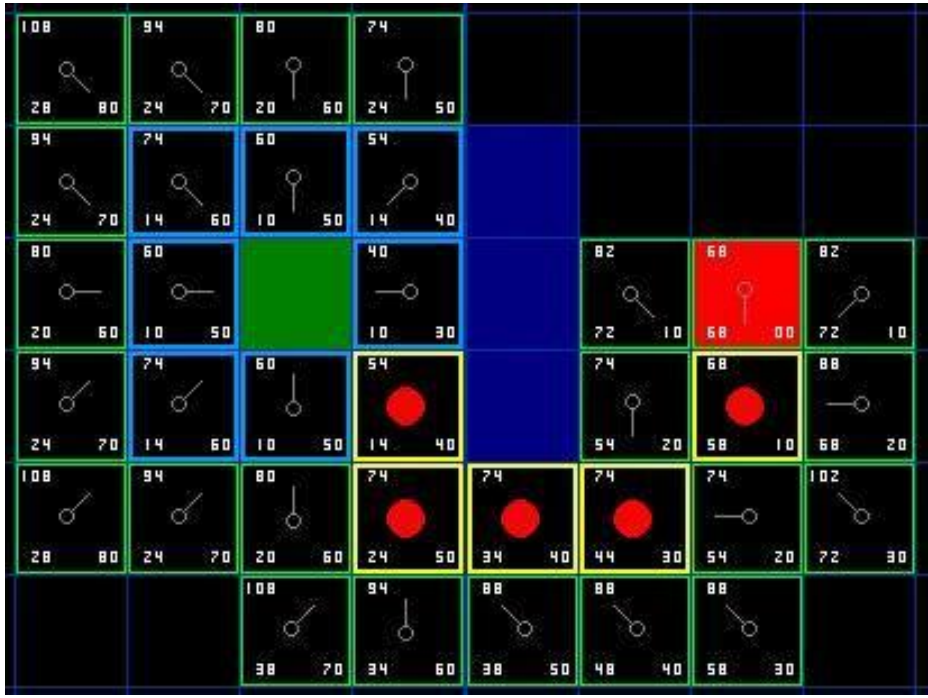


图 3.5　A* 算法的搜索过程，边缘结点下的等式表示 $g(n)+h(n)=f(n)$

A*算法实际是Dijkstra算法的扩展，用于路径规划与图的遍历，比如星际争霸游戏。下面举一个例子，用A*算法求解机器人从A到目标位置B的最短路径





把要搜索区域划分成正方形格子， 可描述为二维数组，数组的每个元素对应一个小方格。

A*搜索性能分析

A*算法的完备性和最优性取决于搜索问题和启发函数的性质。

- 可容性（admissible）
- 一致性（consistency）

表 3.3　本节中使用的若干符号

符号	含义
$h(n)$	结点 n 的启发函数取值
$g(n)$	从起始结点到结点 n 所对应路径的代价
$f(n)$	结点 n 的评价函数取值
$c(n, a, n')$	从结点 n 执行动作 a 到达结点 n' 的单步代价
$h^*(n)$	从结点 n 出发到达终止结点的最小代价

A*搜索性能分析：可容性

对于任意结点n，有$h(n) \leq h^*(n)$，如果n是目标结点，则有$h(n) = 0$。$h^*(n)$是从结点n出发到达终止结点所付出的最小代价。可以这样理解满足可容性的启发函数，启发函数不会过高估计（over-estimate）从结点n到终止结点所应该付出的代价（即估计代价小于等于实际代价）。在上述问题中，对每个结点而言，其状态（城市）到目标状态（城市）K之间的行驶距离不会小于两城之间的直接距离，因此列车以最大行驶速度通过直线距离花费的时间必然不会超过实际代价，将该时间作为启发函数值，是满足可容性条件的。

A*搜索性能分析：一致性

启发函数的一致性指满足条件 $h(n) \leq c(n,a,n') + h(n')$，这里$c(n,a,n')$表示结点$n$通过动作$a$到达其相应的后继结点$n'$的代价。在图3.1.1中，假设结点$n$和$n'$对应的状态（城市）为$s$和$s'$，如果将单步代价$c(n,a,n')$重新定义为$s$和$s'$在二维平面上的欧氏距离，则根据欧氏距离三角不等式原则，城市s到目标城市K之间直线距离一定小于等于从城市s到其相邻城市s'的直线距离与城市s'到目标城市K之间直线距离之和，此时，将当前状态（城市）与目标状态（城市）K之间直线距离作为启发函数值，则启发函数满足一致性条件。

A*搜索性能分析：一致性必然导致可容性

- 一致性是一个比可容性更严格的性质，即一个启发函数如果满足一致性条件，那么该启发函数必然满足可容性条件。

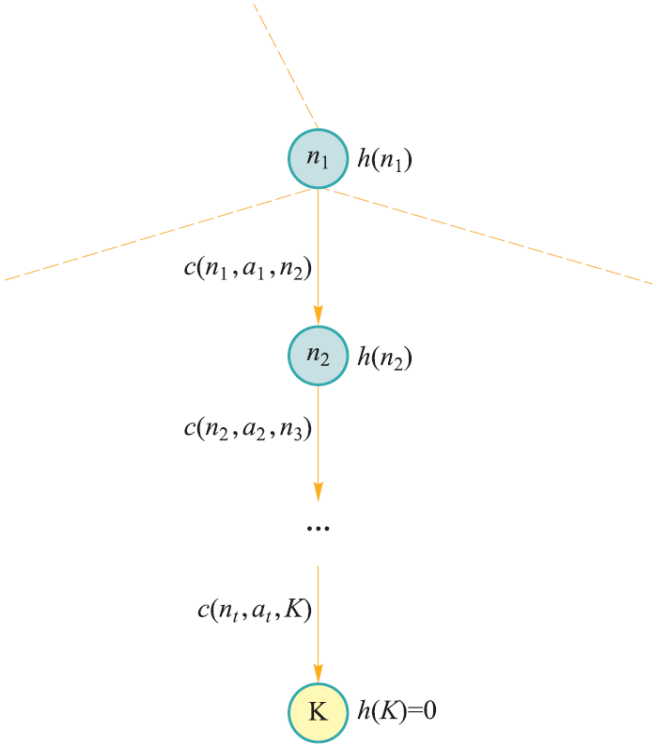


图 3.6　从 n_1 结点到终止结点的最短路径

如图中以n_1为根结点的子树，假设从该结点到达终止结点代价最小的路径为$n_1 \to n_2 \to \cdots \to \mathrm{K}$。由于$h(\mathrm{K}) = 0$且启发函数满足一致性条件，因此存在：

$$\begin{aligned} h(n_1) &\le h(n_2) + c(n_1, a_1, n_2) \\ &\le h(n_3) + c(n_2, a_2, n_3) \\ &\le \cdots \\ &\le c(n_1, a_1, n_2) + c(n_2, a_2, n_3) + \cdots \\ &+ c(n_t, a_t, \mathrm{K}) \\ &= h^*(n_1) \end{aligned}$$

即满足一致性条件的启发函数一定满足可容性条件。

A*搜索性能分析：A*算法是完备的么?

如果所求解问题和启发函数满足以下条件，则A*算法是完备的：

- 搜索树中分支数量是有限的，即每个结点的后继结点数量是有限的
- 单步代价的下界是一个正数
- 启发函数有下界

A*搜索性能分析：A*算法是最优的么?

如果启发函数是可容的，那么A*算法满足最优性

假设A*算法找到的第一个终止结点为n。对于此时边缘集合中任意结点n'，根据算法每次扩展时的策略，即选择评价函数取值最小的边缘结点进行扩展，有$f(n) \leq f(n')$。由于A*算法对评价函数定义为$f(n) = g(n) + h(n)$，且$h(n) = 0$，有$f(n) = g(n) + h(n) = g(n)$，$f(n') = g(n') + h(n')$，综合可得

$$g(n) \leq g(n') + h(n') \leq g(n') + h^*(n')$$

其中$g(n)$为从起始结点到结点n所对应路径的代价，$g(n') + h^*(n')$表示从初始结点出发经过结点n'到达终止结点的最小代价。也就是说此时扩展其他任何一个边缘结点都不可能找到比结点n所对应路径代价更小的路径，因此结点n对应的是一条最短路径，即算法满足最优性。



A*算法的性质

定理3.1 对可解状态空间图来说，A*算法一定结束。

推论：OPEN表中任何$f(n)<f^*(S_0)$的结点n都将被A*算法选作扩展结点。

定理3.2 A*算法是可采纳的。

定理3.3 由A*算法选来扩展的结点n都满足性质$f(n)\leq f^*(S)$。

A*算法在人工智能中是一种典型的启发式搜索算法，在游戏（尤其是角色扮演和即时战略游戏）编程中进行最短路径搜索时经常会用到。

推荐阅读

https://www.cnblogs.com/zhoug2020/p/3468167.html



图搜索的一般框架(GRAPHSEARCH)

(1) 把初始节点S_o放入OPEN表中。建立初始为空的已扩展结点表Closed表;

(2) **LOOP：**若OPEN表为空, 则搜索失败, 退出。

(3) 移出OPEN表中第一个节点放入CLOSED表中，并冠以编号n。

(4) 若目标节点S_g=n, 则搜索成功, 结束。

(5)若n不可扩展, 则转步(2)。

(6)**扩展节点n，同时生成不是n的祖先的那些后继节点的集合M，把M的这些成员作为n的后继子节点，对这组子节点M做如下处理：**

① 删除N的先辈节点(如果有的话)。

②对已存在于OPEN表的节点(如果有的话)也删除之；但删除之前要比较其返回初始节点的新路径与原路径,如果新路径“短”，则修改这些节点在OPEN表中的原返回指针,使其沿新路返回(如图3-5所示)。

③对已存在于CLOSED表的节点(如果有的话)，做与(2)同样的处理，并且再将其移出CLOSED表，放入OPEN表重新扩展(为了重新计算代价)。

④对其余子节点配上指向N的返回指针后放入OPEN表中某处, 或对OPEN表进行重新排序, 转步(2)。

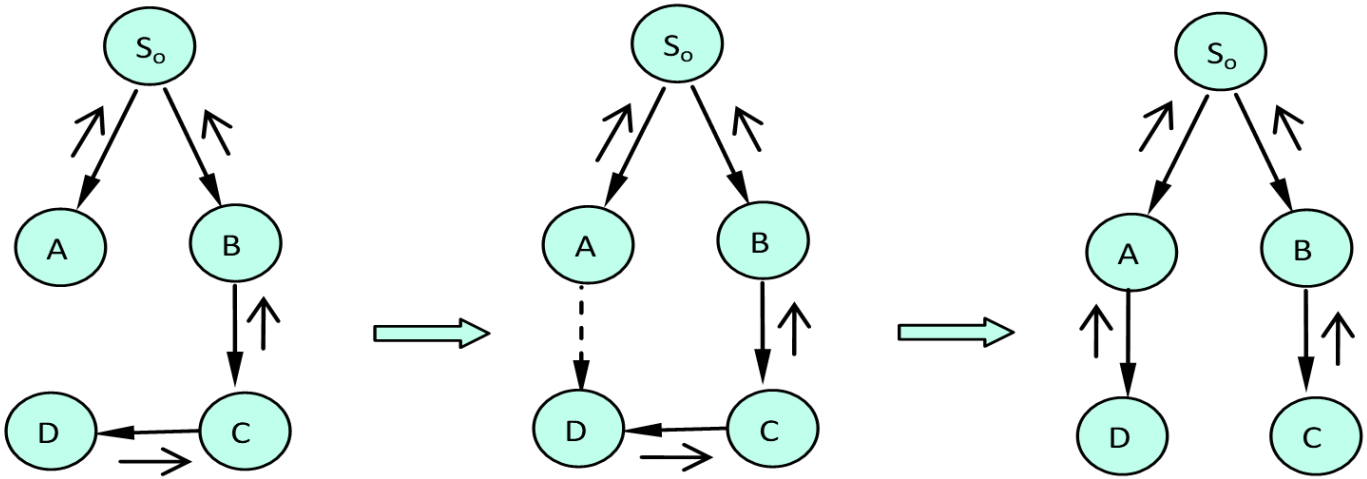


图 3-5　状态图搜索过程中修改返回指针示例

说明：

(1) 这里的返回指针也就是父节点在CLOSED表中的编号。

(2) 步6中修改返回指针的原因是，因为这些节点又被第二次生成，所以它们返回初始节点的路径已有两条，但这两条路径的“长度”可能不同。那么，当新路短时自然要走新路。

(3) 这里对路径的长短是按路径上的节点数来衡量的，后面我们将会看到路径的长短也可以其“代价”(如距离、费用、时间等)衡量。若按其代价衡量，则在需修改返回指针的同时还要修改相应的代价值，或者不修改返回指针也要修改代价值(为了实现代价小者优先扩展)。



（7）按某一任意方式或按某个探试值，对OPEN表重新排序，然后转步2循环。

•如果令f(x)=d(x)，且按f(x)递增排序，则是广度优先搜索法，如按f(x)递减方式排序，则是深度优先法；

•如果令f(x)=g(x)，则是分支界限法或瞎子爬山法；

•如果令f(x)=h(x), 则分全局择优(贪婪最佳搜索)和局部择优；

• 如果令f(x)=g(x)＋h(x) 则是A算法和A* 算法。

线式搜索算法：

●不回溯的线式搜索

(1) 把初始节点S_0放入CLOSED表中。

(2) 令$N = S_0$。

(3) 若N是目标节点,则搜索成功,结束。

(4) 若N不可扩展,则搜索失败,退出。

(5) 扩展N,选取其一个未在CLOSED表中出现过的子节点N_1放入CLOSED表中, 令$N = N_1$, 转步(3)。



- **可回溯的线式搜索**

(1) 把初始节点S_o放入CLOSED表中。

(2) 令$N = S_o$。

(3) 若N是目标节点, 则搜索成功, 结束。

(4) 若N不可扩展, 则移出CLOSED表的末端节点N_e,若$N_e = S_o$,则搜索失败, 退出。否则, 以CLOSED表新的末端节点N_e作为N,即令$N = N_e$, 转步(4)。

(5) 扩展N, 选取其一个未在CLOSED表用出现过的子节点N_1放入CLOSED表中, 令$N = N_1$,转步(3)。

状态图搜索策略小结

- 树式搜索
  - 盲目搜索
    - 穷举式
      - 广度优先
      - 深度优先
        - 有界深度优先
  - 启发式搜索
    - 全局择优(最好优先，基于启发函数$h(x)$)
    - 局部择优(瞎子爬山，基于$h(x)$)
    - 分支界限(全局最小代价优先，基于代价函数$g(x)$)
    - 最近择优(瞎子爬山，局部最小代价优先，基于$g(x)$)
    - A算法(基于估价函数$f(x)=g(x)+h(x)$)
      - A*算法(最佳图搜索，$h(x)\leqslant h^*(x)$)
- 线式搜索
  - 盲目搜索
    - 随机碰撞
    - 回溯穷举(生成—测试法)
  - 启发式搜索
    - 不回溯(瞎子爬山，基于$h(x)$、$g(x)$或$f(x)$)
    - 智能回溯(启发式生成—测试法)

状态图搜索策略小结

影响搜索策略的要素有哪些？有四个要素决定了一个搜索策略A的性质：

(1)选择下一个被考察结点的范围。有两种不同的范围

- 全局的—所有已生成而未被考察的结点(不管它是谁的子结点)；
- 局部的——自己的子结点集。

(2)选择下一个被考察结点的标准。有四种不同的标准：

- d(n)——搜索树中从S_0到N的路径长度；
- g(n)——代价树中从S_0到N的路径的代价；
- h(n)——当前n结点到 Sg的路径的估计代价；
- f(n)——代价树中从 S_0出发，经过n结点再到 Sg的路径的最小代价。它既考虑了已经付出的代价g(n)，又对将要付出的代价给出了估计h(n)，所以比d(n)、g(n)和h(n)都要准确。

在深度优先搜索策略中，又有两个要素需考虑：

(3) 试探性　局部型策略都是试探性的，在OPEN表中选择

(4) 有界性

搜索策略的诸要素和分类

<table>
<tr><td rowspan="2">搜索策略</td><td colspan="4">选择下一个被考察结点的范围</td><td rowspan="2">标准</td></tr>
<tr><td>全局</td><td colspan="3">局部</td></tr>
<tr><td>问题树</td><td>广度优先</td><td>深度优先</td><td colspan="2">不可撤回</td><td>d(N):从 S_0 到 N 的路径长度</td></tr>
<tr><td></td><td></td><td></td><td rowspan="2">试探性</td><td>有界</td><td></td></tr>
<tr><td></td><td></td><td></td><td>无界</td><td></td></tr>
<tr><td>代价树</td><td>分支界限</td><td colspan="3">瞎子爬山</td><td>g(N):代价树中从 S_0 到 N 的路径代价</td></tr>
<tr><td>启发式搜索</td><td>最好优先</td><td colspan="3">局部择优</td><td>f(N):代价树中从 S_0 出发经过 N 再到 Sg 的路径的最小代价</td></tr>
</table>

2.3 与或图问题求解

2.3.1 什么是与或图

我们仍用例子引入与或图的概念。

例2.15 如图2–11所示，设有四边形ABCD和A’B’C’D’要求证明它们全等。

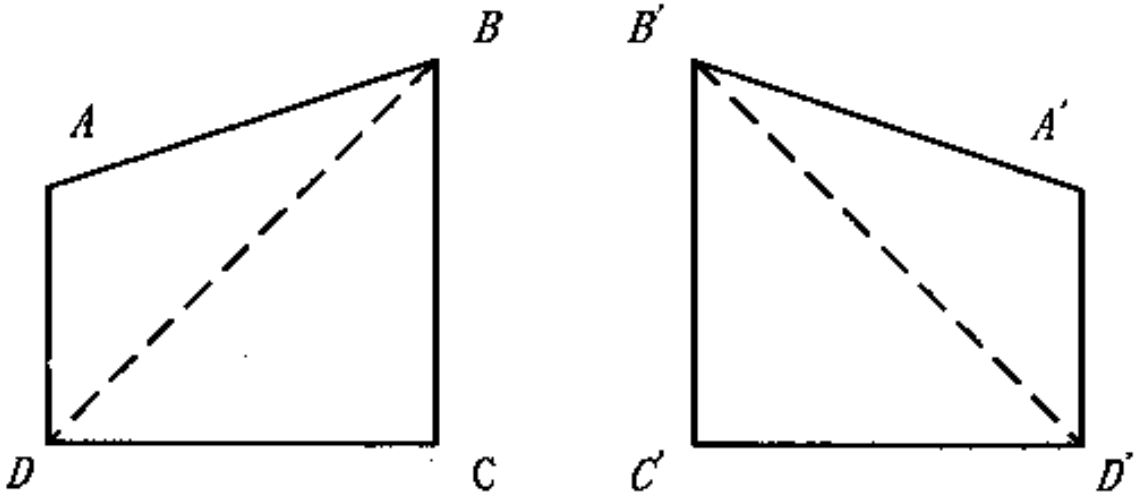


图2-11 四边形ABCD和A′B′C′D′

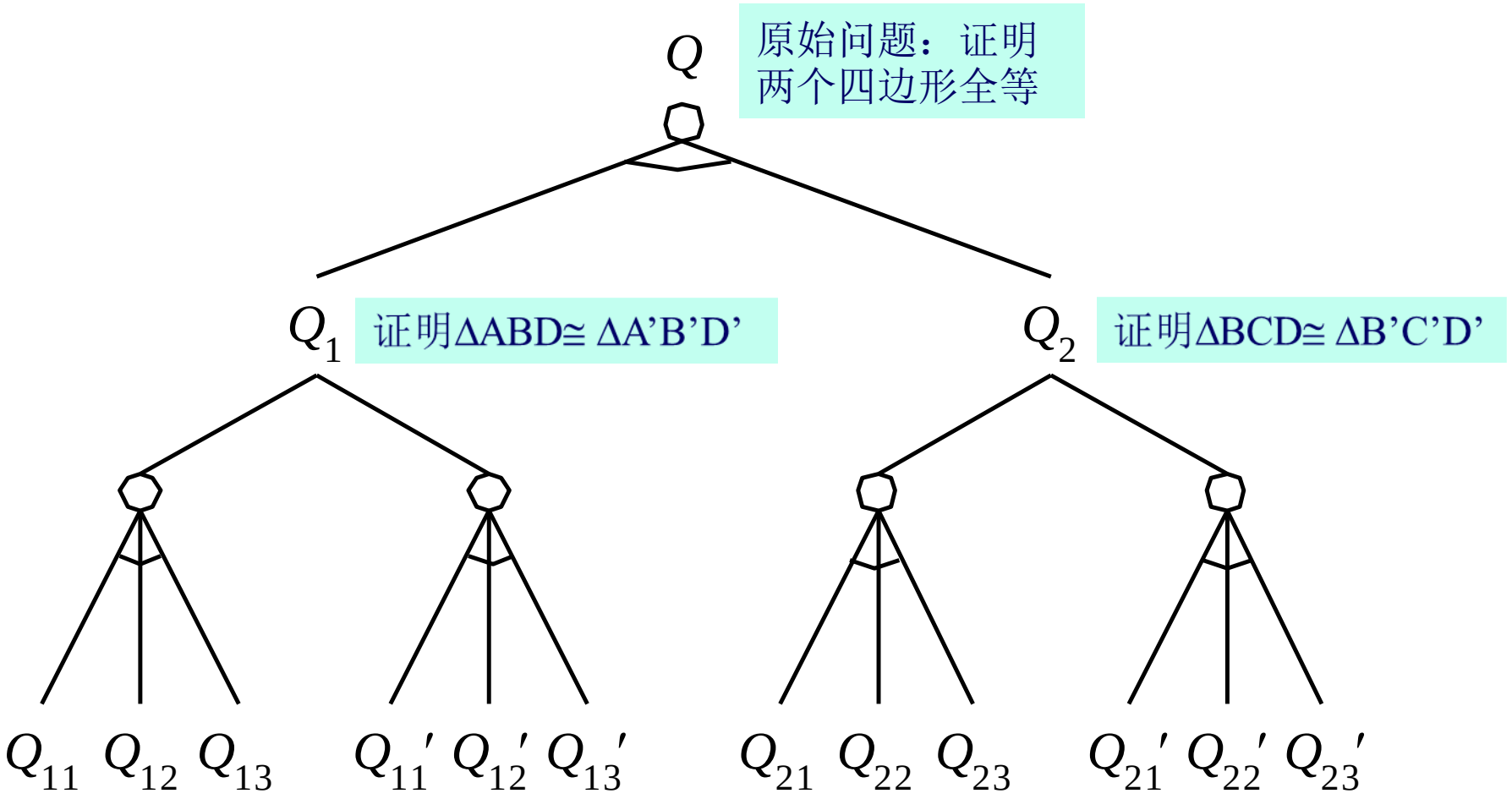
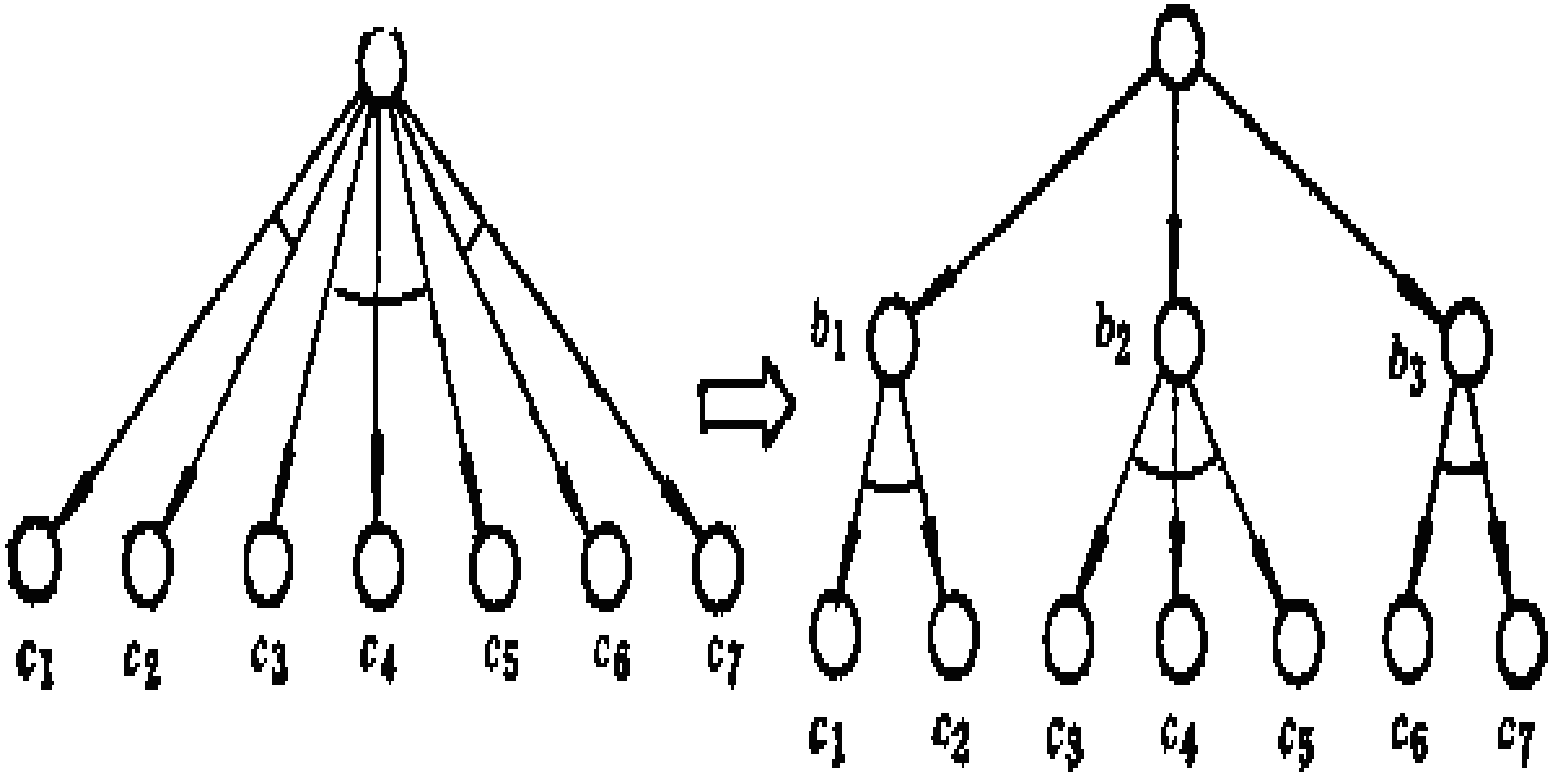


图2-12　问题的分解与变换

现在考虑原问题与这两组子问题的关系，我们便得到图2–12。图中的弧线表示所连边为“与”关系，不带弧线的边为或关系。**这个图中既有与关系又有或关系，因此被称为与或图**。但这个与或图是一种特殊的与或图，称为**与或树**。

与或图可以用来描述一类问题的求解过程。事实上，当我们把待解的原问题作为初始节点，把由原问题经一系列分解或变换而得到的直接可解的简单问题作为目标节点，那么，**问题求解过程也就是在一个与或图中寻找一个从初始节点到目标节点的路径问题。**



与或节点的扩展方法

- 可以看出，从与或关系来看，前面的状态图，实际就是或图。这就是说，与或图是状态图的推广，而状态图是与或图的特例。
- 求解与或图问题就是在与或图中搜索解图或解树的问题。



2.3.2 问题的与或图表示

与或图是描述问题求解的另一种有向图。与或图一般表示问题的变换过程（而不是状态变换）。具体讲，它是从原问题出发，通过运用某些规则不断进行问题分解（为子问题，得到与分支）或变换（为等价问题，得到或分支），而得到一个与或图。换句话说，与或图的节点一般代表问题。那么，整个图也就表示问题空间。

例2.19 三阶梵塔问题

对于n阶梵塔问题，我们也可以这样考虑：为把1号杆上的n个盘子搬到3号杆，可先把上面的n-1个盘子搬到2号杆上；再把剩下的一个大盘子搬到3号杆；然后再将2号杆上的n-1个盘子搬到3号杆。这样，就把原来的一个问题分解为三个子问题。这三个子问题都比原问题简单，其中第二个子问题已是直接可解的问题。对于第一和第三两个子问题，可用上面n个盘子的方法，做同样的处理。

根据这一思想，我们可把三阶梵塔问题分解为下面的三个子问题：(1)把A、B盘从1号杆移到2号杆。

(2)把C盘从1号杆移到3号杆。

(3)把A、B盘从2号杆移到3号杆。

其中子问题(1)、(3)又分别可分解为三个子问题。于是，我们可得到三阶梵塔问题的与或树表示，如下图。

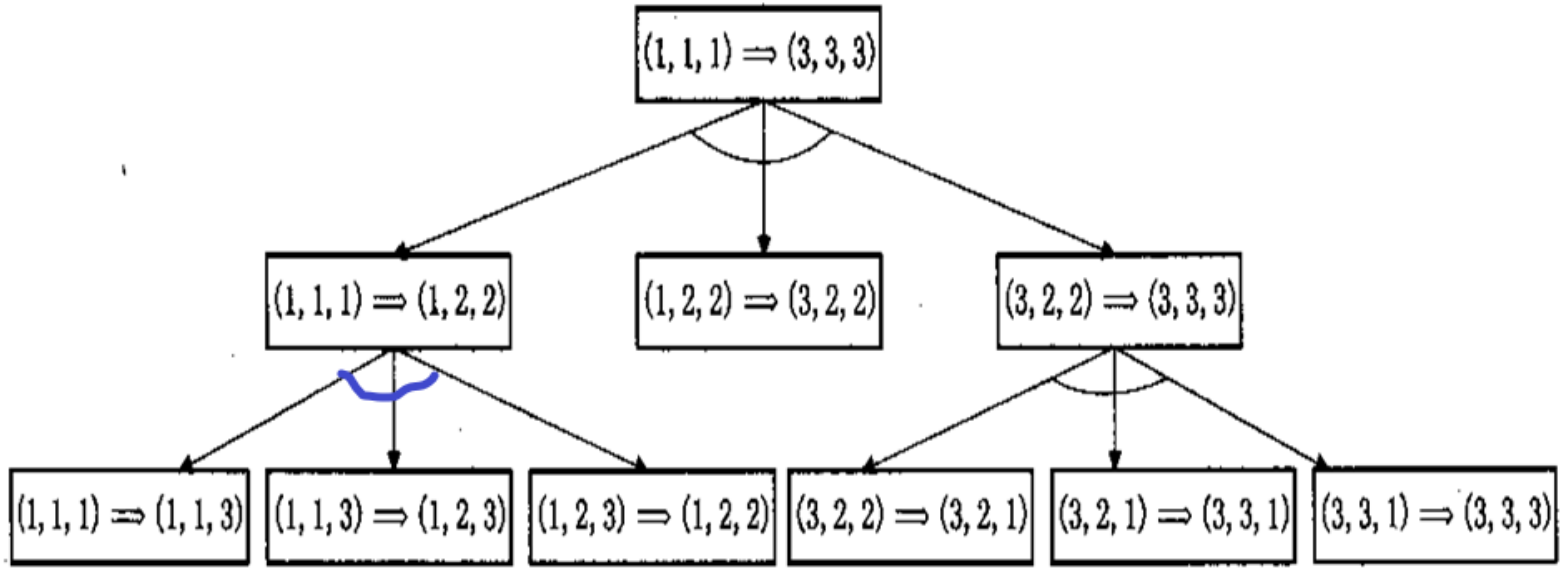


图2–14　三阶梵塔问题的与或树

需说明的是，三元组 (i, j, k)

i——代表金盘C所在的杆号;

j——代表金盘B所在的杆号，

k——代表金盘A所在的杆号。在图2-14所示的与或树中，共有七个终止节点，对应于七个本原问题，它们是通过“分解”得到的。把这些本原问题的解按从左至右的顺序排列，就得到了原始问题的解如下图：

$$(1,1,1) \Rightarrow (1,1,3) \Rightarrow (1,2,3) \Rightarrow (1,2,2) \Rightarrow (3,2,2)$$

$$\Rightarrow (3,2,1) \Rightarrow (3,3,1) \Rightarrow (3,3,3)$$



2.4 与或图(树)搜索

1. 搜索方式与解树

2. 可解结点与不可解结点

3.与或树的启发式搜索

- 在与或树中计算解树的代价F(T)
- 希望解树

4.与或树的最好优先搜索



1. 搜索方式与解树

与或图搜索也分为树式和线式两种类型。对于树式搜索来讲，其搜索过程也是不断地扩展节点，并配以返回指针，而形成一棵不断生长的搜索树。

与或图搜索解树，不像在状态图中那样只是简单地寻找目标节点，而是边扩展节点边进行逻辑判断，以确定初始节点是否可解。一旦能够确定初始节点的可解性，则搜索停止。这时，根据返回指针便可从搜索树中得到一个解树。

在与／或树中用标记法对结点自下而上进行标记。能导致根结点可解的那些可解结点及有关枝组成的子树叫该与／或树的解树，一个与／或树的解树可能有多个。所以，解树实际上是由可解节点形成的一个子树，这个子树的根为初始节点，叶为终止节点。

为了叙述方便，下面引入一些新概念。在与或图（树）中有两大类节点：

- **与节点：一个节点的子节点之间如果是“与”关系，则该节点便称为与节点；**
- **或节点：一个节点的子节点如果是“或”关系，则该节点便称为或节点；**

与或树中无子节点的节点称为端节点，它可以被标记为两类节点——终止节点和非终止节点；

直接可解的简单问题称为本原问题 ，本原问题对应的节点称为终止节点，反之为非终止节点。

注意，终止节点一定是端节点，但端节点不一定是终止节点。



2.标记法进行可解性判别

在与／或树中可解结点用“黑点”表示，不可解结点用“×”表示。

怎样判断一个节点的可解性呢？下面我们给出判别准则。

(1)一个节点是可解节点，则节点须满足下列条件之一：

①终止节点是可解节点；

②一个与节点可解，当且仅当其子节点全都可解；

③一个或节点可解，只要其子节点至少有一个可解。

(2)一个节点是不可解节点，则节点须满足下列条件之一：

①非终止节点的端节点是不可解节点；

②一个与节点不可解，只要其子节点至少有一个不可解；

③一个或节点不可解，当且仅当其子节点全都不可解。

3.与或树的启发式搜索

为了求得最优解树，就要在每次确定欲扩展的节点时，先往前多看几步，计算一下扩展这个节点可能要付出的代价，并选择代价最小的节点进行扩展。这种根据代价决定搜索路线的方法称为与或树的有序搜索，它是一种重要的启发式搜索策略。

（1）. 在与或树中计算解树的代价F(T)

解树的代价就是树根（初始节点S_0）的代价。树根的代价是从树叶开始自下而上逐层计算而求得的。因此在与或树的搜索过程中，代价的计算方向与搜索树的生长方向相反。这一点是与状态图不同的。具体计算方法如下：

① 若x是终止结点，其代价g（x）＝0;

② 若x是或结点，$g(x) = \min\{ c(x,y_i)+g(y_i) \}$ $(1 \le i \le n)$

③ 若x是与结点，它有多个子结点y_i (i=1，2，…，n)，其代价根据问题的不同有两种不同的计算法则：

$g(x) = \sum [c(x,y_i)+g(y_i)]$　　(和代价法则)

$g(x) = \max [c(x,y_i)+g(y_i)]$　　（最大代价法则）

④ 对非终止的端节点x,$g(x)=\infty$

⑤ 根结点S的代价g(S)即为解树T的总代价，记作 F（T）。

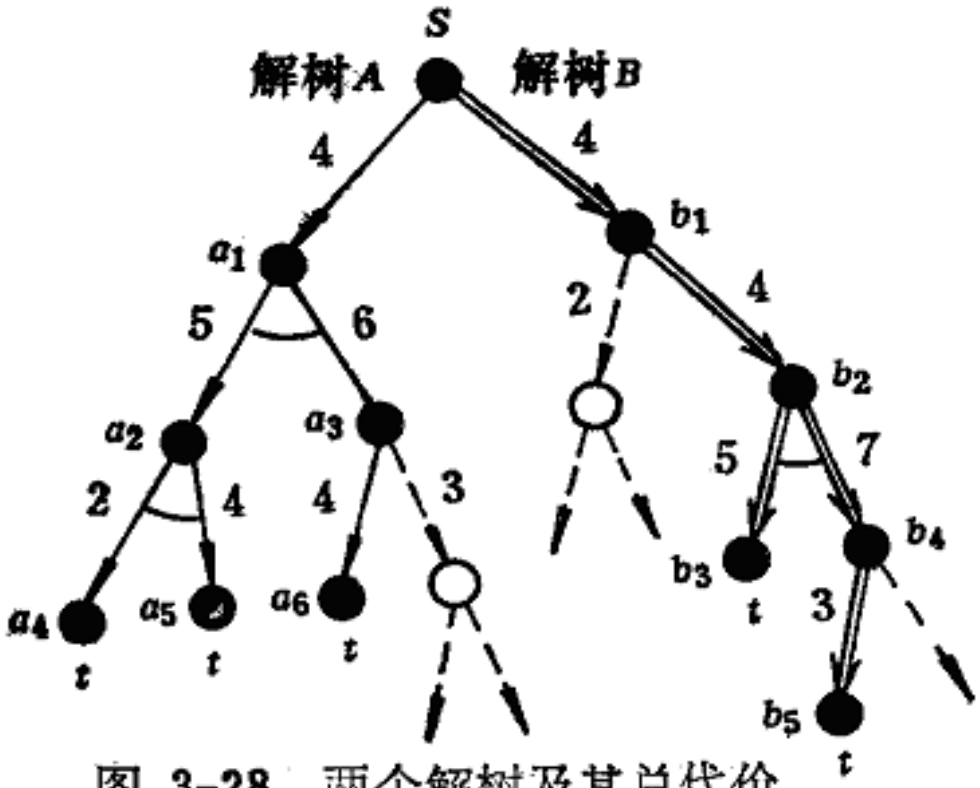


图 3-28　两个解树及其总代价

表 3-1　解树总代价的计算过程

解树 A	结　点	a_6	a_5	a_4	a_3	a_2	a_1	S
	和代价	0	0	0	4	6	21	25
	最大代价	0	0	0	4	4	10	14
解树 B	结　点		b_5	b_4	b_3	b_2	b_1	S
	和代价		0	3	0	15	19	23
	最大代价		0	3	0	10	14	18

例2.17 如下图所示的与或树，其中包括两棵解树，一棵解树由S_0,A,t_1和t_2组成；另一棵解树由S_0,B,D,G,t_4和t_5组成。在此与或树中，t_1,t_2,t_2,t_4,t_5为终止节点；E,F是端节点，其代价均为∞。

由右边的解树可得
按和代价：**g(A)=11, g(S_0)=13;**
按最大代价：**g(A)=6, g(S_0)=8**

由左边的解树可得：

按和代价：

g(G)=3,g(D)=4,g(B)=6,g(S_0)=8

按最大代价：

g(G)=2,g(D)=3,g(B)=5,g(S_0)=7

显然，若按和代价计算，左边的解树是最优解树，其代价为8

若按最大代价计算，左边的解树仍然是最优解树，其代价是7。

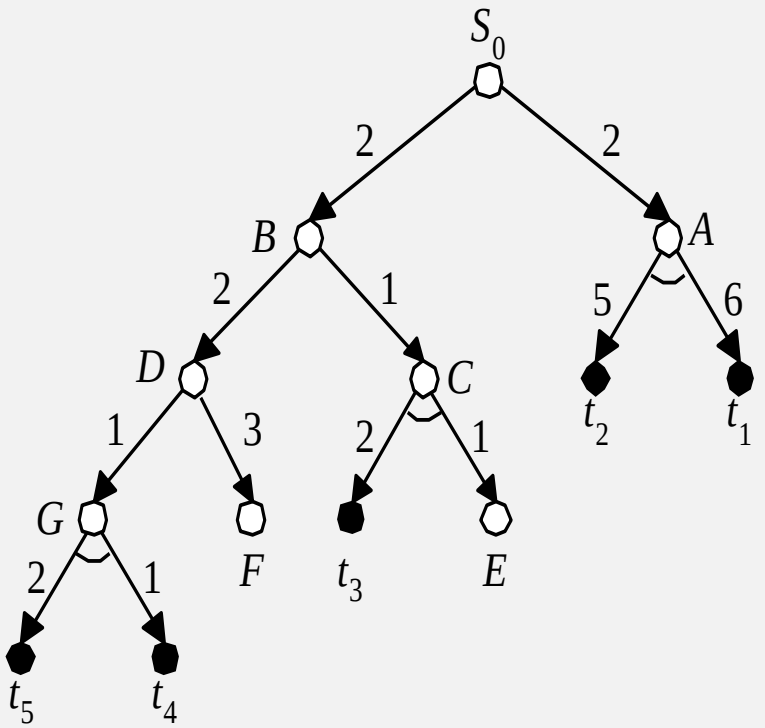


图2—16 含代价的与或树



（2）希望解树τ0

无论是用和代价法还是最大代价法，当要计算任一节点x的代价g(x)时，都要求已知其子节点y_i的代价$g(y_i)$。但是，搜索是自上而下进行的，即先有父节点，后有子节点，除非节点x的全部子节点都是不可扩展节点，否则子节点的代价是不知道的。此时节点x的代价g(x)如何计算？



有序搜索的目的是求出最优解树，即代价最小的解树。这就要求搜索过程中任一时刻求出的部分解树其代价都应是最小的。为此，**每次选择欲扩展的节点时都应挑选有希望成为最优解树一部分的节点进行扩展。由于这些节点及其先辈节点所构成的与或树有可能成为最优解树的一部分，因此称它为“希望树”**。

在搜索过程中，随着新节点的不断生成，节点的代价值是不断变化的，因此希望树也在不断变化。但不管如何变化，任一时刻的希望树都必须包含初始节点S_0。在与／或树的启发式搜索中，**希望树的意义在于，它总是对最优解树近根部分的某种估计**。它的定义如下：



希望解树定义

①初始结点S_0在希望解树τ_0中；

②如果n在希望解树τ_0中，则必有：

（a）如果 n是具有子结点集$\{n_i \mid i=1,2,\ldots k\}$的或结点，则具有 $\min[c(n，n_i)+b(n_i)$值的那个子结点n_i 应在希望解树τ_0中；

（b）如果n是与结点，则它的全部子结点n_i（i＝1，…，k）应在希望解树τ_0中。

4. 与或树的最好优先搜索

与或树的有序搜索过程是一个**不断选择、修正希望解树的过程**。如果问题有解，则经有序搜索将找到最优解树。

例2.18 下面我们举例说明上述搜索过程。

设初始节点为S0，每次扩展两层，并设S_0经扩展后得到如图2–17(a)所示的与或树，其中子节点B，C，E，F用启发函数估算出的g值分别是

g(B)=3，g(C)=3，g(E)=3，g(F)=2

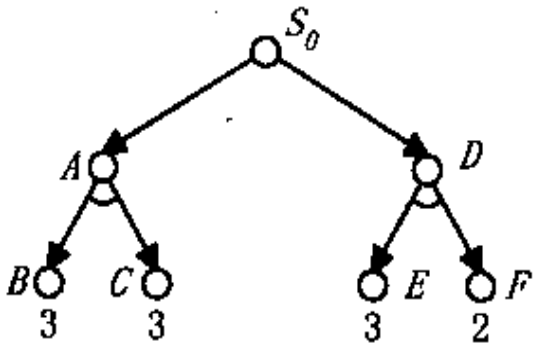
若按和代价计算，则得到

g(A)=8，g(D)=7，g(S0)=8

(注:这里把边代价一律按1计算)

此时S0的右子树是希望树。

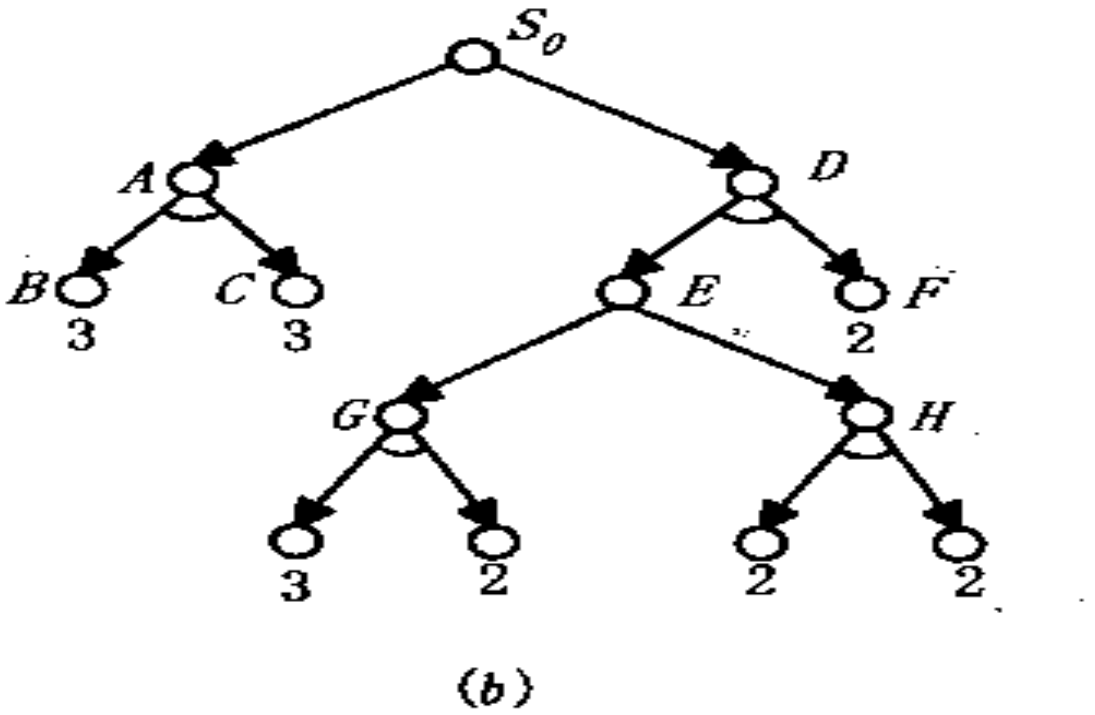
下面对此希望树的节点进行扩展。



(a)

设对节点E扩展两层后得到如图2–17(b)所示的与或树，节点旁的数字为用启发函数估算出的g值。则按和代价法计算得到 g(G)=7,g(H)=6,g(E)=7,g(D)=11

此时，由S_0的右子树算出的$g(S_0)=12$。但是，由左子树算出的$g(S_0)=9$。显然，左子树的代价小，所以现在改取左子树作为当前的希望树。

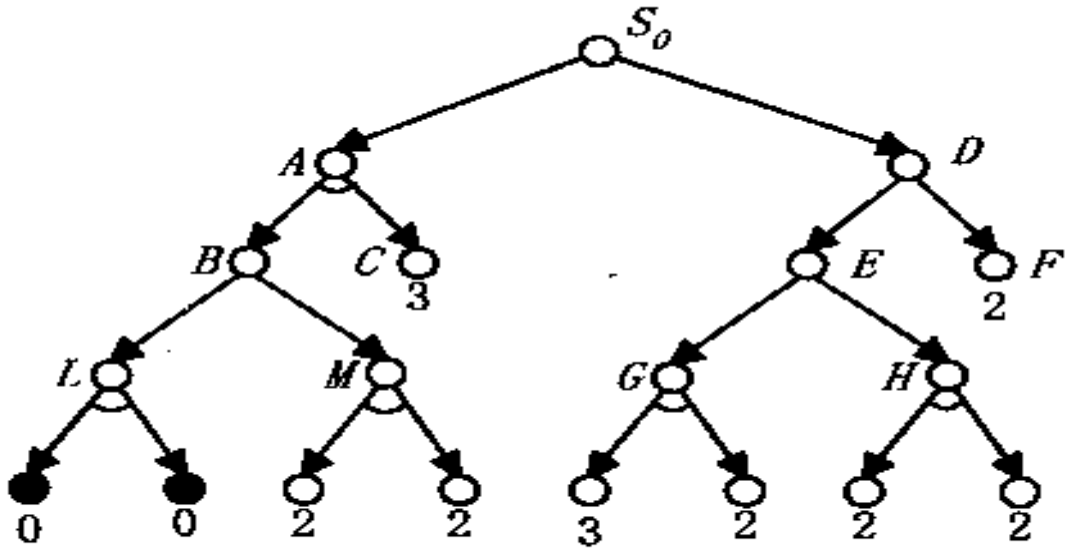


(b)

假设对节点B扩展两层后得到如图2–17(c)所示的与或树，节点旁的数字是对相应节点的估算值，节点L的两个子节点是终止节点，则按和代价法计算得到

$$g(L)=2,g(M)=6,g(B)=3,g(A)=8$$

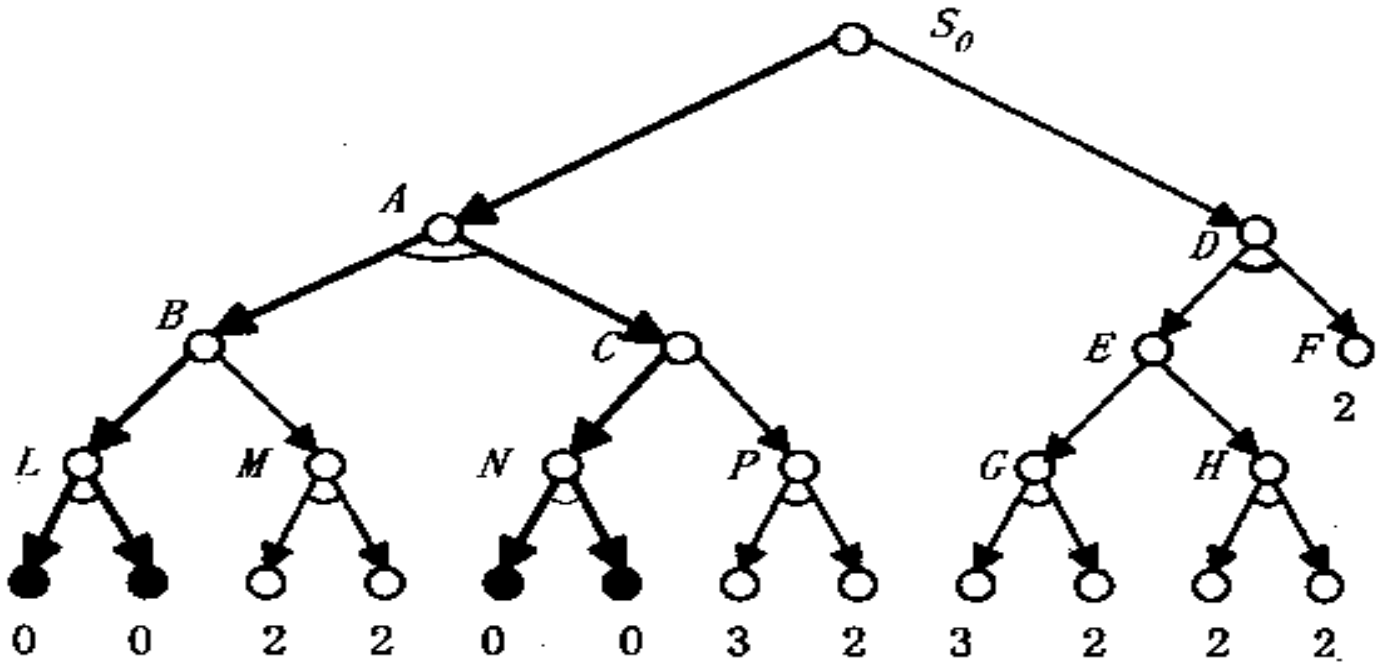
由此可推算出$g(S_0)=9$。这时，左子树仍然是希望树，继续对其扩展。该扩展节点C。



(c)

假设节点C扩展两层后得到如图2–17(d)所示的与或树，节点旁的数字是对相应节点的估算值，节点N的两个子节点是终止节点。按和代价计算得到

$$g(N)=2,g(P)=7,g(C)=3,g(A)=8$$



(d)

由此可推算出$g(S_0)=9$。另外，由于N的两个子节点都是终止节点，所以N和C都是可解节点。再由前面推出的B是可解节点，可推出A和S_0都是可解节点。这样就求出了代价最小的解树，即最优解树——图2–17(d)中粗线部分所示。该最优解树是用和代价法求出来的，解树的代价为9。

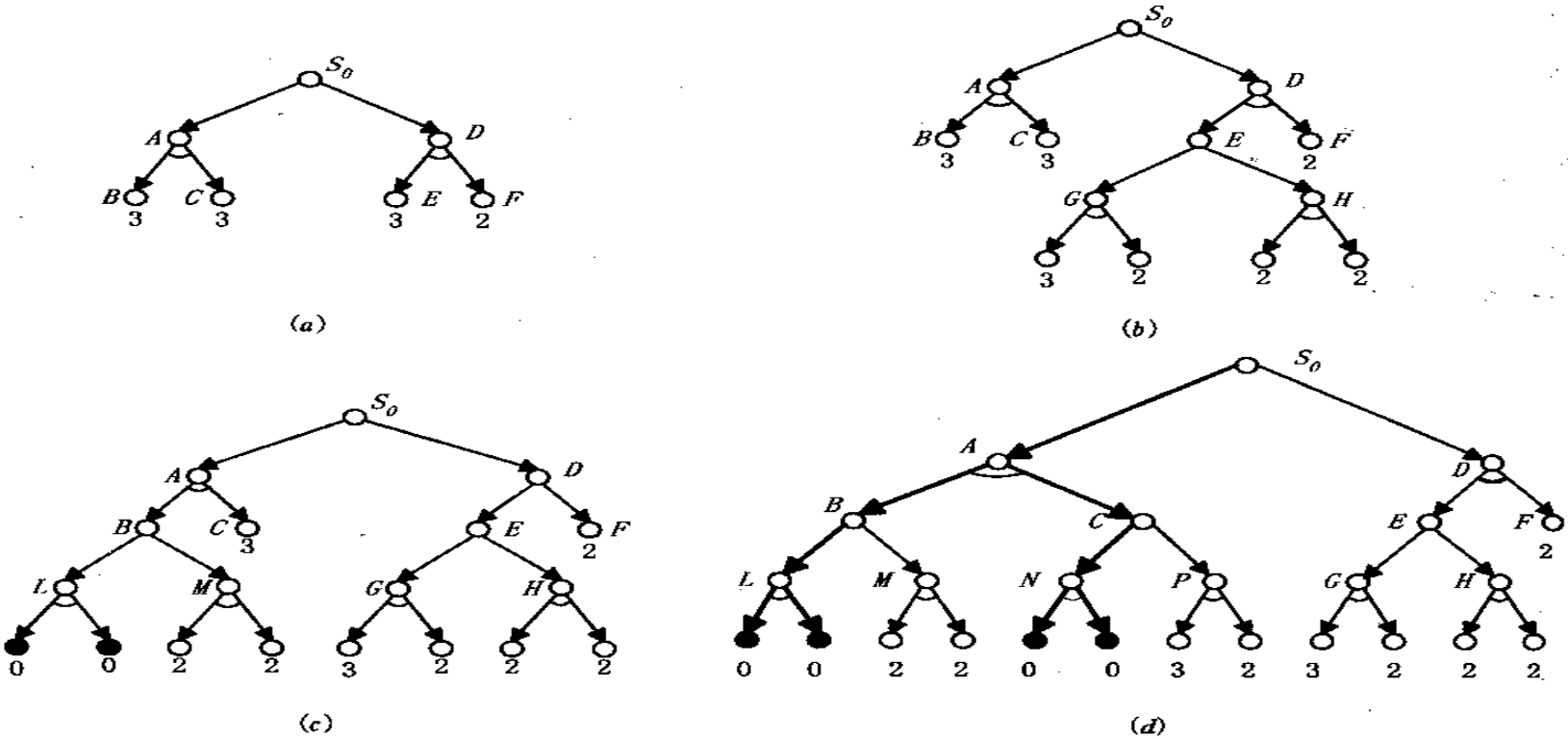


图2-17 与或树的启发式搜索例

与或树的最好优先搜索算法

(1)把初始节点S_0放入OPEN表中。

(2)求出希望树T，即根据当前搜索树中节点的代价g求出以S_0为根的希望树T。

(3)依次把OPEN表中T的端节点N选出放入CLOSED表中。

(4)如果节点N是终止节点，则做下列工作：

①标示N为可解节点。

②对T应用可解标记过程，把N的先辈节点中的可解节点都标记为可解节点。

③若初始节点S0能被标记为可解节点，则T就是最优解树，成功退出。

④否则，从OPEN表中删去具有可解先辈的所有节点。

(5)如果节点N不是终止节点，且它不可扩展，则做下列工作：

①标示N为不可解节点。

②对T应用不可解标记过程，把N的先辈节点中的不可解节点都标记为不可解节点。

③若初始节点S0也被标记为不可解节点，则失败退出。

④否则，从OPEN表中删去具有不可解先辈的所有节点。

(6)如果节点N不是终止节点，但它可扩展，则可做下列工作：

①扩展节点N，产生N的所有子节点。

②把这些子节点都放入OPEN表中，并为每一个子节点配置指向父节点（节点N）的指针。

③计算这些子节点的g值及其先辈节点的g值。

(7)转(2)。



2.5　博弈树搜索

2.5.1 博弈树的概念

在博弈过程中，任何一方都希望自己取得胜利。因此，当某一方当前有多个行动方案可供选择时，他总是挑选对自己最为有利而对对方最为不利的那个行动方案。

此时,如果我们站在某一方（如A方)的立场上，则可供A方选择的若干行动方案之间是“或”关系，因为主动权在A方手里，他可以选择其中任何一个方案。

同时，对方（B方）也有若干可供选择的行动方案，则对A方来说这些行动方案之间是“与”关系，因为这时主动权在B方手里，可供选择的行动方案中的任何一个都可能被B方选中，A方必须考虑到对自己最不利的情况的发生。



把上述博弈过程用图表示出来，则得到的是一棵“与或树”。描述博弈过程的与或树称为**博弈树**，它有如下特点：

(1)博弈的初始格局是初始节点。

(2)在博弈树中，“或”节点和“与”节点是逐层交替出现的。自己一方扩展的节点之间是“或”关系，对方扩展的节点之间是“与”关系。双方轮流地扩展节点。

(3)所有自己一方获胜的终局都是本原问题，相应的节点是可解节点；所有使对方获胜的终局都是不可解节点。



2.5.2 “二人零和、全信息、非偶然”博弈

诸如下棋、打牌、竞技、战争等一类竞争性智能活动称为博弈。其中最简单的一种称为“二人零和、全信息、非偶然”博弈。所谓“二人零和、全信息、非偶然”博弈是指：

(1)对垒的A，B双方轮流采取行动，博弈的结果只有三种情况：A方胜，B方败；B方胜，A方败；双方战成平局。

(2)在对垒过程中，任何一方都了解当前的格局及过去的历史。

(3) 任何一方在采取行动前都要根据当前的实际情况，进行得失分析，选取对自己最为有利而对对方最为不利的对策，不存在“碰运气”的偶然因素。即双方都是很理智地决定自己的行动。

(4) 对弈双方的利益是完全对立的，它们各有一策略集D_0, D_1，每次只能使用其中的一个策略对弈，得失由自己的赢得函数φ_i：$D_0 \times D_1 \rightarrow R$（i=0，1）来决定，并有$\psi_0+\psi_1 \equiv 0$。

中国象棋：

一盘棋平均走50步，总状态数约为10的161次方。假设1毫微秒走一步，约需10的145次方年。

结论：不可能穷举。

在博弈中通常使用的是保险策略。即每方都是立足于在最坏的情况下去争取最好的结局。对i方来说（i＝0，1），它按照

$$Max\,[\,min\ \varphi_i(d_0, d_1)]\quad (j = 1\text{-}i,\ d_0 \in D_0,\ d_1 \in D_l)$$

$$d_i \in D_i,\ d_j \in D_j$$

来选择自己的策略,若有$d_0^* \in D_0$，$d_1^* \in D_l$使得

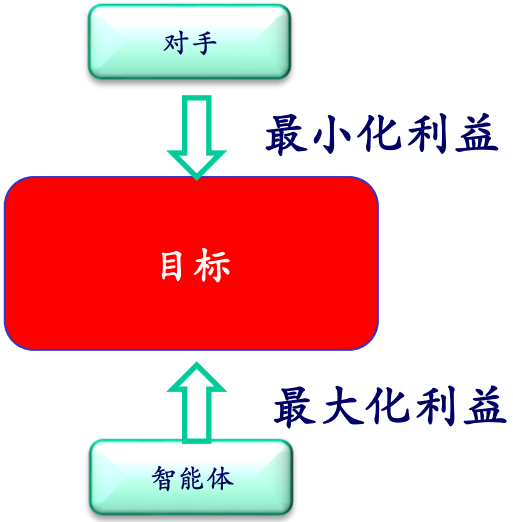
- $\varphi_0(d_0^*, d_1^*) = max\,[min\varphi_0(d_0, d_1)]\quad d_0 \in D_0,\ d_1 \in D_l$
- $\varphi_1(d_0^*, d_1^*) = max\,[min\varphi_1(d_0, d_1)]\quad d_1 \in D_1,\ d_0 \in D_0$

成立，则称(d_0^*, d_1^*)为博弈双方保险策略的平衡解。

2.5.3 极小极大分析法

两个智能体同处于一个竞争的环境，智能体之间通过竞争来实现各自相反利益，即一方想最大化自身的利益，而另一方则想最小化对手的利益。通俗地说，对抗搜索（adversarial search）的过程就是两个智能体各自选择对自己有利的策略。又叫Minimax搜索或博弈搜索（game search）

由于每个玩家都想要最大化自己的得分，所以轮到玩家A行动时，他会总是选择能够最大化终局得分的搜索树分支，而玩家B总是选择能够最小化终局得分的搜索树分支，最小最大算法的名字即由此而来



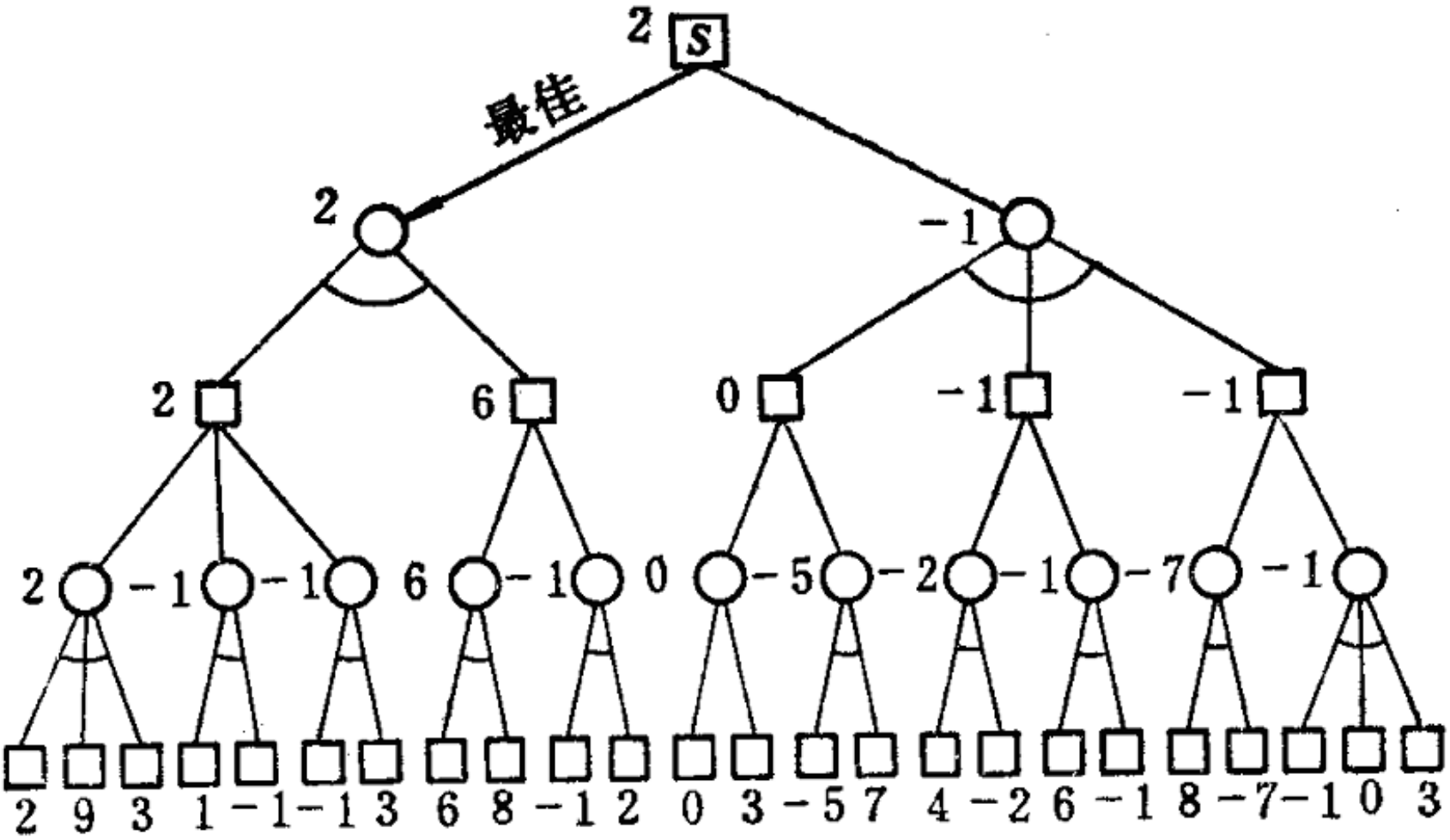


按照 MAX的观点看：

- 所有属于MIN的结点都是与结点。因为 MIN必然选择最不利于MAX的一着，只要该点的合法走法中有一着对MAX不利，该结点就对MAX不利。
- 所有属于 MAX的结点都是或结点。因为选择策略的主动权在 MAX手上，他必然选择最有利于自己的走法。只要该结点有一着对MAX有利，该结点就对 MAX是有利的。
- 所有使MAX获胜的终局都是本原问题，是“可解”的端结点，所有使MIN获胜的终局都是 “不可解”的端结点。设可解端结点得分为十b，不可解端结点得分为-b。
- 按照MAX和MIN的走法，从端结点向根部一步步推算出各中间结点和根结点的得分，叫做博弈树的极小极大化分析。



- 根据这些得分，MAX可以选择使自己获胜的每一步走法，MIN可以选择使对方失利的每一步走法。
- 为计算得分，需要根据问题的特性信息定义一个估价函数，用来估算当前博弈树端节点的得分。
- 当端节点的估值计算出来后，再推算出父节点的得分，推算的方法是：对“或”节点，选其子节点中一个最大的得分作为父节点的得分，这是为了使自己在可供选择的方案中选一个对自己最有利的方案；对“与”节点，选其子节点中一个最小的得分作为父节点的得分，这是为了立足于最坏的情况。
- 如果一个行动方案能获得较大的倒推值，则它就是当前最好的行动方案。



极小极大化分析



【例14】 一字棋游戏：在一个3X3的方格棋盘上，X、O两人轮流出子，每人每次可出一子占据一格，最先把自己的棋子布成三子一线者为胜，九格占满后仍分不出胜负者为和棋。在一字棋中，规定 MAX为X方，MIN为O方，X方先走，试用极小极大化分析法求双方的最佳策略。

解：这个问题的状态空间有9！个结点。除去同构的棋局，仍然是一个很大的数字，用强力搜索是很不现实的，需要用启发式搜索。给定启发函数h（n）（在此令表示得分的静态估值函数f(n) = h(n) ）后，就可以用极小极大化分析法求最佳走步。 我们称棋盘中的整行，或整列或整对角线为路。如果一条路上只有i方的棋子，则称该路为留给i方的路。

开局时，我们可以这样来定义 X方的静态得分估值函数 h_1（n）：

①如果n是非终局结点，则

$h_1(n)$＝（X方的路数一 O方的路数）

②如果n是和局，则　h_1（n）= t_l（n）= 0

③如果n是X方获胜的终局结点，则

h_1（n）= t_l（n）= 8

④ 如果n是O方获胜的终局结点，则

h_1（n）=t_l（n）＝－8

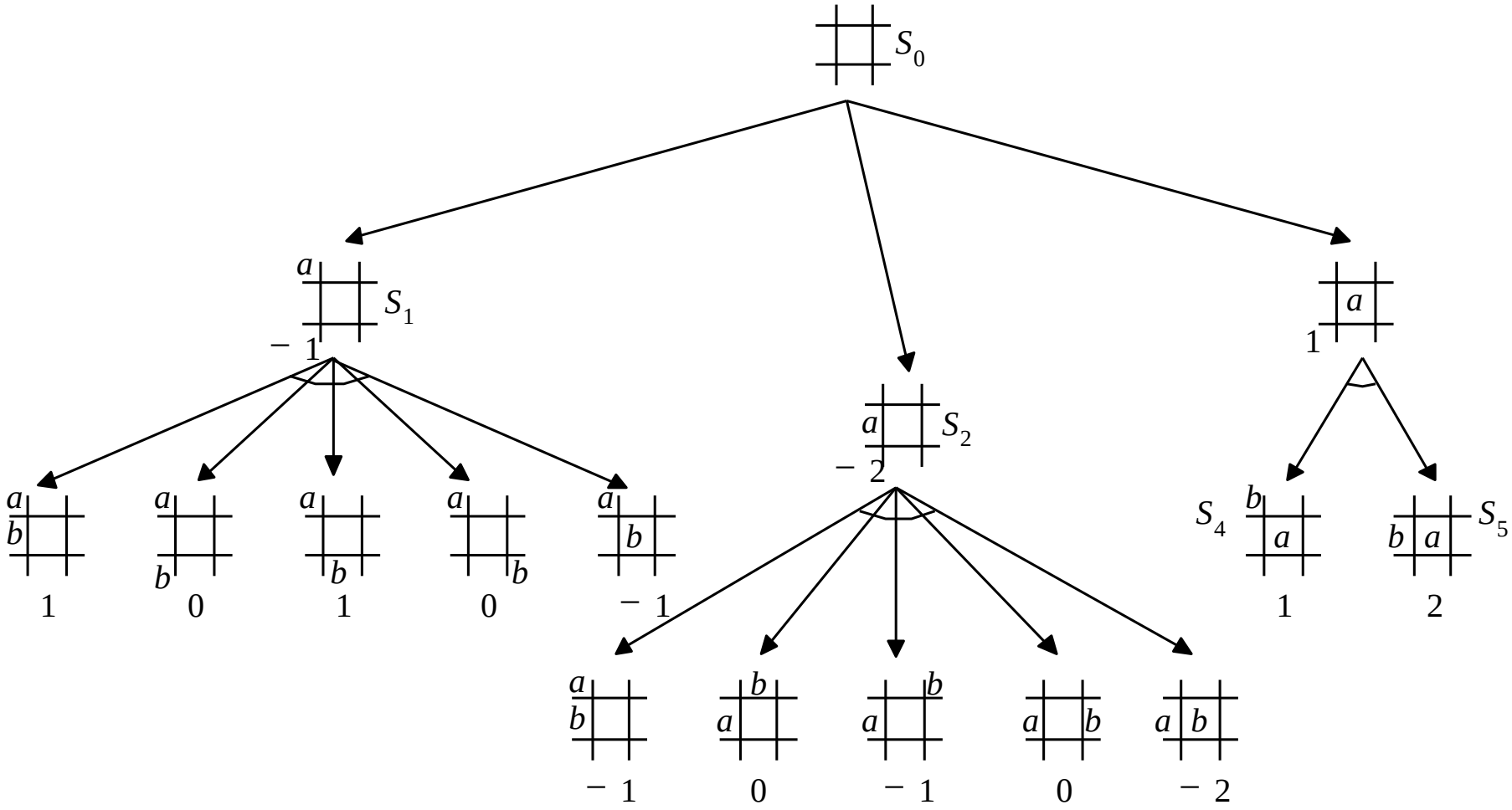
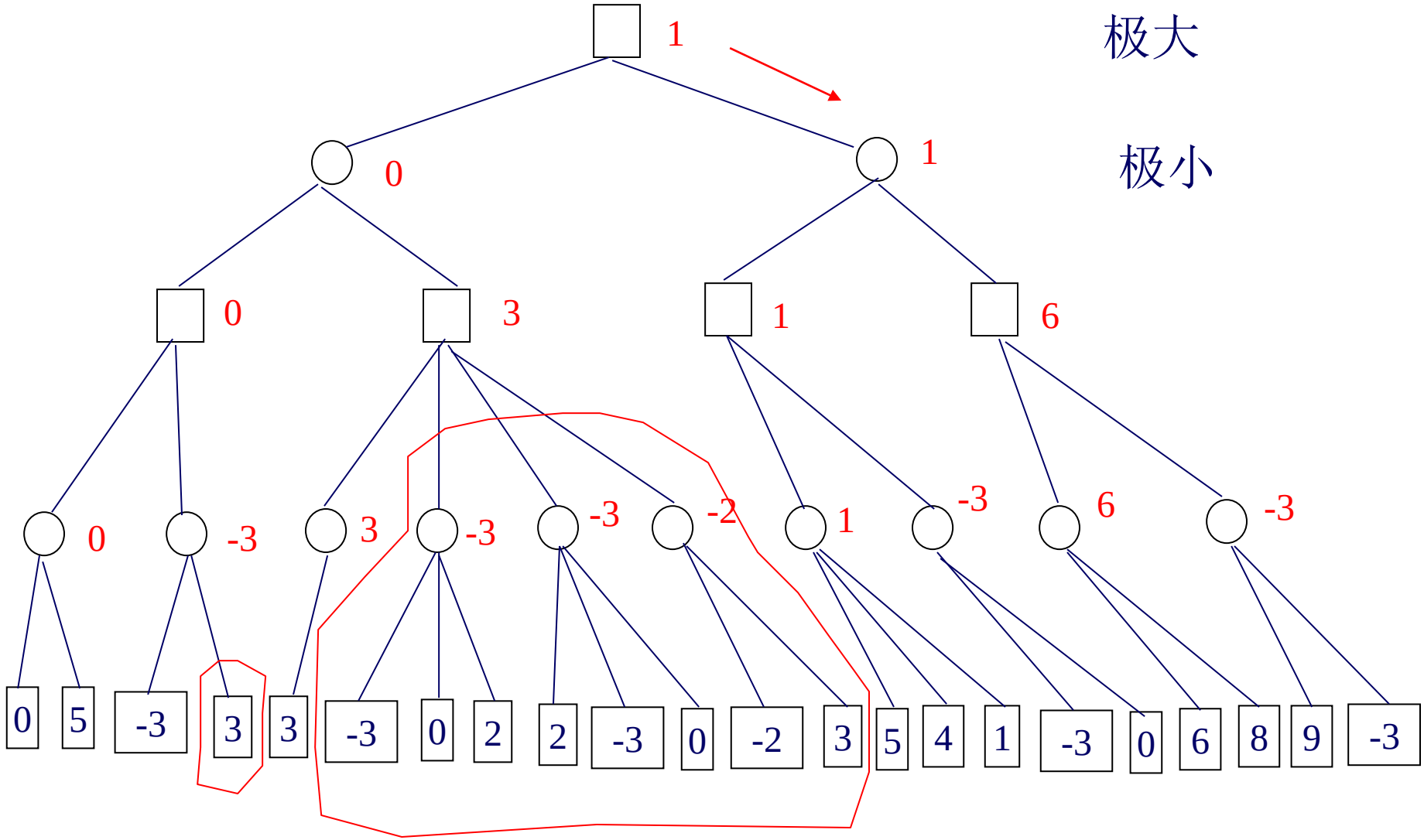


图 2-18一字棋极小极大搜索

极小极大过程



算法假设玩家MAX为初始行动玩家。这个假设并不会使问题失去一般性，如果实际上初始行动的是玩家MIN，则可以通过将终局得分取相反数的方法来交换玩家MIN和玩家MAX。函数MinValue和MaxValue递归求解了每个搜索树结点的分数，函数MinimaxDecision根据每个结点的分数求解玩家MAX在当前盘面下所采取的最优动作。

Minimax算法对游戏搜索树执行了一个完整的深度优先搜索。因此，如果搜索树是有限的，那么Minimax算法必然能在有限时间内终止；如果对手总是理性地按照最优策略来交替采取行动，那么Minimax算法得到的解是最优的。如果m是游戏树的最大深度，在每个结点最多存在b种有效走法，那么Minimax算法的时间复杂度为$O(b^m)$，空间复杂度为$O(bm)$。

算法 3.2　通过最小最大搜索求解最优动作的算法

函数：MinimaxDecision
输入：当前的盘面状态 s
输出：玩家 MAX 行动下，当前最优动作 a^*

```
a* ← argmax_{a∈actions(s)} MinValue(result(s, a))
```

函数：MaxValue
输入：当前的盘面状态 s
输出：玩家 MAX 行动下，当前状态的得分 $v = \text{minimax}(s, \text{MAX})$

```
if terminal_test(s) then return utility(s)
v ← −∞
foreach a ∈actions(s) do
    v ← max(v, MinValue(result(s, a)))
end
```

函数：MinValue
输入：当前的盘面状态 s
输出：玩家 MIN 行动下，当前状态的得分 $v = \text{minimax}(s, \text{MIN})$

```
if terminal_test(s) then return utility(s)
v ← +∞
foreach a ∈actions(s) do
    v ← min(v, MaxValue(result(s, a)))
end
```

在博弈过程中，对弈双方都力图在状态空间中寻找一条使自己得分最多的路径。即使一个很小的博弈状态空间，也可能生成十分庞大的博弈树。它可能有的终局数也十分庞大，企图对它用完整的极小极大分析来决定最佳策略是不可能的，实际可行的办法是在一个被分析的结点下面，只生成“似合理”的博弈树的一部分，并用静态估值函数f对树的各叶结点进行得分估值，然后按照极小极大分析计算树中各中间结点的得分估值，以便选择当前估计的最佳策略。这些计算所得的估值b(n)叫倒推估值。静态估值和倒推估值都是对某个结点的理论得分值得一种估计。注意，这里f代表得分而不是代价，所以越大越好。

突破组合爆炸之难

香农之数：在象棋比赛中，深蓝需要判断某一时刻棋局落子对整个棋局胜负会带来怎样的影响。由于可选棋局众多，尽管深蓝平均每秒能够对1亿个棋局进行判断评估，还是无法在规定时间内计算得到当前棋局对胜负的潜在影响。1950年，香农（Claud Shannon）发表了一篇有关国际象棋编程的论文。在这篇论文中，香农估算国际象棋比赛中落子选择从第一次移动时的20种会增加到第二次移动时的400种，在第六次移动时可能的落子选择达到1.19亿种。香农甚至估算认为国际象棋的落子总数为10的120次方种，远远超出了10的82次方这一宇宙原子总数，这就是国际象棋中“组合爆炸”难题。

剪枝--眼不见心不烦：为了从海量可能落子中选择一种合适落子，以克服组合爆炸挑战，深蓝采用了由1971年图灵奖获得者约翰·麦卡锡 (John McCarthy)发明的“阿尔法-贝塔（Alpha-Beta）”剪枝搜索算法。



2.5.4 α—β剪枝技术

上述的极小极大分析法，实际是先生成一棵博弈树，然后再计算其倒推值，它是将生成部分博弈树与计算估值完全分开进行的，这样做的缺点是效率较低。如果能边生成博弈树边计算估值，则可以减少许多搜索和计算的工作量。这个技术叫α–β剪枝技术。

该技术的基本思想是：边生成博弈树边计算评估各节点的倒推值，并且根据评估出的倒推值范围，及时停止扩展那些无必要再扩展的子节点，相当于减去了博弈树的一些分支，节省了机器开销，提高了搜索效率。

例1 下图所示的博弈树搜索, n就采用了α-β剪枝技术。

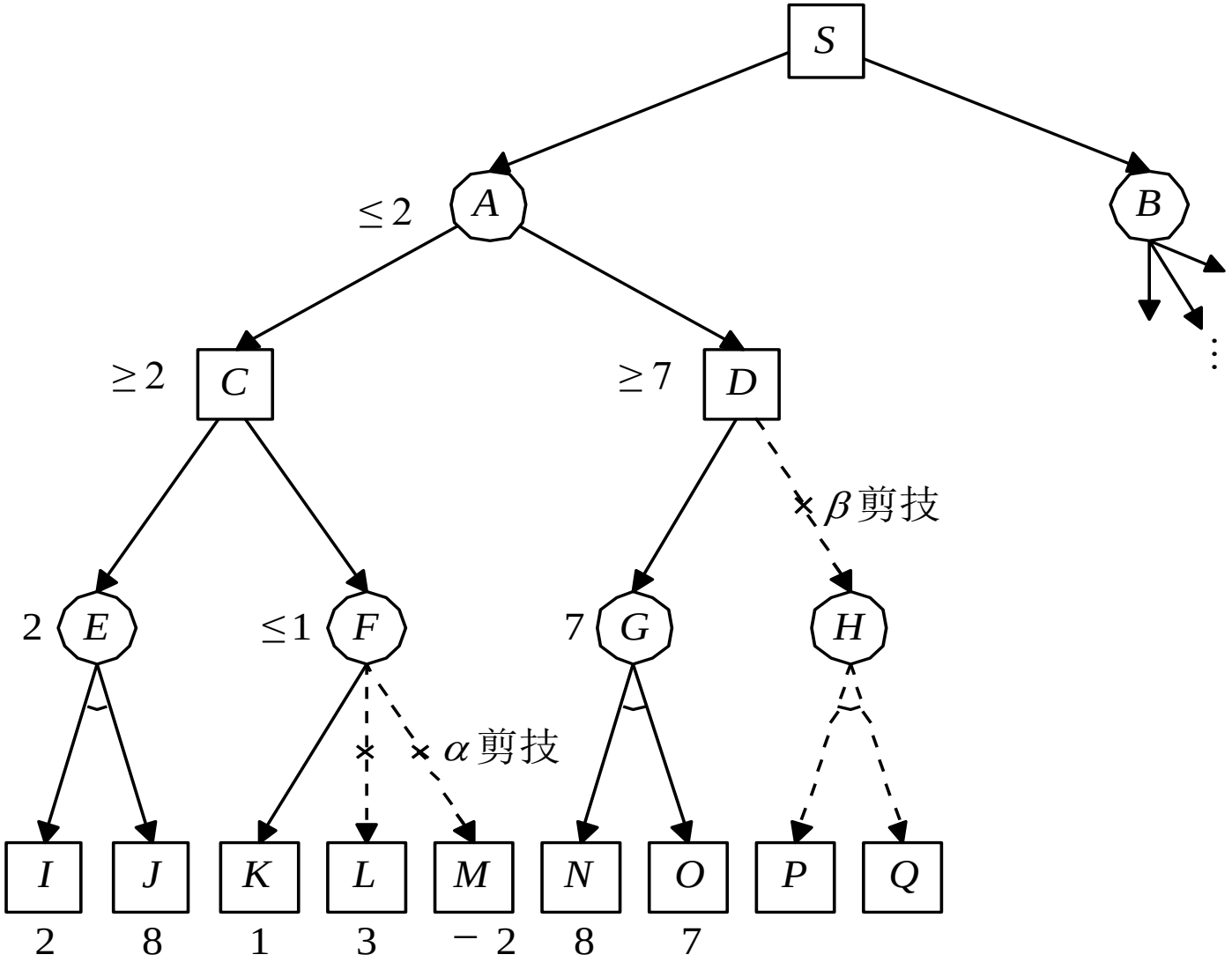


图2-20　α—β剪枝 例1

- 或节点的下界叫α值，与节点的上界叫β值。

剪枝的条件（具体的剪枝方法）：

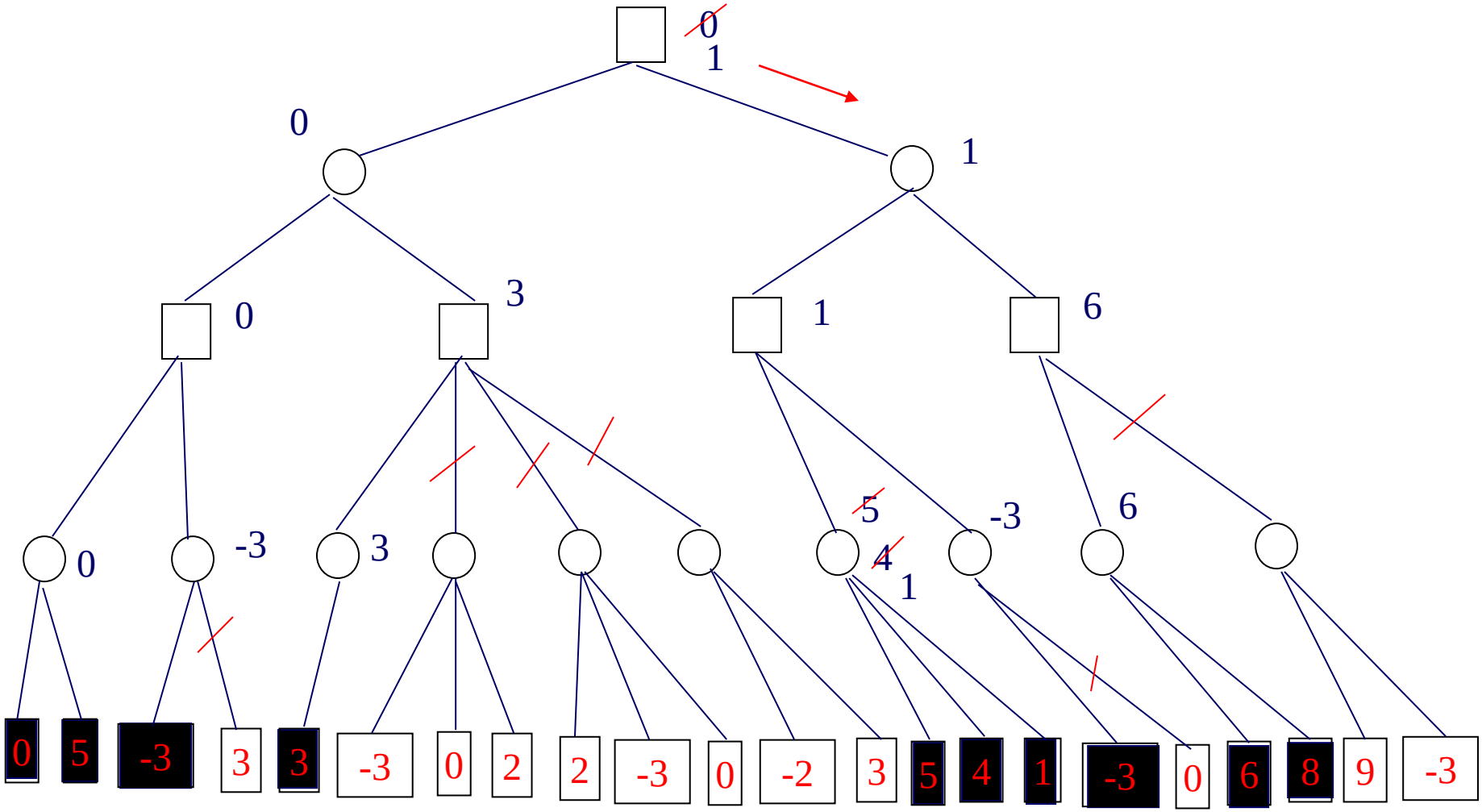
◆任何“或”节点x的α值如果不能降低其父节点的β值,则对节点x以下的分支可停止搜索，并使x的倒推值为α。这种剪枝称为**β剪枝。**

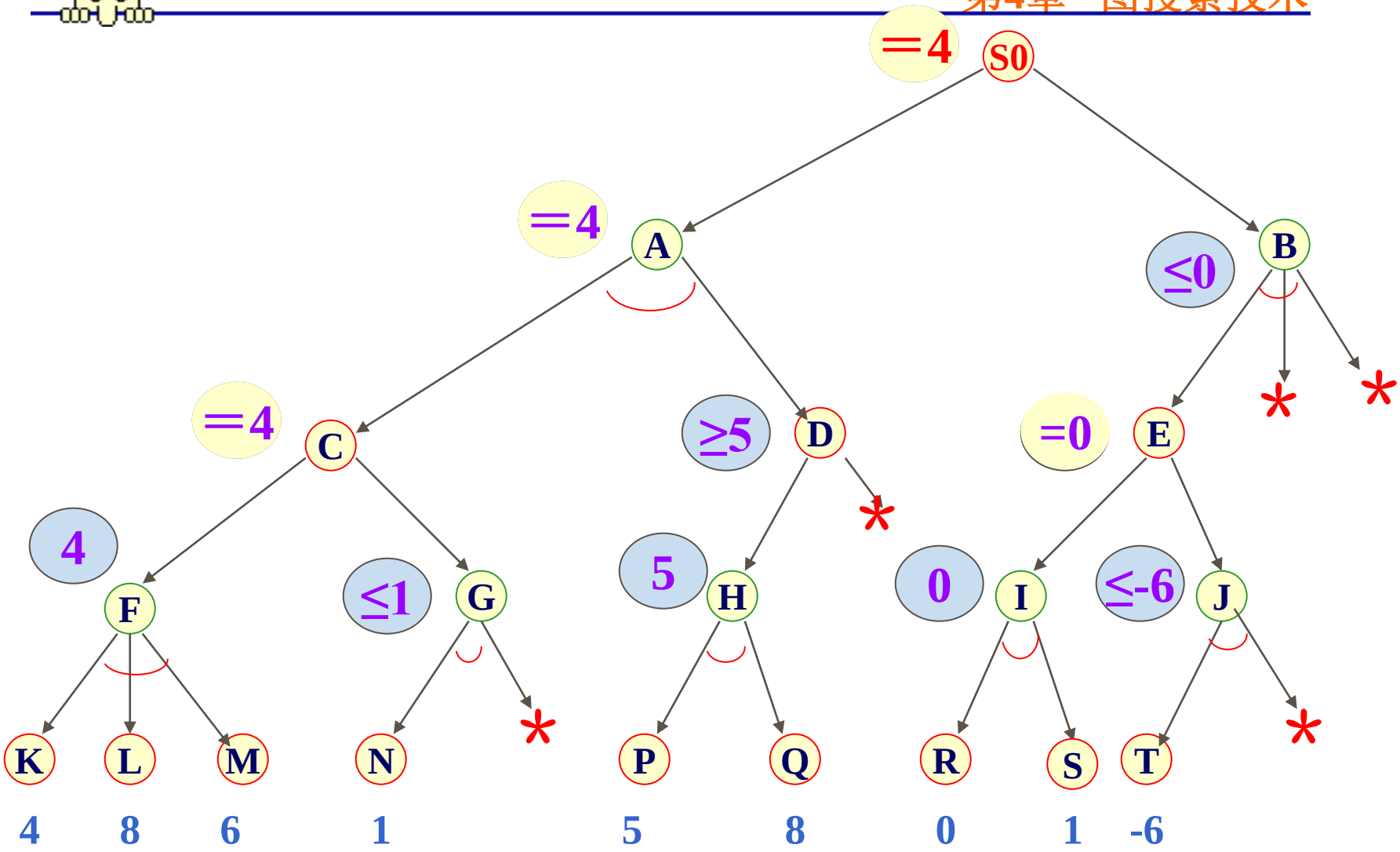
即后辈节点的α 值≥祖先节点的β值时， 则β剪枝

◆任何“与”节点x的β 值如果不能升高其父节点的α值,则对节点x以下的分支可停止搜索，并使x的倒推值为β 。这种剪枝称为**α 剪枝。**

即后辈节点的β值≤祖先节点的α值时，则 α剪枝

α-β剪枝例2





α-β剪枝例

AlphaBeta剪枝算法旨在减少在其搜索树中，被极大极小算法评估的节点数。这是一个常用人机游戏对抗的搜索算法。它的基本思想是对节点进行采样，而非逐一搜索，用剪枝的技术减少搜索的节点（根据上一层已经得到的当前最优结果，决定目前的搜索是否要继续下去）。

Alpha-Beta只能用递归来实现。这个思想是在搜索中传递两个值，第一个值是Alpha值，即搜索到的最好值，任何比它更小的值就没用了，因为任何小于或等于Alpha的值都不会有所提高。简单说就是max节点目前得到的最高得分（收益）。第二个值是Beta值，即对于对手来说最坏的值。这是对手所能承受的最坏的结果，因为我们知道在对手看来，他总是会找到一个对策不比Beta更坏的。简单来说就是min节点目前可给对手的最小收益。

α-β剪枝技术

- AlphaBeta剪枝算法是对Minimax方法的优化，它们产生的结果是完全相同的（即剪枝本身不影响算法输出结果），只不过运行效率不一样。
- **节点的先后排列顺序会影响α－β搜索的效率**。
- 如果节点次序“恰到好处”，则剪枝的时间复杂度为O（$b^{m/2}$），即搜索中子结点的产生顺序正好满足：或结点中估值最高的子结点最先产生；与结点中估值最低的子结点最先产生。则剪掉的结点数最多，产生的端结点数目最少，这就叫**最佳α－β搜索**。而之前的MiniMax搜索的时间复杂度为O（b^m）
- α－β剪枝的应用: 故障诊断, 风险投资， **IBM的深蓝就是采用剪枝技术的成功例子。**



2.6　蒙特卡洛树搜索

蒙特卡洛树搜索（Monte Carlo Tree Search,简称MCTS）是 Rémi Coulom 在 2006 年在它的围棋人机对战引擎(Crazy Stone)中首次发明并使用的，并且取得了很好的效果。它是一种用于决策制定和游戏策略的启发式搜索算法，特别适用于具有巨大搜索空间的游戏和问题，例如围棋、国际象棋和复杂的组合优化问题。

2.6.1 单一状态蒙特卡洛规划：多臂赌博机

2.6.2 上置信区间策略**UCB**

2.6.3 蒙特卡洛树搜索**(Upper Confidence Bounds on Trees_UCT)**

2.6.1 单一状态蒙特卡洛规划：多臂赌博机

多臂赌博机（Multi-armed Bandits）是强化学习中的经典问题，是一种序列决策问题，在有限的时间内，从多台赌博机（臂）中选择，以最大化累积奖励。

问题描述：假设智能体面前有K个赌博机，每个赌博机有一个臂膀。每次转动一个赌博机臂膀，赌博机则会随机吐出一些硬币或不吐出硬币，将每次吐出的硬币的币值表示为收益分数。现在假设智能体玩了T (T > K)次赌博机之后，下一次摇动哪个赌博机的臂膀会取得最大的回报？（即目标是在T次尝试中选择a_t，使得累积奖励R最大）

或许可以让智能体先把K个赌博机的臂膀依次转动一遍，观察在摇动每个赌博机臂膀时的收益分数，然后去转动那些收益分数高的赌博机臂膀。但是，由于从每个赌博机获得的收益分数是随机的，一个刚刚给用户带来可观收益分数的赌博机在下一次摇动其臂膀时可能不会继续获得可观收益分数，因此这并不可取。那么有什么方法能够指导智能体高效地获取收益分数呢？

蒙特卡洛树搜索：策略计算

智能体不能预先计算好最优策略，然后实行，而是需要一边探索一边调整自己的策略，争取获得更好的收益。

一种直观的做法是，智能体记录下每次摇动的赌博机臂膀和获得的相应收益分数。给定第$i(1 \le i \le K)$个赌博机，记在过去$t-1$次摇动赌博机臂膀的行动中，一共摇动第i个赌博机臂膀的次数为第$T_{(i,t-1)}$。于是，可以计算得到第i个赌博机在过去$T_{(i,t-1)}$次被摇动过程中的收益分数平均值$\bar{x}_{i,T_{(i,t-1)}}$。这样，智能体在第t步，只要选择$\bar{x}_{i,T_{(i,t-1)}}$值最大的赌博机臂膀进行摇动，这是贪心算法的思路。

假设一共有五台赌博机（即$K=5$），第i个赌博机的得分满足均值为μ_i、方差为1/75（即支撑区间长度为0.4）的均匀分布，其中均值$(\mu_1,\cdots,\mu_5)=(0.3,0.4,0.5,0.6,0.7)$。假设某个赌博机的臂膀没有被摇动时，因为无法计算其平均收益分数，不妨将这个赌博机得分均值设定正无穷大。在计算机上模拟执行$T=10000$次贪心算法，来摇动五台赌博机。可见在10000次模拟中，智能体趋向于摇动4号赌博机，其他编号赌博机被摇动次数很少。由于其他编号赌博机被摇动次数较少，可见对这些赌博机平均收益分数的估计结果与实际均值结果相差很大。如果重新进行一个10000次贪心算法的模拟，由于随机性的存在，其结果会存在不同。比如在另外一个10000次模拟中，贪心算法首先选择编号为2的赌博机臂膀摇动多次，则会导致后续行动中会对编号为2的赌博机摇动次数最多，忽略了其他编号的赌博机。

蒙特卡洛树搜索：盲从过去历史而导致不足

在上面的实验中，可以发现：贪心算法对编号4的赌博机“情有独钟”，很少去摇动其他编号的赌博机。因此，虽然对编号4赌博机的平均收益分数估计越来越准确，但是却难以尝试去摇动其他编号赌博机的臂膀，从而无法对其他赌博机的平均收益分数就行准确估计。

上述困境体现了探索（exploration）和利用（exploitation）之间存在对立关系。贪心算法基本上是**利用**从已有尝试结果中所得估计来指导后续动作（如倾向于选择4号赌博机），但问题是所得估计往往不能准确反映未被（大量）探索过的动作，如由于很少摇动其他编号的赌博机而无法准确估计其他编号赌博机可能带来的收益分数。因此，需要在贪心算法中增加一个能够改变其“惯性”的内在动力，以使得贪心算法能够访问那些尚未被（充分）访问过的空间。

UCB算法试图进行探索与利用的权衡

在多臂赌博机问题中，需要在探索（尝试不同的臂以了解其潜在奖励）和利用（选择当前估计最优的臂以获取高奖励）之间取得平衡。

利用(Exploitation)——过去决策中得到的最佳回报；

探索(Exploration)——希望在未来从很少摇动或没有摇动的赌博机得到更大回报。

也就是说，在第t时刻，UCB算法一般会选择具有如下最大值的第j个赌博机：

$$UCB = \bar{X}_j + \sqrt{\frac{2In\,n}{n_j}} \text{ 或者 } UCB = \bar{X}_j + C \times \sqrt{\frac{2In\,n}{n_j}}$$

$\bar{X}_j$是第j个赌博机在过去时间内所获得的平均奖赏值，n_j是在过去时间内拉动第j个赌博机臂膀的总次数，n是过去时间内拉动所有赌博机臂膀的总次数。C 是一个平衡因子，其决定着在选择时偏重探索还是利用。

从这里可看出UCB算法如何在探索-利用（exploration-exploitation）之间寻找平衡：既需要拉动在过去时间内获得最大平均奖赏的赌博机，又希望去选择拉动臂膀次数最少的赌博机。

2.6.3 蒙特卡洛树搜索

现有的搜索算法中，对搜索算法进行优化以提高搜索效率基本上是在解决如下两个问题：优先扩展哪些结点以及放弃扩展哪些结点，综合来看也可以概括为如何高效地扩展搜索树。如果将目标稍微降低，改为求解一个近似最优解，则上述问题可以看成是如下探索性问题：算法从根结点开始，每一步动作为选择（在非叶子结点）或扩展（在叶子结点）一个孩子结点。可以用执行该动作后所收获奖励来判断该动作优劣。奖励可以根据从当前结点出发到达目标路径的代价或游戏终局分数来定义。算法会倾向于扩展获得奖励较高的结点。

蒙特卡洛树搜索通过在决策树中进行随机模拟，逐步构建搜索树，是从上向下搜索未扩展的叶子节点；然后在其下添加分支进行扩展；再模拟对弈得到胜负结果；最后将胜负结果从下往上反传到根节点。反复的进行，直到终止。最终这步棋的最优走法在第二层，取模拟次数最大者。

我们回想一下我们下棋时的思维——并没有在脑海里面把所有可能列出来，而是根据「棋感」在脑海里大致筛选出了几种「最可能」的走法，然后再想走了这几种走法之后对手「最可能」的走法，然后再想自己接下来「最可能」的走法。这其实就是 MCTS 算法的设计思路。

蒙特卡洛树搜索算法的关键步骤

- **选择（Selection）**：从搜索树的根结点开始，向下递归选择子结点，直至到达叶子结点或者到达尚未被完全扩展的的结点L，如图(a)。这个可通过UCB算法来实现，选择最具“潜力”的后续节点。过程中记录下每个结点被选择次数和每个结点得到的奖励均值。

- **扩展（Expansion）**：如果结点 L 不是一个终止结点（或对抗搜索的终局结点），则随机扩展它的一个未被扩展过的后继结点M，如图 (b)。

- **模拟（simulation）**：

从结点M出发，模拟扩展搜索树，直到找到一个终止结点，如图 (c)所示。模拟借鉴了我们上面说的蒙特卡洛方法，快速走子，*只走一盘*，分出个胜负。我们每个节点（每个节点代表每个不同的局面）都有两个值，代表这个节点以及它的子节点模拟的次数和赢的次数，比如模拟了 10 次，赢了 4 盘，记为 4/10。

- **回溯（Back Propagation）**：

回溯更新模拟路径中M以上（含M）结点的奖励均值和被访问次数，如图(d)所示。将模拟的结果反馈到树中，更新节点的统计信息，如访问次数、奖励等。这个过程会一直重复，直到达到预设的停止条件，如时间限制、模拟次数等。

UCB应用于游戏树就叫UCT，它平衡了探索（选择访问次数少的节点）和利用（选择具有高胜率的节点）。该考虑了两个因素，一个是获胜率，另一个是模拟对弈次数。获胜率高，当然不错，但如果模拟次数少，偶然性会很大；而模拟次数少的，对其了解不多，可能会是非常优秀的走法，值得进一步深入探索。c越大，就越偏向于模拟次数少的节点；c越小，就越偏向于胜率高的节点。如何平衡这个关系设定，要凭经验和根据问题的需要。其中， wi 是i节点的取胜次数， ni 是i节点的模拟对弈次数， Ni 是其父节点的全部模拟次数，c是探索常数也叫平衡因子，通常取c= 2 。

$$\text{score} = \frac{w_i}{n_i} + c\sqrt{\frac{\ln N_i}{n_i}}$$

$$\mathbb{UCT}(v_i, v) = \frac{Q(v_i)}{N(v_i)} + c\sqrt{\frac{\log(N(v))}{N(v_i)}}$$

知乎 @胡祖俊

Q(v) 是该节点赢的次数，N(v) 是该节点模拟的次数，C 是一个常数。因此我们每次选择的过程如下——从根节点出发，遵循最大最小原则，每次选择己方 UCT 值最优的一个节点，向下搜索，直到找到一个「未完全展开的节点」，根据我们上面的定义，未完全展开的节点一定有未访问的子节点，随便选一个进行扩展。

蒙特卡洛树搜索

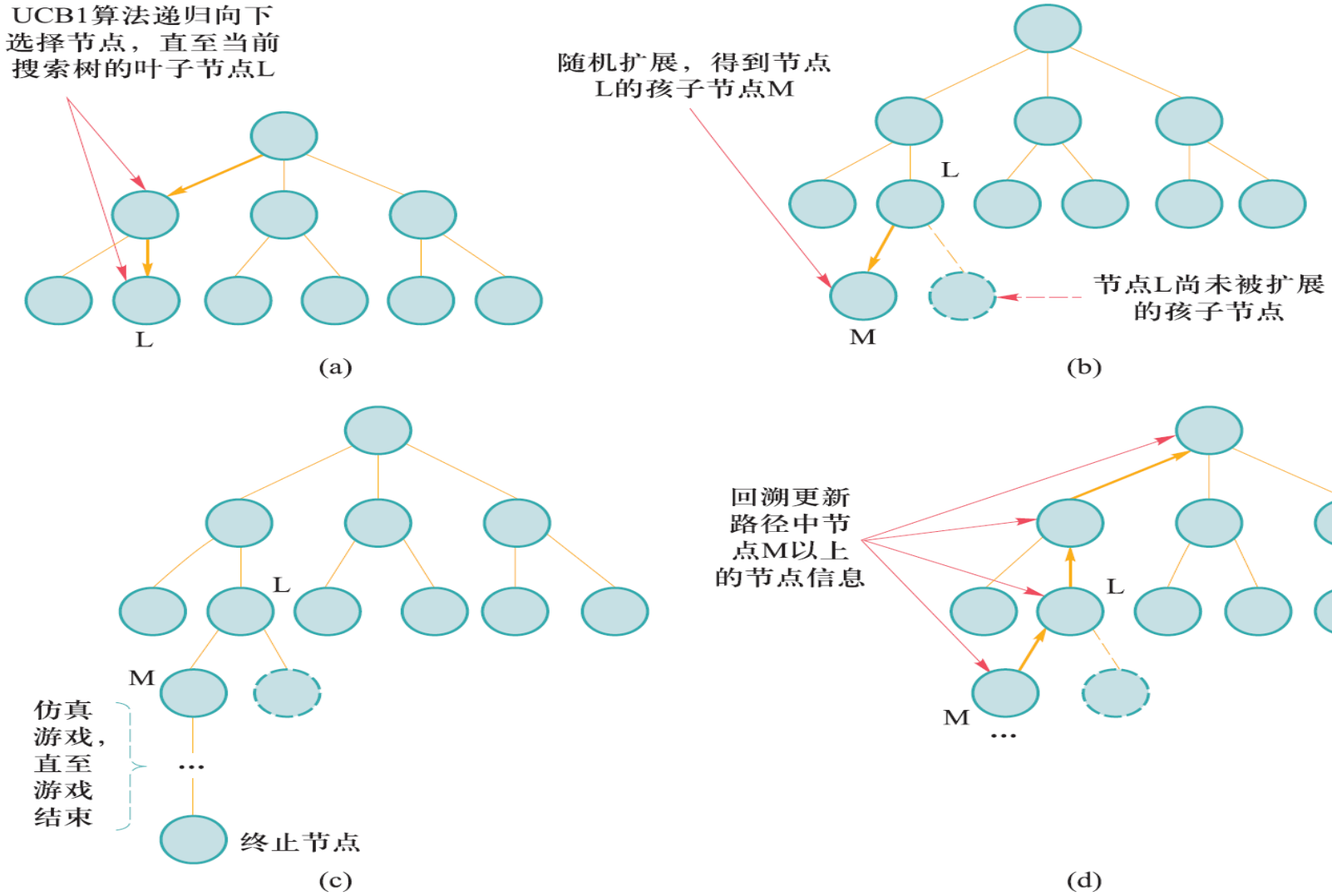


图 3.20　蒙特卡洛树搜索的四个步骤

蒙特卡洛树搜索

蒙特卡洛树搜索算法也被称为上限置信区间树搜索（Upper Confidence Bounds for Trees, UCT）算法。图3.21展示了蒙特卡洛树搜索的一次迭代中四个完整步骤，其中结点中数字记录了经过该结点所得收益分数之和（左侧数字）和访问该结点的次数（右侧数字）。

在图 (a)中，搜索树中已经有三个结点，注意此时根结点的分数之和为负数，这是用蒙特卡洛树搜索求解对抗搜索问题时的特殊措施，原因我们将在讨论反向传播步骤时提到。此时分别计算第二层结点的UCB值：左侧结点为$\frac{10}{1}+\sqrt{\frac{2\ln 2}{1}}=11.18$，右侧结点为$\frac{5}{1}+\sqrt{\frac{2\ln 2}{1}}=6.18$，因此算法选择第二层左侧的结点L，由于该结点有尚未扩展的子结点，因此选择阶段结束。

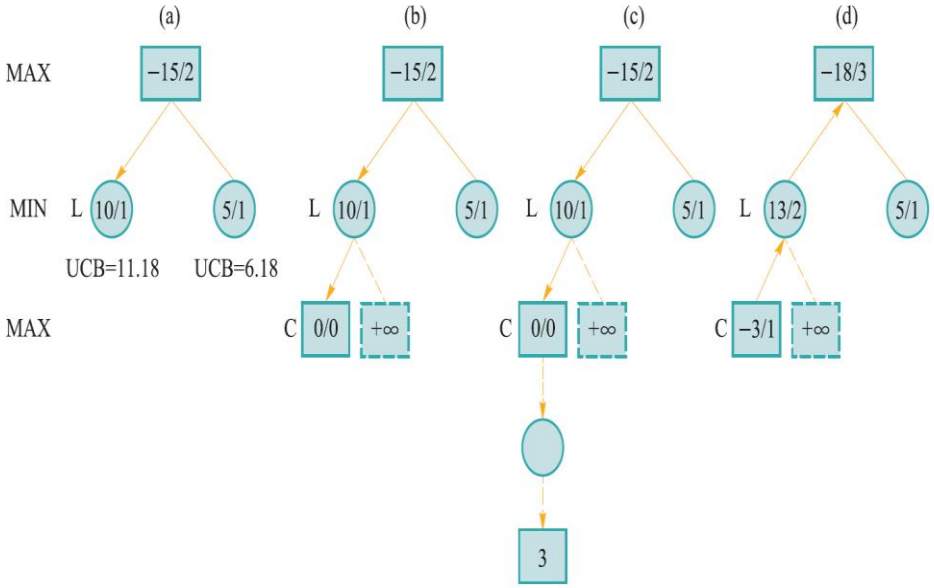


图 3.21　蒙特卡洛树搜索算法

（注：结点中的数字代表总收益分数 / 被访问次数）

蒙特卡洛树搜索算法。结点中数字代表总收益分数/被访问次数

蒙特卡洛树搜索

在图 (b)中，算法随机扩展了L的子结点C，将其总分数和被访问次数均初始化为0。注意，为了清晰地展示算法选择扩展的结点，图(b)画出了L的其他未被扩展的子结点，并标记其UCB值为正无穷大，以表示算法下次访问到L时必然扩展这些未被扩展的结点。图(c)中采用随机策略模拟游戏直至完成游戏。当游戏完成时，终局得分为3。

在图 (d)中C结点的总分被更新为-3，被访问次数被更新为1；L结点的总分被更新为13，被访问次数被更新为2；根结点的总分被更新为-18，被访问次数被更新为3。在更新时，会将MIN层结点现有总分加上终局得分分数，MAX层结点现有总分减去终局得分分数。这是因为在对抗搜索中，玩家MIN总是期望最小化终局得分，因此在MIN层选择其子结点时，其目标并非选取奖励最大化的子结点，而是选择奖励最小化的结点，为了统一使用UCB1算法求解，算法将MIN层的子结点（即MAX层结点）的总分记为其相反数。

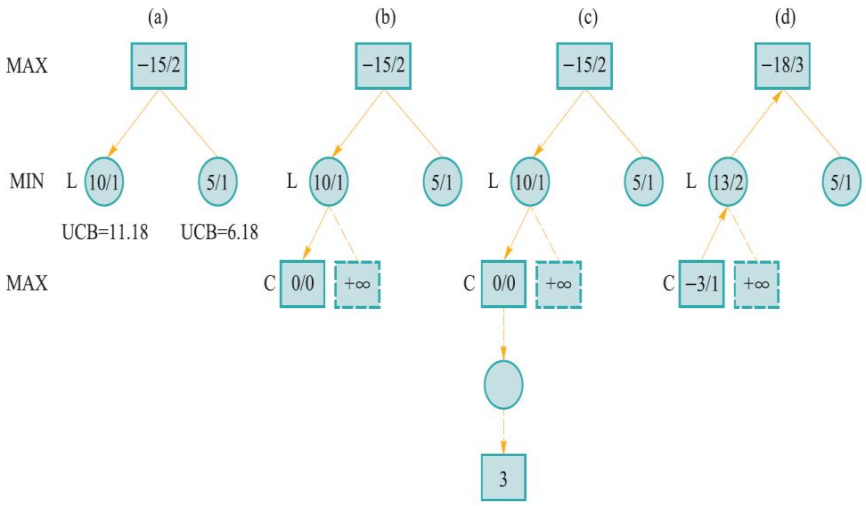
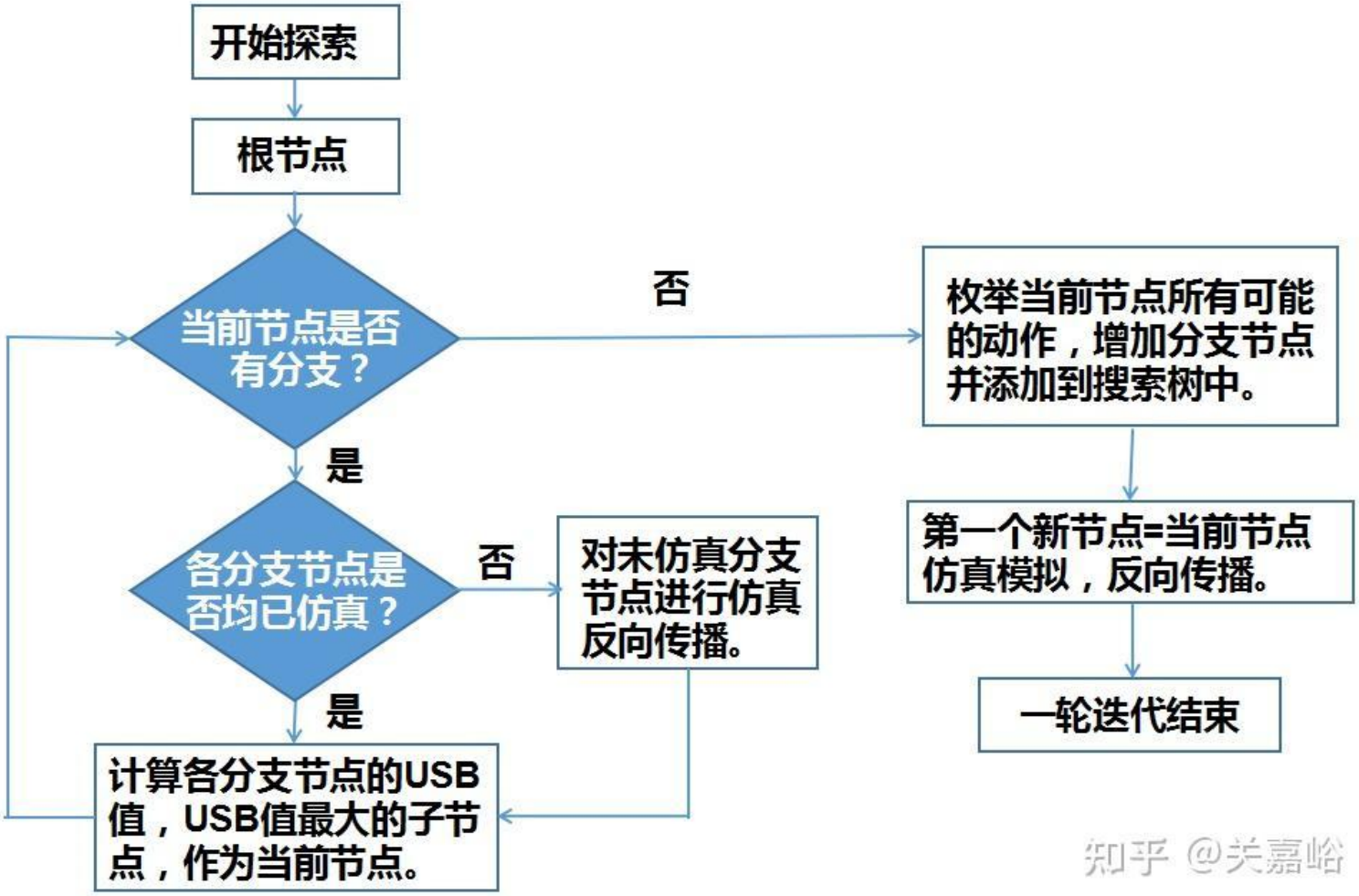


图 3.21　蒙特卡洛树搜索算法

（注：结点中的数字代表总收益分数 / 被访问次数）

蒙特卡洛树搜索算法。结点中数字代表总收益分数/被访问次数

前面的Minimax和剪枝都是穷举整个游戏树搜索过程，本质上都尽可能访问那些所有应该被访问的节点。但是围棋局面巨大，试图访问所有节点是不现实的。利用UCB算法可以随机创建未访问的节点，通过采样的非穷举式方式完成了游戏的对决。



知乎 @关嘉峪

蒙特卡洛树搜索

算法 3.4 基于最小最大搜索的蒙特卡洛树搜索算法

函数： UCTSearch
输入： 当前状态 s_0
输出： 玩家 MAX 行动下，当前最优动作 a^*

```
v0 ← create_node(s0)
while 未达到最大迭代次数 do
    vl ← SelectPolicy(v0)
    st ← SimulatePolicy(vl.state)
    BackPropagate(vl, st)
end
a* ← UCB1(v0, 0)
```

函数： SelectPolicy
输入： 选择的起始结点 v_0
输出： 选择步骤的结束结点 v

```
v ← v0
while not terminal_test(v.state) do
    if v 存在未被扩展的子结点 then
        return Expand(v)
    else
        v ← UCB1(v, Cp)
    end
end
```

函数： SimulatePolicy
输入： 状态 s_0
输出： 模拟的终止状态 s

```
s ← s0
while not terminal_test(s) do
    a ←从 actions(s) 中随机采样
    s ← result(s, a)
end
```

函数： BackPropagate
输入： 反向传播更新的起始结点 v，终局状态 s_t

```
while v is not null do
    v.N ← v.N + 1
    v.Q ← v.Q−utility(st, player(v.state))
    v ← v.parent
end
```

函数： Expand
输入： 结点 v
输出： 未被扩展的后继结点 v'

```
a ← actions(v.state) 中随机的未探索动作
v' ← create_node(result(v.state, a))
v'.N ← 0
v'.Q ← 0
v'.parent ← v
v'.children ← ∅
v.children ← v.children∪{v'}
```

函数： UCB1
输入： 结点 v，超参数 c
输出： 置信上限最大的动作 a^*

$$a^* \leftarrow \operatorname{argmax}_{v' \in v.children} \frac{v'.Q}{v'.N} + c\sqrt{\frac{2\ln v.N}{v'.N}}$$

蒙特卡洛树搜索

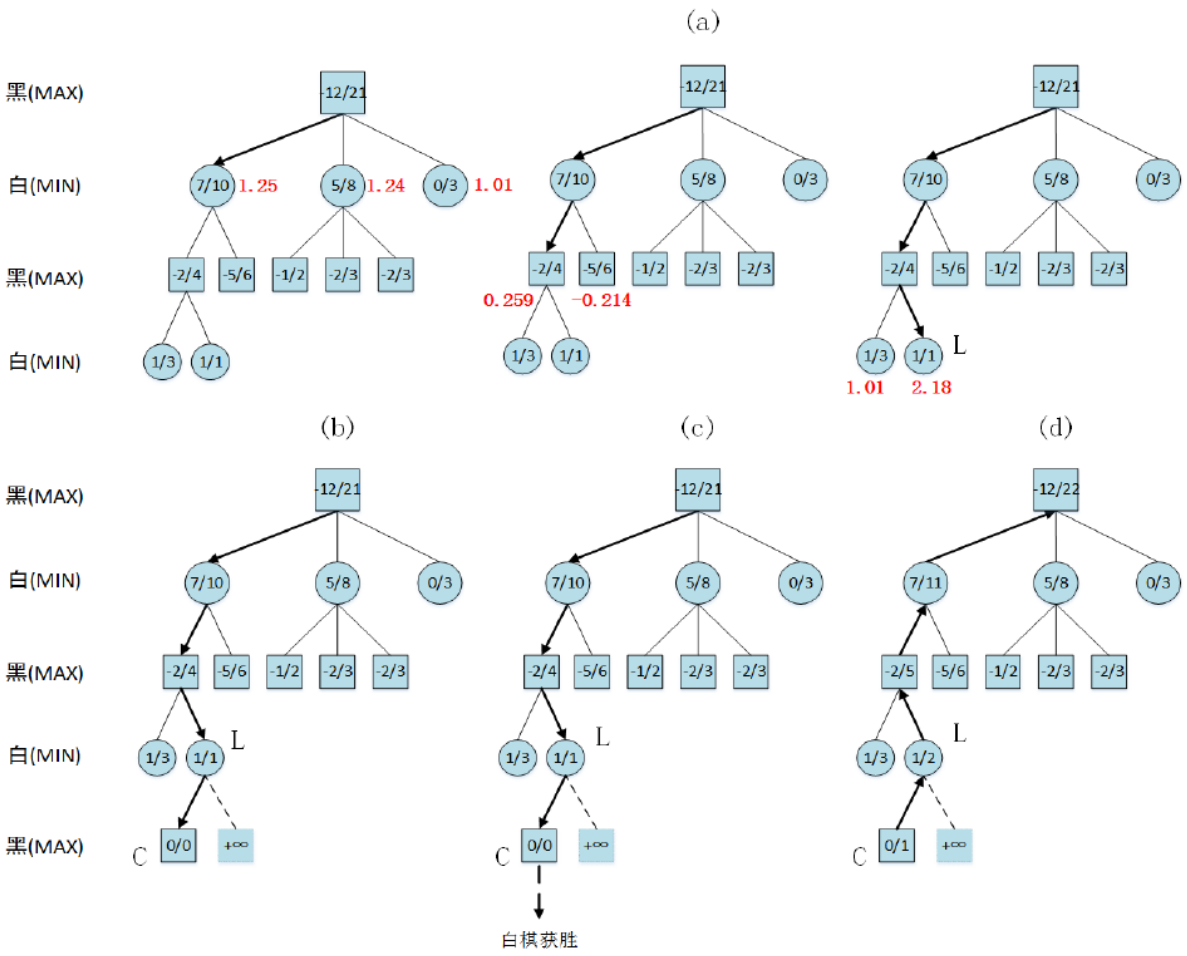
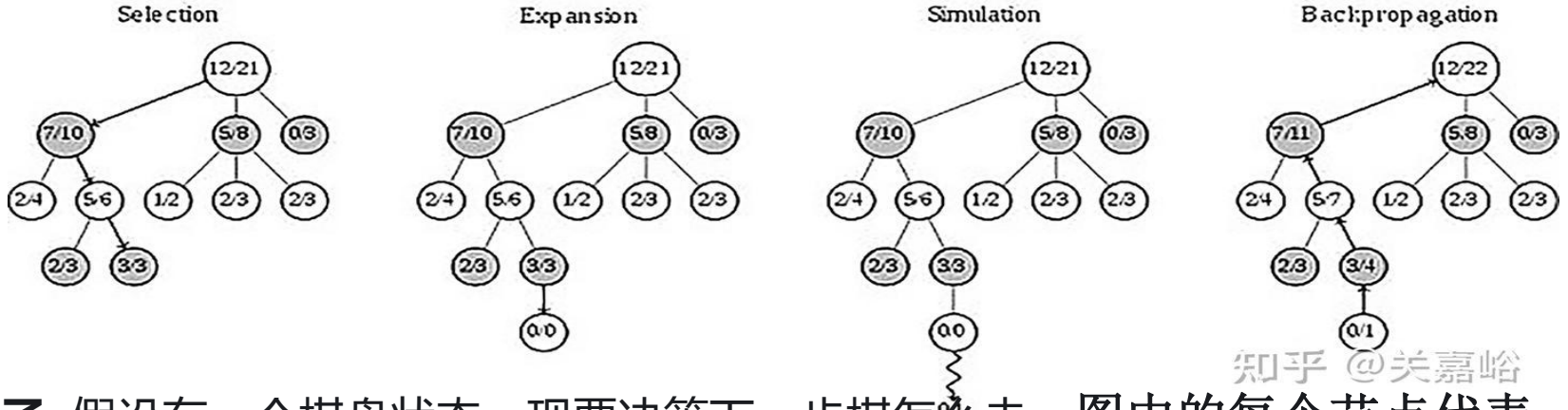


图3.22　一个更复杂的蒙特卡洛树搜索的例子
(UCB值在对应结点边用红色字体标出)



知乎 @关嘉峪

例子 假设有一个棋盘状态，现要决策下一步棋怎么走。图中的每个节点代表一个棋盘状态，节点中数据A/B，表示这个节点被模拟对弈了B次，胜了A次。比如，根节点是12/21，表示总共模拟了21 次，胜利了12 次。

（1）选择　用UCB公式计算节点 7/10、5/8、0/3 的分数（score）：分数最大的是1.25，对应的节点是节点7/10，即第二层左边第一个节点7/10，将其作为当前节点。

$$7/10\text{对应的分数：}\quad 7/10+\sqrt{\frac{\log(21)}{10}}=0.7+0.55=1.25$$

$$5/8\ \text{对应的分数：}\quad 5/8+\sqrt{\frac{\log(21)}{8}}=0.625+0.62=1.245$$

$$0/3\ \text{对应的分数：}\quad 0/3+\sqrt{\frac{\log(21)}{3}}=0+1.00=1$$

知乎 @关嘉峪

继续向下探索选择：节点7/10下面有分支节点 2/4 和 5/6 ，应该在它们之中选择。两个节点都仿真过，所以用UCB公式计算：5/6的分数大，应该选5/6作为当前节点。

$$2/4\text{对应的分数：}\quad 2/4+\sqrt{\frac{\log(10)}{4}}=0.5+0.76=1.26$$

$$5/6\text{对应的分数：}\quad 5/6+\sqrt{\frac{\log(10)}{6}}=0.79+0.67=1.46$$

知乎 @吴嘉皓

继续向下搜索，当前5/6节点下有分支，仍用USB公式计算，再选。假设节点3/3的USB值大，则选它为当前节点。

（2）扩展（Expansion）

因为节点3/3下面没有分支（已经是叶子节点），所以在其下添加新的分支 （比如，把节点3/3当前的状态输入策略神经网络，网络通过计算，可以输出此状态下的多个比较好的动作。）在叶子节点 3/3下添加子节点，图中只画出了其中的一个。该节点里的数据当然是 0/0。这个增加分支节点的过程就是扩展。

（3）仿真（模拟对弈） 从新节点0/0的状态开始，与对手模拟对弈，直到分出输赢，获得一个准确的回报。假设棋输了，所以得到的是 0/1。仿真模拟的过程，可采取双方随机走子的策略，或阿尔法GO中的策略网络和价值网络，以及快速走子策略，这可大大加快双方对弈的过程。

（4）反向传播

从0/0开始向上，用得到的 0/1更新从它到根节点的路径的所有节点中的数据。注意，其它节点不更新。见上图第4个树，从0/0 节点开始向上遍历父节点，直到根节点，这条路径上的每个节点都增加一个 0/1。

至此，此轮迭代结束。如果整个迭代结束，这步棋的最优走法是取第二层访问次数最多者。